



User Guide

Amazon Aurora DSQL



Amazon Aurora DSQL: User Guide

Copyright © 2025 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

What is Amazon Aurora DSQL?	1
When to use	1
Key features	1
AWS Region availability	2
Multi-Region clusters	4
Pricing	5
What's next?	5
Getting started	6
Prerequisites	6
Accessing Aurora DSQL	7
Console access	7
SQL clients	7
PostgreSQL protocol	11
Create a single-Region cluster	12
Connect to a cluster	13
Run SQL commands	14
Create a multi-Region cluster	16
Authentication and authorization	19
Managing your cluster	19
Connecting to your cluster	19
PostgreSQL and IAM roles	20
Using IAM policy actions with Aurora DSQL	21
Using IAM policy actions to connect to clusters	21
Using IAM policy actions to manage clusters	22
Revoking authorization using IAM and PostgreSQL	23
Generate an authentication token	24
Console	24
AWS CloudShell	25
AWS CLI	26
Aurora DSQL SDKs	27
Database roles and IAM authentication	36
IAM roles	36
IAM users	36
Connect	36

Query	37
View mappings	37
Revoke	38
Aurora DSQL and PostgreSQL	39
Compatibility highlights	39
Key architectural differences	40
SQL compatibility	41
Supported data types	41
Supported SQL features	46
Supported subsets of SQL commands	50
Unsupported PostgreSQL features	61
Concurrency control	65
Transaction conflicts	65
Guidelines for optimizing transaction performance	65
DDL and distributed transactions	66
Primary keys	67
Data structure and storage	67
Guidelines for choosing a primary key	67
Asynchronous indexes	68
Syntax	69
Parameters	69
Usage notes	70
Creating an index	71
Querying an index	71
Unique index build failures	73
Uniqueness violations	73
System tables and commands	75
System tables	75
Useful system queries	84
The ANALYZE command	85
EXPLAIN plans	85
PostgreSQL EXPLAIN plans	86
Key elements	87
Filtering	87
Reading EXPLAIN plans	88
Managing Aurora DSQL clusters	92

Single-Region clusters	92
Using AWS SDKs	92
Using AWS CLI	131
Multi-Region clusters	134
Using AWS SDKs	134
Using AWS CLI	188
AWS CloudFormation	195
Initial configuration	195
Find clusters	195
Update configuration	196
Programming with Aurora DSQL	197
.....	197
Database Connectivity Tools	198
Database Drivers	198
ORM Libraries	199
Aurora DSQL Adapters	200
Connectors	200
JDBC connector	201
Backup and restore	206
Getting started with AWS Backup	206
Restoring your backups	206
Restoring single-Region clusters	207
Restoring multi-Region clusters	207
Monitoring and compliance	207
Additional resources	208
Monitoring and logging	209
Viewing cluster status	209
Cluster statuses	209
Viewing cluster statuses	210
Monitoring with CloudWatch	211
Observability	212
Usage	213
Logging with CloudTrail	214
Management events	215
Data events	216
Security	218

AWS managed policies	219
AmazonAuroraDSQLEFullAccess	219
AmazonAuroraDSQLReadOnlyAccess	220
AmazonAuroraDSQLConsoleFullAccess	221
AuroraDSQLServiceRolePolicy	223
Policy updates	223
Data protection	229
Data encryption	230
Data Protection in witness Regions	231
SSL/TLS certificates	231
Data encryption	230
KMS key types	238
Encryption at rest	238
Using KMS and data keys	240
Authorizing your KMS key	242
Encryption context	244
Monitoring AWS KMS	244
Creating an encrypted cluster	247
Removing or updating a key	249
Considerations	251
Identity and access management	252
Audience	253
Authenticating with identities	253
Managing access using policies	254
How Aurora DSQL works with IAM	256
Identity-based policy examples	261
Troubleshooting	264
Resource-based policies	265
When to Use	266
Create with policies	267
Add and edit policies	270
View Policy	273
Remove Policy	274
Policy examples	275
Block public access	280
API Operations	283

Using a service-linked role	285
Service-linked role permissions for Aurora DSQL	286
Create a service-linked role	286
Edit a service-linked role	287
Delete a service-linked role	287
Supported Regions for Aurora DSQL service-linked roles	287
Using IAM condition keys	287
Create a cluster in a specific Region	287
Create a multi-Region cluster in specific Regions	288
Create a multi-Region cluster with specific witness Region	289
Incident response	290
Compliance validation	291
Resilience	291
Backup and restore	291
Replication	292
High availability	292
Fault injection testing	293
Infrastructure Security	293
Managing clusters using AWS PrivateLink	294
Configuration and vulnerability analysis	303
Cross-service confused deputy prevention	303
Security best practices	305
Detective security best practices	305
Preventative security best practices	306
Tagging resources	308
Name tag	308
Tagging requirements	308
Tagging usage notes	309
Considerations	310
Quotas and limits	311
Cluster quotas	311
Database limits	312
API reference	316
Troubleshooting	317
Connection errors	317
Authentication errors	318

Authorization errors	318
SQL errors	319
OCC errors	319
SSL/TLS connections	320
Document history	321

What is Amazon Aurora DSQL?

Amazon Aurora DSQL is a serverless, distributed relational database service optimized for transactional workloads. Aurora DSQL offers virtually unlimited scale and doesn't require you to manage infrastructure. The active-active highly available architecture provides 99.99% single-Region and 99.999% multi-Region availability.

When to use Aurora DSQL

Aurora DSQL is optimized for transactional workloads that benefit from ACID transactions and a relational data model. Because it's serverless, Aurora DSQL is ideal for application patterns of microservice, serverless, and event-driven architectures. Aurora DSQL is PostgreSQL-compatible, so you can use familiar drivers, object-relational mappings (ORMs), frameworks, and SQL features.

Aurora DSQL automatically manages system infrastructure and scales compute, I/O, and storage based on your workload. Because you have no servers to provision or manage, you don't have to worry about maintenance downtime related to provisioning, patching, or infrastructure upgrades.

Aurora DSQL helps you to build and maintain enterprise applications that are always available at any scale. The active-active serverless design automates failure recovery, so you don't need to worry about traditional database failover. Your applications benefit from Multi-AZ and multi-Region availability, and you don't have to be concerned about eventual consistency or missing data related to failovers.

Key features in Aurora DSQL

The following key features help you create a serverless distributed database to support your high-availability applications:

Distributed architecture

Aurora DSQL is composed of the following multi-tenant components:

- Relay and connectivity
- Compute and databases
- Transaction log, concurrency control, and isolation
- Storage

A control plane coordinates the preceding components. Each component provide redundancy across three Availability Zones (AZs), with automatic cluster scaling and self-healing in case of component failures. To learn more about how this architecture supports high availability, see [Resilience in Amazon Aurora DSQL](#).

Single-Region and multi-Region clusters

Aurora DSQL clusters provide the following benefits:

- Synchronous data replication
- Consistent read operations
- Automatic failure recovery
- Data consistency across multiple AZs or Regions

If an infrastructure component fails, Aurora DSQL automatically routes requests to healthy infrastructure without manual intervention. Aurora DSQL provides *atomicity, consistency, isolation, and durability (ACID) transactions* with strong consistency, snapshot isolation, atomicity, and cross-AZ and cross-Region durability.

Multi-Region peered clusters provide the same resilience and connectivity as single-Region clusters. But they improve availability by offering two Regional endpoints, one in each peered cluster Region. Both endpoints of a peered cluster present a single logical database. They are available for concurrent read and write operations, and provide strong data consistency. You can build applications that run in multiple Regions at the same time for performance and resilience—and know that readers always see the same data.

Compatibility with PostgreSQL databases

The distributed database layer (compute) in Aurora DSQL is based on a current major version of PostgreSQL. You can connect to Aurora DSQL with familiar PostgreSQL drivers and tools, such as `psql`. Aurora DSQL is currently compatible with PostgreSQL version 16 and supports a subset of PostgreSQL features, expressions, and data types. For more information about the supported SQL features, see [SQL feature compatibility in Aurora DSQL](#).

Region availability for Aurora DSQL

With Amazon Aurora DSQL, you can deploy database instances across multiple AWS Regions to support global applications and meet data residency requirements. Region availability determines where you can create and manage Aurora DSQL database clusters. Database administrators and application architects who need to design highly available, globally distributed database systems

often need to understand Region support for their workloads. Common use cases include setting up cross-Region disaster recovery, serving users from geographically closer database instances to reduce latency, and maintaining data copies in specific locations for compliance.

The following table shows the AWS Regions where Aurora DSQL is currently available and the endpoint for each AWS Region.

Region Name	Region	Endpoint	Protocol	
US East (Ohio)	us-east-2	dsql.us-east-2.api.aws	HTTPS	
		dsql-fips.us-east-2.api.aws	HTTPS	
US East (N. Virginia)	us-east-1	dsql.us-east-1.api.aws	HTTPS	
		dsql-fips.us-east-1.api.aws	HTTPS	
US West (Oregon)	us-west-2	dsql.us-west-2.api.aws	HTTPS	
		dsql-fips.us-west-2.api.aws	HTTPS	
Asia Pacific (Osaka)	ap-northeast-3	dsql.ap-northeast-3.api.aws	HTTPS	
Asia Pacific (Seoul)	ap-northeast-2	dsql.ap-northeast-2.api.aws	HTTPS	
Asia Pacific (Tokyo)	ap-northeast-1	dsql.ap-northeast-1.api.aws	HTTPS	
Europe (Frankfurt)	eu-central-1	dsql.eu-central-1.api.aws	HTTPS	
Europe (Ireland)	eu-west-1	dsql.eu-west-1.api.aws	HTTPS	

Region Name	Region	Endpoint	Protocol	
Europe (London)	eu-west-2	dsql.eu-west-2.api.aws	HTTPS	
Europe (Paris)	eu-west-3	dsql.eu-west-3.api.aws	HTTPS	

Multi-Region cluster availability for Aurora DSQL

You can create Aurora DSQL multi-Region clusters within specific AWS Region sets. Each Region set groups geographically related Regions that can work together in a multi-Region cluster.

US Regions

- US East (N. Virginia)
- US East (Ohio)
- US West (Oregon)

Asia Pacific Regions

- Asia Pacific (Osaka)
- Asia Pacific (Seoul)
- Asia Pacific (Tokyo)

European Regions

- Europe (Frankfurt)
- Europe (Ireland)
- Europe (London)
- Europe (Paris)

Important Limitations

Multi-Region clusters must be created within a single Region set. For example, you can't create a cluster that includes both US East (N. Virginia) and Europe (Ireland) Regions.

Important

Aurora DSQL currently doesn't support cross-continent multi-Region clusters.

Pricing for Aurora DSQL

For cost information, see [Aurora DSQL pricing](#).

What's next?

For information about the core components in Aurora DSQL and to get started with the service, see the following:

- [Getting started with Aurora DSQL](#)
- [SQL feature compatibility in Aurora DSQL](#)
- [Accessing Aurora DSQL](#)
- [Aurora DSQL and PostgreSQL](#)

Getting started with Aurora DSQL

Amazon Aurora DSQL is a serverless, distributed relational database optimized for transactional workloads. In the following sections, you'll learn how to create single-Region and multi-Region Aurora DSQL clusters, connect to them, and create and load a sample schema. You will access clusters with the AWS Management Console and interact with your database using the `psql` utility. By the end, you'll have a working Aurora DSQL cluster set up and ready to use for test or production workloads.

Topics

- [Prerequisites](#)
- [Accessing Aurora DSQL](#)
- [Step 1: Create an Aurora DSQL single-Region cluster](#)
- [Step 2: Connect to your Aurora DSQL cluster](#)
- [Step 3: Run sample SQL commands in Aurora DSQL](#)
- [Step 4: Create a multi-Region cluster](#)

Prerequisites

Before you can begin using Aurora DSQL, make sure you meet the following prerequisites:

- Your IAM identity must have permission to [sign in to the AWS Management Console](#).
- Your IAM identity must meet either of the following criteria:
 - Access to perform any action on any resource in your AWS account
 - The IAM permission `iam:CreateServiceLinkedRole` and the ability to get access to the IAM policy action `dsql:*`
- If you use the AWS CLI in a Unix-like environment, make sure that Python version 3.8+ and `psql` version 14+ are installed. To check your application versions, run the following commands.

```
python3 --version
psql --version
```

If you use the AWS CLI in a different environment, make sure that you manually set up Python version 3.8+ and `psql` version 14+.

- If you intend to access Aurora DSQL using AWS CloudShell, Python version 3.8+ and `psql` version 14+ are provided with no extra setup. For more information about AWS CloudShell, see [What is AWS CloudShell?](#).
- If you intend to access Aurora DSQL using a GUI, use DBeaver or JetBrains DataGrip. For more information, see [Accessing Aurora DSQL with DBeaver](#) and [Accessing Aurora DSQL with JetBrains DataGrip](#).

Accessing Aurora DSQL

You can access Aurora DSQL through the following techniques. To learn how to use the CLI, APIs, and SDKs, see [Accessing Aurora DSQL](#).

Topics

- [Accessing Aurora DSQL through the AWS Management Console](#)
- [Accessing Aurora DSQL using SQL clients](#)
- [Using the PostgreSQL protocol with Aurora DSQL](#)

Accessing Aurora DSQL through the AWS Management Console

You can access the AWS Management Console for Aurora DSQL at <https://console.aws.amazon.com/dsql>.

Accessing Aurora DSQL using SQL clients

Aurora DSQL uses the PostgreSQL protocol. Use your preferred interactive client by providing a signed IAM [authentication token](#) as the password when connecting to your cluster. An authentication token is a unique string of characters that Aurora DSQL generates dynamically using AWS Signature Version 4.

Aurora DSQL uses the token only for authentication. The token doesn't affect the connection after it is established. If you try to reconnect using an expired token, the connection request is denied. For more information, see [Generating an authentication token in Amazon Aurora DSQL](#).

Topics

- [Accessing Aurora DSQL with `psql` \(PostgreSQL interactive terminal\)](#)
- [Accessing Aurora DSQL with DBeaver](#)

- [Accessing Aurora DSQL with JetBrains DataGrip](#)

Accessing Aurora DSQL with psql (PostgreSQL interactive terminal)

The `psql` utility is a terminal-based front-end to PostgreSQL. It enables you to type in queries interactively, issue them to PostgreSQL, and see the query results. To improve query response times, use the PostgreSQL version 17 client.

Download your operating system's installer from the [PostgreSQL Downloads](#) page. For more information about `psql`, see <https://www.postgresql.org/docs/current/app-psql.htm>.

If you already have the AWS CLI installed, use the following example to connect to your cluster. You can either use AWS CloudShell, which comes with `psql` preinstalled, or you can install `psql` directly.

```
# Aurora DSQL requires a valid IAM token as the password when connecting.
# Aurora DSQL provides tools for this and here we're using Python.
export PGPASSWORD=$(aws dsq1 generate-db-connect-admin-auth-token \
  --region us-east-1 \
  --expires-in 3600 \
  --hostname your_cluster_endpoint)

# Aurora DSQL requires SSL and will reject your connection without it.
export PGSSLMODE=require

# Connect with psql, which automatically uses the values set in PGPASSWORD and
PGSSLMODE.
# Quiet mode suppresses unnecessary warnings and chatty responses but still outputs
errors.
psql --quiet \
  --username admin \
  --dbname postgres \
  --host your_cluster_endpoint
```

Accessing Aurora DSQL with DBeaver

DBeaver is an open-source, GUI-based database tool. You can use it to connect to and manage your database. To download DBeaver, see the [download page](#) on the DBeaver Community website. The following steps explain how to connect to your cluster using DBeaver.

To set up a new Aurora DSQL connection in DBeaver

1. Choose **New Database Connection**.
2. In the **New Database Connection** window, choose PostgreSQL.
3. In the **Connection settings/Main** tab, choose **Connect by: Host** and enter the following information.
 - **Host** – Use your cluster endpoint.
 - Database** – Enter postgres
 - Authentication** – Choose Database Native
 - Username** – Enter admin
 - Password** – Generate an [authentication token](#). Copy the generated token and use it as your password.
4. Ignore any warnings and paste your authentication token into the **DBeaver Password** field.

Note

You must set SSL mode in the client connections. Aurora DSQL supports SSLMODE=require. Aurora DSQL enforces SSL communication on the server side and rejects non-SSL connections.

5. You should be connected to your cluster and can start running SQL statements.

Important

The administrative features provided by DBeaver for the PostgreSQL databases (such as **Session Manager** and **Lock Manager**) don't apply to a database, due to its unique architecture. While accessible, these screens don't provide reliable information on the database health or status.

Authentication credentials expiry for DBeaver

Established sessions remain authenticated for a maximum of 1 hour or until DBeaver disconnects or times out. To establish new connections, provide a valid authentication token in the **Password**

field of **Connection settings**. Trying to open a new session (for example, to list new tables, or a new SQL console) forces a new authentication attempt. If the authentication token configured in the **Connection settings** is no longer valid, the new session fails, and DBeaver invalidates all previously opened sessions. Keep this in mind when choosing the duration of your IAM authentication token with the `expires-in` option.

Accessing Aurora DSQL with JetBrains DataGrip

JetBrains DataGrip is a cross-platform IDE for working with SQL and databases, including PostgreSQL. DataGrip includes a robust GUI with an intelligent SQL editor. To download DataGrip, go to the [download page](#) on the JetBrains website.

To set up a new Aurora DSQL connection in JetBrains DataGrip

1. Choose **New Data Source** and choose PostgreSQL.
2. In the Data Sources/General tab, enter the following information:
 - **Host** – Use your cluster endpoint.
 - Port** – Aurora DSQL uses the PostgreSQL default: 5432
 - Database** – Aurora DSQL uses the PostgreSQL default of postgres
 - Authentication** – Choose User & Password .
 - Username** – Enter admin.
 - Password** – [Generate a token](#) and paste it into this field.
 - URL** – Don't modify this field. It will be auto-populated based on the other fields.
3. **Password** – Provide this by generating an authentication token. Copy the resulting output of the token generator and paste it into the password field.

Note

You must set SSL mode in the client connections. Aurora DSQL supports PGSSLMODE=require. Aurora DSQL enforces SSL communication on the server side and will reject non-SSL connections.

4. You should be connected to your cluster and can start running SQL statements:

⚠ Important

Some views provided by DataGrip for the PostgreSQL databases (such as Sessions) don't apply to a database because of its unique architecture. While accessible, these screens don't provide reliable information on the actual sessions connected to the database.

Authentication credentials expiration

Established sessions remain authenticated for a maximum of 1 hour or until an explicit disconnect or a client-side timeout takes place. If new connections need to be established, a new Authentication token must be generated and provided in the **Password** field of the **Data Source Properties**. Trying to open a new session (for example, to list new tables, or a new SQL console) forces a new authentication attempt. If the authentication token configured in the **Connection** settings is no longer valid, that new session will fail and all previously opened sessions will become invalid.

Using the PostgreSQL protocol with Aurora DSQL

PostgreSQL uses a message-based protocol for communication between clients and servers. The protocol is supported over TCP/IP and also over Unix-domain sockets. The following table shows how Aurora DSQL supports the [PostgreSQL protocol](#).

PostgreSQL	Aurora DSQL	Notes
Role (also known as User or Group)	Database Role	Aurora DSQL creates a role for you named <code>admin</code> . When you create custom database roles, you must use the <code>admin</code> role to associate them with IAM roles for authenticating when connecting to your cluster. For more information, see Configure custom database roles .
Host (also known as hostname or hostspec)	Cluster Endpoint	Aurora DSQL single-Region clusters provide a single managed endpoint and automatically redirect traffic if there is unavailability within the Region.

PostgreSQL	Aurora DSQL	Notes
Port	N/A – use default 5432	This is the PostgreSQL default.
Database (dbname)	use postgres	Aurora DSQL creates this database for you when you create the cluster.
SSL Mode	SSL is always enabled server-side	In Aurora DSQL, Aurora DSQL supports the require SSL Mode. Connections without SSL are rejected by Aurora DSQL.
Password	Authentication Token	Aurora DSQL requires temporary authentication tokens instead of long-lived passwords. To learn more, see Generating an authentication token in Amazon Aurora DSQL .

Step 1: Create an Aurora DSQL single-Region cluster

The basic unit of Aurora DSQL is the cluster, which is where you store your data. In this task, you create a cluster in a single AWS Region.

To create a single-Region cluster in Aurora DSQL

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql>.
2. Choose **Create cluster** and then **Single-Region**.
3. (Optional) In **Cluster settings**, select any of the following options:
 - Select **Customize encryption settings (advanced)** to choose or create an AWS KMS key.
 - Select **Enable deletion protection** to prevent a delete operation from removing your cluster. By default, deletion protection is selected.
4. (Optional) In **Tags**, choose or enter a tag for this cluster.
5. Choose **Create cluster**.

Step 2: Connect to your Aurora DSQL cluster

A *cluster endpoint* is automatically generated when you create an Aurora DSQL cluster based on its cluster ID and Region. The naming format is *clusterid*.*dsq1*.*region*.on.aws. A client uses the endpoint to create a network connection to your cluster.

Authentication is managed using IAM so you don't need to store credentials in the database. An *authentication token* is a unique string of characters that is generated dynamically. The token is only used for authentication and doesn't affect the connection after it is established. Before attempting to connect, make sure that your IAM identity has the `dsq1:DbConnectAdmin` permission, as described in [Prerequisites](#).

Note

To optimize database connection speed, use the PostgreSQL version 17 client and set `PGSSLNEGOTIATION` to `direct`: `PGSSLNEGOTIATION=direct`.

To connect to your cluster with an authentication token

1. In the Aurora DSQL console, choose the cluster that you want to connect to.
2. Choose **Connect**.
3. Copy the endpoint from **Endpoint (Host)**.
4. Make sure that you **Connect as admin** is chosen in the **Authentication token (Password)** section.
5. Copy the generated authentication token. This token is valid for 15 minutes.
6. On the operating system command line, use the following command to start `psql` and connect to your cluster. Replace *your_cluster_endpoint* with the cluster endpoint that you copied previously.

```
PGSSLMODE=require \  
psql --dbname postgres \  
--username admin \  
--host your_cluster_endpoint
```

When prompted for a password, enter the authentication token that you copied previously. If you try to reconnect using an expired token, the connection request is denied. For more information, see [Generating an authentication token in Amazon Aurora DSQL](#).

7. Press **Enter**. You should see a PostgreSQL prompt.

```
postgres=>
```

If you get an access denied error, make sure that your IAM identity has the `dsql:DbConnectAdmin` permission. If you have the permission and continue to get access deny errors, see [Troubleshoot IAM](#) and [How can I troubleshoot access denied or unauthorized operation errors with an IAM policy?](#).

Step 3: Run sample SQL commands in Aurora DSQL

Test your Aurora DSQL cluster by running SQL statements. The following sample statements require the data files named `department-insert-multirow.sql` and `invoice.csv`, which you can download from the [aws-samples/aurora-dsql-samples](#) repository on GitHub.

To run sample SQL commands in Aurora DSQL

1. Create a schema named `example`.

```
CREATE SCHEMA example;
```

2. Create an invoice table that uses an automatically generated UUID as the primary key.

```
CREATE TABLE example.invoice(  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    created timestamp,  
    purchaser int,  
    amount float);
```

3. Create a secondary index that uses the empty table.

```
CREATE INDEX ASYNC invoice_created_idx on example.invoice(created);
```

4. Create a department table.

```
CREATE TABLE example.department(id INT PRIMARY KEY UNIQUE, name text, email text);
```

5. Use the command `psql \include` to load the file named `department-insert-multirow.sql` that you downloaded from the [aws-samples/aurora-dsql-samples](https://aws-samples.github.io/aurora-dsql-samples/) repository on GitHub. Replace *my-path* with the path to your local copy.

```
\include my-path/department-insert-multirow.sql
```

6. Use the command `psql \copy` to load the file named `invoice.csv` that you downloaded from the [aws-samples/aurora-dsql-samples](https://aws-samples.github.io/aurora-dsql-samples/) repository on GitHub. Replace *my-path* with the path to your local copy.

```
\copy example.invoice(created, purchaser, amount) from my-path/invoice.csv csv
```

7. Query the departments and sort them by their total sales.

```
SELECT name, sum(amount) AS sum_amount
FROM example.department LEFT JOIN example.invoice ON
  department.id=invoice.purchaser
GROUP BY name
HAVING sum(amount) > 0
ORDER BY sum_amount DESC;
```

The following sample output shows that Department One has the most sales.

name	sum_amount
Example Department One	32628.75608634601
Example Department Three	32427.43955110429
Example Department Eight	32256.810987098102
Example Department Five	31391.14891163639
Example Department Seven	31253.236846746757
Example Department Six	29699.06014910414
Example Department Two	29465.58360076501
Example Department Four	28764.19185819191

(8 rows)

Step 4: Create a multi-Region cluster

When you create a multi-Region cluster, you specify the following Regions:

Remote Region

This is the Region in which you create a second cluster. You create a second cluster in this Region and peer it to your initial cluster. Aurora DSQL replicates all writes on the initial cluster to the remote cluster. You can read and write on any cluster.

Witness Region

This Region receives all data that is written to the multi-Region cluster. However, witness Regions don't host client endpoints and don't provide user data access. A limited window of the encrypted transaction log is maintained in witness Regions. This log facilitates recovery and supports transactional quorum if a Region becomes unavailable.

The following example shows how to create an initial cluster, create a second cluster in a different Region, and then peer the two clusters to create a multi-Region cluster. It also demonstrates cross-Region write replication and consistent reads from both Regional endpoints.

To create a multi-Region cluster

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql>.
2. In the navigation pane, choose **Clusters**.
3. Choose **Create cluster** and then **Multi-Region**.
4. (Optional) In **Cluster settings**, select any of the following options for your initial cluster:
 - Select **Customize encryption settings (advanced)** to choose or create an AWS KMS key.
 - Select **Enable deletion protection** to prevent a delete operation from removing your cluster. By default, deletion protection is selected.
5. In **Multi-Region settings**, choose the following options for your initial cluster:
 - In **Witness Region**, choose a Region. Currently, only US-based Regions are supported for witness Regions in multi-Region clusters.

- (Optional) In **Remote Region cluster ARN**, enter an ARN for an existing cluster in another Region. If no cluster exists to serve as the second cluster in your multi-Region cluster, complete setup after you create the initial cluster.
6. (Optional) Choose tags for your initial cluster.
7. Choose **Create cluster** to create your initial cluster. If you didn't enter an ARN in the previous step, the console shows the **Cluster setup pending** notification.
8. In the **Cluster setup pending** notification, choose **Complete multi-Region cluster setup**. This action initiates creation of a second cluster in another Region.
9. Choose one of the following options for your second cluster:
 - **Add remote Region cluster ARN** – Choose this option if a cluster exists, and you want it to be the second cluster in your multi-Region cluster.
 - **Create cluster in another Region** – Choose this option to create a second cluster. In **Remote Region**, choose the Region for this second cluster.
10. Choose **Create cluster in *your-second-region***, where *your-second-region* is the location of your second cluster. The console opens in your second Region.
11. (Optional) Choose cluster settings for your second cluster. For example, you can choose an AWS KMS key.
12. Choose **Create cluster** to create your second cluster.
13. Choose **Peer in *initial-cluster-region***, where *initial-cluster-region* is the Region that hosts the first cluster that you created.
14. When prompted, choose **Confirm**. This step completes the creation of your multi-Region cluster.

To connect to your second cluster

1. Open the Aurora DSQL console and choose the Region for your second cluster.
2. Choose **Clusters**.
3. Select the row for the second cluster in your multi-Region cluster.
4. In **Actions**, choose **Open in CloudShell**.
5. Choose **Connect as admin**.
6. Choose **Launch CloudShell**.
7. Choose **Run**.

8. Create a sample schema by following the steps in [Step 3: Run sample SQL commands in Aurora DSQL](#).

Example transactions

Example

```
CREATE SCHEMA example;
CREATE TABLE example.invoice(id UUID PRIMARY KEY DEFAULT gen_random_uuid(), created
timestamp, purchaser int, amount float);
CREATE INDEX ASYNC invoice_created_idx on example.invoice(created);
CREATE TABLE example.department(id INT PRIMARY KEY UNIQUE, name text, email text);
```

9. Use psql copy and include commands to load sample data. For more information, see [Step 3: Run sample SQL commands in Aurora DSQL](#).

```
\copy example.invoice(created, purchaser, amount) from samples/invoice.csv csv
\include samples/department-insert-multirow.sql
```

To query data in the second cluster from the Region hosting your initial cluster

1. In the Aurora DSQL console, choose the Region for your initial cluster.
2. Choose **Clusters**.
3. Select the row for the second cluster in your multi-Region cluster.
4. In **Actions**, choose **Open in CloudShell**.
5. Choose **Connect as admin**.
6. Choose **Launch CloudShell**.
7. Choose **Run**.
8. Query the data that you inserted into the second cluster.

Example

```
SELECT name, sum(amount) AS sum_amount
FROM example.department
LEFT JOIN example.invoice ON department.id=invoice.purchaser
GROUP BY name
HAVING sum(amount) > 0
ORDER BY sum_amount DESC;
```

Authentication and authorization for Aurora DSQL

Aurora DSQL uses IAM roles and policies for cluster authorization. You associate IAM roles with [PostgreSQL database roles](#) for database authorization. This approach combines [benefits from IAM](#) with [PostgreSQL privileges](#). Aurora DSQL uses these features to provide a comprehensive authorization and access policy for your cluster, database, and data.

Managing your cluster using IAM

To manage your cluster, use IAM for authentication and authorization:

IAM authentication

To authenticate your IAM identity when you manage Aurora DSQL clusters, you must use IAM. You can provide authentication using the [AWS Management Console](#), [AWS CLI](#), or the [AWS SDK](#).

IAM authorization

To manage Aurora DSQL clusters, grant authorization using IAM actions for Aurora DSQL. For example, to describe a cluster, make sure that your IAM identity has permissions for the IAM action `dsql:GetCluster`, as in the following sample policy action.

```
{
  "Effect": "Allow",
  "Action": "dsql:GetCluster",
  "Resource": "arn:aws:dsql:us-east-1:123456789012:cluster/my-cluster"
}
```

For more information, see [Using IAM policy actions to manage clusters](#).

Connecting to your cluster using IAM

To connect to your cluster, use IAM for authentication and authorization:

IAM authentication

Generate a temporary authentication token using an IAM identity with authorization to connect to your cluster. To learn more, see [Generating an authentication token in Amazon Aurora DSQL](#).

IAM authorization

Grant the following IAM policy actions to the IAM identity you're using to establish the connection to your cluster's endpoint:

- Use `dsql:DbConnectAdmin` if you're using the admin role. Aurora DSQL creates and manages this role for you. The following sample IAM policy action permits admin to connect to *my-cluster*.

```
{
  "Effect": "Allow",
  "Action": "dsql:DbConnectAdmin",
  "Resource": "arn:aws:dsql:us-east-1:123456789012:cluster/my-cluster"
}
```

- Use `dsql:DbConnect` if you're using a custom database role. You create and manage this role by using SQL commands in your database. The following sample IAM policy action permits a custom database role to connect to *my-cluster* for up to one hour.

```
{
  "Effect": "Allow",
  "Action": "dsql:DbConnect",
  "Resource": "arn:aws:dsql:us-east-1:123456789012:cluster/my-cluster"
}
```

After you establish a connection, your role is authorized for up to one hour for the connection.

Interacting with your database using PostgreSQL database roles and IAM roles

PostgreSQL manages database access permissions using the concept of roles. A role can be thought of as either a database user, or a group of database users, depending on how the role is set up. You create PostgreSQL roles using SQL commands. To manage database-level authorization, grant PostgreSQL permissions to your PostgreSQL database roles.

Aurora DSQL supports two types of database roles: an admin role and custom roles. Aurora DSQL automatically creates a predefined admin role for you in your Aurora DSQL cluster. You can't modify the admin role. When you connect to your database as admin, you can issue SQL to

create new database-level roles to associate with your IAM roles. To let IAM roles connect to your database, associate your custom database roles with your IAM roles.

Authentication

Use the `admin` role to connect to your cluster. After you connect your database, use the command `AWS IAM GRANT` to associate a custom database role with the IAM identity authorized to connect to the cluster, as in the following example.

```
AWS IAM GRANT custom-db-role TO 'arn:aws:iam::account-id:role/iam-role-name';
```

To learn more, see [Authorizing database roles to connect to your cluster](#).

Authorization

Use the `admin` role to connect to your cluster. Run SQL commands to set up custom database roles and grant permissions. To learn more, see [PostgreSQL database roles](#) and [PostgreSQL privileges](#) in the PostgreSQL documentation.

Using IAM policy actions with Aurora DSQL

The IAM policy action you use depends on the role you use to connect to your cluster: either `admin` or a custom database role. The policy also depends on the IAM actions required for this role.

Using IAM policy actions to connect to clusters

When you connect to your cluster with the default database role of `admin`, use an IAM identity with authorization to perform the following IAM policy action.

```
"dsql:DbConnectAdmin"
```

When you connect to your cluster with a custom database role, first associate the IAM role with the database role. The IAM identity you use to connect to your cluster must have authorization to perform the following IAM policy action.

```
"dsql:DbConnect"
```

To learn more about custom database roles, see [Using database roles and IAM authentication](#).

Using IAM policy actions to manage clusters

When managing your Aurora DSQL clusters, specify policy actions only for the actions that your role needs to perform. For example, if your role only needs to get cluster information, you might limit role permissions to only the `GetCluster` and `ListClusters` permissions, as in the following sample policy

JSON

```
{
  "Version": "2012-10-17",
  "Statement" : [
    {
      "Effect" : "Allow",
      "Action" : [
        "dsql:GetCluster",
        "dsql:ListClusters"
      ],
      "Resource": "arn:aws:dsql:us-east-1:123456789012:cluster/my-cluster"
    }
  ]
}
```

The following example policy shows all available IAM policy actions for managing clusters.

JSON

```
{
  "Version": "2012-10-17",
  "Statement" : [
    {
      "Effect" : "Allow",
      "Action" : [
        "dsql:CreateCluster",
        "dsql:GetCluster",
        "dsql:UpdateCluster",
        "dsql>DeleteCluster",
        "dsql:ListClusters",
        "dsql:TagResource",
        "dsql:ListTagsForResource",

```

```
        "dsql:UntagResource"
      ],
      "Resource" : "*"
    }
  ]
}
```

Revoking authorization using IAM and PostgreSQL

You can revoke permissions for your IAM roles to access your database-level roles:

Revoking admin authorization to connect to clusters

To revoke authorization to connect to your cluster with the `admin` role, revoke the IAM identity's access to `dsql:DbConnectAdmin`. Either edit the IAM policy or detach the policy from the identity.

After revoking connection authorization from the IAM identity, Aurora DSQL rejects all new connection attempts from that IAM identity. Any active connections that use the IAM identity might stay authorized for the duration of the connection. For more information on connection durations, see [Quotas and limits](#).

Revoking custom role authorization to connect to clusters

To revoke access to database roles other than `admin`, revoke the IAM identity's access to `dsql:DbConnect`. Either edit the IAM policy or detach the policy from the identity.

You can also remove the association between the database role and IAM by using the command `AWS IAM REVOKE` in your database. To learn more about revoking access from database roles, see [Revoking database authorization from an IAM role](#).

You can't manage permissions of the predefined `admin` database role. To learn how to manage permissions for custom database roles, see [PostgreSQL privileges](#). Modifications to privileges take effect on the next transaction after Aurora DSQL successfully commits the modification transaction.

Generating an authentication token in Amazon Aurora DSQL

To connect to Amazon Aurora DSQL with a SQL client, generate an authentication token to use as the password. This token is used only for authenticating the connection. After the connection is established, the connection remains valid even if the authentication token expires.

If you create an authentication token using the AWS console, the token automatically expires in one hour by default. If you use the AWS CLI or SDKs to create the token, the default is 15 minutes. The maximum duration is 604,800 seconds, which is one week. To connect to Aurora DSQL from your client again, you can use the same authentication token if it hasn't expired, or you can generate a new one.

To get started with generating a token, [create an IAM policy](#) and [a cluster in Aurora DSQL](#). Then use the AWS console, AWS CLI, or the AWS SDKs to generate a token.

At a minimum, you must have the IAM permissions listed in [Connecting to your cluster using IAM](#), depending on which database role you use to connect.

Topics

- [Use the AWS console to generate an authentication token in Aurora DSQL](#)
- [Use AWS CloudShell to generate an authentication token in Aurora DSQL](#)
- [Use the AWS CLI to generate an authentication token in Aurora DSQL](#)
- [Use the SDKs to generate a token in Aurora DSQL](#)

Use the AWS console to generate an authentication token in Aurora DSQL

Aurora DSQL authenticates users with a token rather than a password. You can generate the token from the console.

To generate an authentication token

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql>.
2. Choose the cluster ID of the cluster for which you want to generate an authentication token. If you haven't yet created a cluster, follow the steps in [Step 1: Create an Aurora DSQL single-Region cluster](#) or [Step 4: Create a multi-Region cluster](#).

3. Choose **Connect** and then select **Get Token**.
4. Choose whether you want to connect as an admin or with a [custom database role](#).
5. Copy the generated authentication token and use it for [Accessing Aurora DSQL using SQL clients](#).

To learn more about custom database roles and IAM in Aurora DSQL, see [Authentication and authorization](#).

Use AWS CloudShell to generate an authentication token in Aurora DSQL

Before you can generate an authentication token using AWS CloudShell, make sure that you [Create an Aurora DSQL cluster](#).

To generate an authentication token using AWS CloudShell

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql>.
2. At the bottom left of the AWS console, choose AWS CloudShell.
3. Run the following command to generate an authentication token for the admin role. Replace *us-east-1* with your Region and *your_cluster_endpoint* with the endpoint of your own cluster.

Note

If you're not connecting as admin, use `generate-db-connect-auth-token` instead.

```
aws dsql generate-db-connect-admin-auth-token \  
--expires-in 3600 \  
--region us-east-1 \  
--hostname your_cluster_endpoint
```

If you run into issues, see [Troubleshoot IAM](#) and [How can I troubleshoot access denied or unauthorized operation errors with an IAM policy?](#)

4. Use the following command to use `psql` to start a connection to your cluster.

```
PGSSLMODE=require \  
psql --dbname postgres \  
    --username admin \  
    --host cluster_endpoint
```

5. You should see a prompt to provide a password. Copy the token that you generated, and make sure you don't include any additional spaces or characters. Paste it into the following prompt from psql.

```
Password for user admin:
```

6. Press **Enter**. You should see a PostgreSQL prompt.

```
postgres=>
```

If you get an access denied error, make sure that your IAM identity has the `dsql:DbConnectAdmin` permission. If you have the permission and continue to get access deny errors, see [Troubleshoot IAM](#) and [How can I troubleshoot access denied or unauthorized operation errors with an IAM policy?](#)

To learn more about custom database roles and IAM in Aurora DSQL, see [Authentication and authorization](#).

Use the AWS CLI to generate an authentication token in Aurora DSQL

When your cluster is **ACTIVE**, you can generate an authentication token on the CLI by using the `aws dsq1` command. Use either of the following techniques:

- If you are connecting with the admin role, use the `generate-db-connect-admin-auth-token` option.
- If you are connecting with a custom database role, use the `generate-db-connect-auth-token` option.

The following example uses the following attributes to generate an authentication token for the admin role.

- *your_cluster_endpoint* – The endpoint of the cluster. It follows the format *your_cluster_identifier*.dsql.*region*.on.aws, as in the example 01abc2ldefg3hijklmnopqrstu.dsql.us-east-1.on.aws.
- *region* – The AWS Region, such as us-east-2 or us-east-1.

The following examples set the expiration time for the token to expire in 3600 seconds (1 hour).

Linux and macOS

```
aws dsql generate-db-connect-admin-auth-token \  
  --region region \  
  --expires-in 3600 \  
  --hostname your_cluster_endpoint
```

Windows

```
aws dsql generate-db-connect-admin-auth-token ^  
  --region=region ^  
  --expires-in=3600 ^  
  --hostname=your_cluster_endpoint
```

Use the SDKs to generate a token in Aurora DSQL

You can generate an authentication token for your cluster when it is in ACTIVE status. The SDK examples use the following attributes to generate an authentication token for the admin role:

- *your_cluster_endpoint* (or *yourClusterEndpoint*) – The endpoint of your Aurora DSQL cluster. The naming format is *your_cluster_identifier*.dsql.*region*.on.aws, as in the example 01abc2ldefg3hijklmnopqrstu.dsql.us-east-1.on.aws.
- *region* (or *RegionEndpoint*) – The AWS Region in which your cluster is located, such as us-east-2 or us-east-1.

Python SDK

You can generate the token in the following ways:

- If you're connecting with the admin role, use `generate_db_connect_admin_auth_token`.

- If you're connecting with a custom database role, use `generate_connect_auth_token`.

```
def generate_token(your_cluster_endpoint, region):
    client = boto3.client("dsql", region_name=region)
    # use `generate_db_connect_auth_token` instead if you are not connecting as
    admin.
    token = client.generate_db_connect_admin_auth_token(your_cluster_endpoint,
region)
    print(token)
    return token
```

C++ SDK

You can generate the token in the following ways:

- If you're connecting with the admin role, use `GenerateDBConnectAdminAuthToken`.
- If you're connecting with a custom database role, use `GenerateDBConnectAuthToken`.

```

#include <aws/core/Aws.h>
#include <aws/dsql/DSQLClient.h>
#include <iostream>

using namespace Aws;
using namespace Aws::DSQL;

std::string generateToken(String yourClusterEndpoint, String region) {
    Aws::SDKOptions options;
    Aws::InitAPI(options);
    DSQLClientConfiguration clientConfig;
    clientConfig.region = region;
    DSQLClient client{clientConfig};
    std::string token = "";

    // If you are not using the admin role to connect, use
    GenerateDBConnectAuthToken instead
    const auto presignedString =
    client.GenerateDBConnectAdminAuthToken(yourClusterEndpoint, region);
    if (presignedString.IsSuccess()) {
        token = presignedString.GetResult();
    } else {
        std::cerr << "Token generation failed." << std::endl;
    }

    std::cout << token << std::endl;

    Aws::ShutdownAPI(options);
    return token;
}

```

JavaScript SDK

You can generate the token in the following ways:

- If you're connecting with the admin role, use `getDbConnectAdminAuthToken`.
- If you're connecting with a custom database role, use `getDbConnectAuthToken`.

```
import { DsqlSigner } from "@aws-sdk/dsql-signer";

async function generateToken(yourClusterEndpoint, region) {
  const signer = new DsqlSigner({
    hostname: yourClusterEndpoint,
    region,
  });
  try {
    // Use `getDbConnectAuthToken` if you are _not_ logging in as the `admin` user
    const token = await signer.getDbConnectAdminAuthToken();
    console.log(token);
    return token;
  } catch (error) {
    console.error("Failed to generate token: ", error);
    throw error;
  }
}
```

Java SDK

You can generate the token in the following ways:

- If you're connecting with the admin role, use `generateDbConnectAdminAuthToken`.
- If you're connecting with a custom database role, use `generateDbConnectAuthToken`.

```

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.services.dsdl.DsdlUtilities;
import software.amazon.awssdk.regions.Region;

public class GenerateAuthToken {
    public static String generateToken(String yourClusterEndpoint, Region region) {
        DsdlUtilities utilities = DsdlUtilities.builder()
            .region(region)
            .credentialsProvider(DefaultCredentialsProvider.create())
            .build();

        // Use `generateDbConnectAuthToken` if you are _not_ logging in as `admin`
        user
        String token = utilities.generateDbConnectAdminAuthToken(builder -> {
            builder.hostname(yourClusterEndpoint)
                .region(region);
        });

        System.out.println(token);
        return token;
    }
}

```

Rust SDK

You can generate the token in the following ways:

- If you're connecting with the admin role, use `db_connect_admin_auth_token`.
- If you're connecting with a custom database role, use `db_connect_auth_token`.

```

use aws_config::{BehaviorVersion, Region};
use aws_sdk_dsql::auth_token::{AuthTokenGenerator, Config};

async fn generate_token(your_cluster_endpoint: String, region: String) -> String {
    let sdk_config = aws_config::load_defaults(BehaviorVersion::latest()).await;
    let signer = AuthTokenGenerator::new(
        Config::builder()
            .hostname(&your_cluster_endpoint)
            .region(Region::new(region))
            .build()
            .unwrap(),
    );

    // Use `db_connect_auth_token` if you are _not_ logging in as `admin` user
    let token = signer.db_connect_admin_auth_token(&sdk_config).await.unwrap();
    println!("{}", token);
    token.to_string()
}

```

Ruby SDK

You can generate the token in the following ways:

- If you're connecting with the admin role, use `generate_db_connect_admin_auth_token`.
- If you're connecting with a custom database role, use `generate_db_connect_auth_token`.

```

require 'aws-sdk-dsql'

def generate_token(your_cluster_endpoint, region)
  credentials = Aws::SharedCredentials.new()

  begin
    token_generator = Aws::DSQL::AuthTokenGenerator.new({
      :credentials => credentials
    })

    # if you're not using admin role, use generate_db_connect_auth_token instead
    token = token_generator.generate_db_connect_admin_auth_token({
      :endpoint => your_cluster_endpoint,
      :region => region
    })
  rescue => error

```



```
    puts error.full_message
  end
end
```

.NET

Note

The official SDK for .NET doesn't include a built-in API call to generate an authentication token for Aurora DSQL. Instead, you must use `DSQLAuthTokenGenerator`, which is a utility class. The following code sample shows how to generate the authentication token for .NET.

You can generate the token in the following ways:

- If you're connecting with the admin role, use `DbConnectAdmin`.
- If you're connecting with a custom database role, use `DbConnect`.

The following example uses the `DSQLAuthTokenGenerator` utility class to generate the authentication token for a user with the admin role. Replace *insert-dsql-cluster-endpoint* with your cluster endpoint.

```
using Amazon;
using Amazon.DSQL.Util;
using Amazon.Runtime;

var yourClusterEndpoint = "insert-dsql-cluster-endpoint";

AWSCredentials credentials = FallbackCredentialsFactory.GetCredentials();

var token = DSQLAuthTokenGenerator.GenerateDbConnectAdminAuthToken(credentials,
    RegionEndpoint.USEast1, yourClusterEndpoint);

Console.WriteLine(token);
```

Golang

Note

The Golang SDK doesn't provide a built-in method for generating a pre-signed token. You must manually construct the signed request, as shown in the following code example.

In the following code example, specify the action based on the PostgreSQL user:

- If you're connecting with the admin role, use the `DbConnectAdmin` action.
- If you're connecting with a custom database role, use the `DbConnect` action.

In addition to *yourClusterEndpoint* and *region*, the following example uses *action*. Specify the *action* based on the PostgreSQL user.

```

func GenerateDbConnectAdminAuthToken(yourClusterEndpoint string, region
string, action string) (string, error) {
// Fetch credentials
sess, err := session.NewSession()
if err != nil {
    return "", err
}

creds, err := sess.Config.Credentials.Get()
if err != nil {
    return "", err
}
staticCredentials := credentials.NewStaticCredentials(
    creds.AccessKeyID,
    creds.SecretAccessKey,
    creds.SessionToken,
)

// The scheme is arbitrary and is only needed because validation of the URL
requires one.
endpoint := "https://" + yourClusterEndpoint
req, err := http.NewRequest("GET", endpoint, nil)
if err != nil {
    return "", err
}
values := req.URL.Query()
values.Set("Action", action)
req.URL.RawQuery = values.Encode()

signer := v4.Signer{
    Credentials: staticCredentials,
}
_, err = signer.Presign(req, nil, "dsql", region, 15*time.Minute, time.Now())
if err != nil {
    return "", err
}

url := req.URL.String()[len("https://"): ]

return url, nil
}

```

Using database roles and IAM authentication

Aurora DSQL supports authentication using both IAM roles and IAM users. You can use either method to authenticate and access Aurora DSQL databases.

IAM roles

An IAM role is an identity within your AWS account that has specific permissions but is not associated with a specific person. Using IAM roles provide temporary security credentials. You can temporarily assume an IAM role in several ways:

- By switching roles in the AWS Management Console
- By calling an AWS CLI or AWS API operation
- By using a custom URL

After assuming a role, you can access Aurora DSQL using the role's temporary credentials. For more information about methods for using roles, see [IAM Identities](#) in the *IAM user guide*.

IAM users

An IAM user is an identity within your AWS account that has specific permissions and is associated with a single person or application. IAM users have long-term credentials such as passwords and access keys that can be used to access Aurora DSQL.

Note

To run SQL commands with IAM authentication, you can use either IAM role ARNs or IAM user ARNs in the examples below.

Authorizing database roles to connect to your cluster

Create an IAM role and grant connection authorization with the IAM policy action: `dsql:DbConnect`.

The IAM policy must also grant permission to access the cluster resources. Use a wildcard (*) or follow the instructions in [Using IAM condition keys with Amazon Aurora DSQL](#).

Authorizing database roles to use SQL in your database

You must use an IAM role with authorization to connect to your cluster.

1. Connect to your Aurora DSQL cluster using a SQL utility.

Use the `admin` database role with an IAM identity that is authorized for IAM action `dsql:DbConnectAdmin` to connect to your cluster.

2. Create a new database role, making sure to specify the `WITH LOGIN` option.

```
CREATE ROLE example WITH LOGIN;
```

3. Associate the database role with the IAM role ARN.

```
AWS IAM GRANT example TO 'arn:aws:iam::012345678912:role/example';
```

4. Grant database-level permissions to the database role

The following examples use the `GRANT` command to provide authorization within the database.

```
GRANT USAGE ON SCHEMA myschema TO example;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA myschema TO example;
```

For more information, see [PostgreSQL GRANT](#) and [PostgreSQL Privileges](#) in the PostgreSQL documentation.

Viewing IAM to database role mappings

To view the mappings between IAM roles and database roles, query the `sys.iam_pg_role_mappings` system table.

```
SELECT * FROM sys.iam_pg_role_mappings;
```

Example output:

iam_oid	arn	pg_role_oid	pg_role_name
grantor_pg_role_oid	grantor_pg_role_name		

-----+-----+-----+-----			
+-----+-----			
26398	arn:aws:iam::012345678912:role/example	26396	example
15579	admin		
(1 row)			

This table shows all the mappings between IAM roles (identified by their ARN) and PostgreSQL database roles.

Revoking database authorization from an IAM role

To revoke database authorization, use the AWS IAM REVOKE operation.

```
AWS IAM REVOKE example FROM 'arn:aws:iam::012345678912:role/example';
```

To learn more about revoking authorization, see [Revoking authorization using IAM and PostgreSQL](#).

Aurora DSQL and PostgreSQL

Aurora DSQL is a PostgreSQL-compatible, distributed relational database designed for transactional workloads. Aurora DSQL uses core PostgreSQL components such as the parser, planner, optimizer, and type system.

The Aurora DSQL design ensures that all supported PostgreSQL syntax provides compatible behavior and yields identical query results. For example, Aurora DSQL provides type conversions, arithmetic operations, and numerical precision and scale that are identical to PostgreSQL. Any deviations are documented.

Aurora DSQL also introduces advanced capabilities such as optimistic concurrency control and distributed schema management. With these features, you can use the familiar tooling of PostgreSQL while benefiting from the performance and scalability required for modern, cloud-native, distributed applications.

PostgreSQL compatibility highlights

Aurora DSQL is currently based on PostgreSQL version 16. Key compatibilities include the following:

Wire protocol

Aurora DSQL uses the standard PostgreSQL v3 wire protocol. This enables integration with standard PostgreSQL clients, drivers, and tools. For example, Aurora DSQL is compatible with `psql`, `pgjdbc`, and `psycopg`.

SQL compatibility

Aurora DSQL supports a wide range of standard PostgreSQL expressions and functions commonly used in transactional workloads. Supported SQL expressions yield identical results to PostgreSQL, including the following:

- Handling of nulls
- Sort order behavior
- Scale and precision for numeric operations
- Equivalence for string operations

For more information, see [SQL feature compatibility in Aurora DSQL](#).

Transaction management

Aurora DSQL preserves the primary characteristics of PostgreSQL, such as ACID transactions and an isolation level equivalent to PostgreSQL Repeatable Read. For more information, see [Concurrency control in Aurora DSQL](#).

Key architectural differences

The distributed, shared-nothing design of Aurora DSQL results in a few foundational differences from traditional PostgreSQL. These differences are integral to the Aurora DSQL architecture and provide many performance and scalability benefits. Key differences include the following:

Optimistic Concurrency Control (OCC)

Aurora DSQL uses an optimistic concurrency control model. This lock-free approach prevents transactions from blocking one another, eliminates deadlocks, and enables high-throughput parallel execution. These features make Aurora DSQL particularly valuable for applications requiring consistent performance at scale. For more example, see [Concurrency control in Aurora DSQL](#).

Asynchronous DDL operations

Aurora DSQL runs DDL operations asynchronously, which allows uninterrupted reads and writes during schema changes. Its distributed architecture allows Aurora DSQL to perform the following actions:

- Run DDL operations as background tasks, minimizing disruption.
- Coordinate catalog changes as strongly consistent distributed transactions. This ensures atomic visibility across all nodes, even during failures or concurrent operations.
- Operate in a fully distributed, leaderless manner across multiple Availability Zones with decoupled compute and storage layers.

For more on using the EXPLAIN command in PostgreSQL, see [DDL and distributed transactions in Aurora DSQL](#).

SQL feature compatibility in Aurora DSQL

Aurora DSQL and PostgreSQL return identical results for all SQL queries. Note that Aurora DSQL differs from PostgreSQL without an `ORDER BY` clause. In the following sections, learn about Aurora DSQL support for PostgreSQL data types and SQL commands.

Topics

- [Supported data types in Aurora DSQL](#)
- [Supported SQL for Aurora DSQL](#)
- [Supported subsets of SQL commands in Aurora DSQL](#)
- [Unsupported PostgreSQL features in Aurora DSQL](#)

Supported data types in Aurora DSQL

Aurora DSQL supports a subset of the common PostgreSQL types.

Topics

- [Numeric data types](#)
- [Character data types](#)
- [Date and time data types](#)
- [Miscellaneous data types](#)
- [Query runtime data types](#)

Numeric data types

Aurora DSQL supports the following PostgreSQL numeric data types.

Name	Aliases	Range and precision	Storage size	Index support
<code>smallint</code>	<code>int2</code>	-32768 to +32767	2 bytes	Yes
<code>integer</code>	<code>int</code> , <code>int4</code>	-2147483648 to +2147483647	4 bytes	Yes

Name	Aliases	Range and precision	Storage size	Index support
bigint	int8	-9223372036854775808 to +9223372036854775807	8 bytes	Yes
real	float4	6 decimal digits precision	4 bytes	Yes
double precision	float8	15 decimal digits precision	8 bytes	Yes
numeric [(p, s)]	decimal [(p, s)] dec[(p, s)]	Exact numeric of selectable precision. The maximum precision is 38 and the maximum scale is 37. ¹ The default is numeric (18,6).	8 bytes + 2 bytes per precision digit. Maximum size is 27 bytes.	No

¹ – If you don't explicitly specify a size when you run CREATE TABLE or ALTER TABLE ADD COLUMN, Aurora DSQL enforces the defaults. Aurora DSQL applies limits when you run INSERT or UPDATE statements.

Character data types

Aurora DSQL supports the following PostgreSQL character data types.

Name	Aliases	Description	Aurora DSQL limit	Storage size	Index support
character [(n)]	char [(n)]	Fixed-length character string	4096 bytes ¹	Variable up to 4100 bytes	Yes
character varying [(n)]	varchar [(n)]	Variable-length character string	65535 bytes ¹	Variable up to 65539 bytes	Yes

Name	Aliases	Description	Aurora DSQL limit	Storage size	Index support
<code>bpchar [(n)]</code>		If fixed length, this is an alias for <code>char</code> . If variable length, this is an alias for <code>varchar</code> , where trailing spaces are semantically insignificant.	4096 bytes ¹	Variable up to 4100 bytes	Yes
<code>text</code>		Variable-length character string	1 MiB ¹	Variable up to 1 MiB	Yes

¹ – If you don't explicitly specify a size when you run `CREATE TABLE` or `ALTER TABLE ADD COLUMN`, then Aurora DSQL enforces the defaults. Aurora DSQL applies limits when you run `INSERT` or `UPDATE` statements.

Date and time data types

Aurora DSQL supports the following PostgreSQL date and time data types.

Name	Aliases	Description	Range	Resolution	Storage size	Index support
<code>date</code>		Calendar date (year, month, day)	4713 BC – 5874897 AD	1 day	4 bytes	Yes
<code>time [(p)] [without time zone]</code>	<code>time</code>	Time of day, with no time zone	0 – 1	1 microsecond	8 bytes	Yes

Name	Aliases	Description	Range	Resolution	Storage size	Index support
<code>time [(p)] with time zone</code>	<code>time</code>	time of day, including time zone	00:00:00+1559 – 24:00:00 –1559	1 microsecond	12 bytes	No
<code>timestamp [(p)] [without time zone]</code>		Date and time, with no time zone	4713 BC – 294276 AD	1 microsecond	8 bytes	Yes
<code>timestamp [(p)] with time zone</code>	<code>timestamp</code>	Date and time, including time zone	4713 BC – 294276 AD	1 microsecond	8 bytes	Yes
<code>interval [fields] [(p)]</code>		Time span	-178000000 years – 178000000 years	1 microsecond	16 bytes	No

Miscellaneous data types

Aurora DSQL supports the following miscellaneous PostgreSQL data types.

Name	Aliases	Description	Aurora DSQL limit	Storage size	Index support
<code>boolean</code>	<code>bool</code>	Logical Boolean (true/false)		1 byte	Yes
<code>bytea</code>		Binary data ("byte array")	1 MiB ¹	Variable up to 1 MiB limit	No

Name	Aliases	Description	Aurora DSQL limit	Storage size	Index support
UUID		Universally unique identifier		16 bytes	Yes

¹ – If you don't explicitly specify a size when you run `CREATE TABLE` or `ALTER TABLE ADD COLUMN`, then Aurora DSQL enforces the defaults. Aurora DSQL applies limits when you run `INSERT` or `UPDATE` statements.

Query runtime data types

Query runtime data types are internal data types used at query execution time. These types are distinct from the PostgreSQL-compatible types like `varchar` and `integer` that you define in your schema. Instead, these types are runtime representations that Aurora DSQL uses when processing a query.

The following data types are supported only during query runtime:

Array type

Aurora DSQL supports arrays of the supported data types. For example, you can have an array of integers. The function `string_to_array` splits a string into a PostgreSQL-style array with the comma delimiter (,) as shown in the following example. You can use arrays in expressions, function outputs, or temporary computations during query execution.

```
SELECT string_to_array('1,2', ',');
```

The function returns a response similar to the following:

```
string_to_array
-----
{1,2}
(1 row)
```

inet type

The data type represents IPv4, IPv6 host addresses, and their subnets. This type is useful when parsing logs, filtering on IP subnets, or doing network calculations within a query. For more information, see [inet in the PostgreSQL documentation](#).

JSON runtime functions

Aurora DSQL supports JSON and JSONB as runtime data types for query processing, though these cannot be used as column data types in table schemas. You can store JSON data as text and cast it to JSON during query execution to use PostgreSQL JSON functions and operators.

Aurora DSQL supports most PostgreSQL JSON functions from [section 9.1.6 JSON Functions and Operators](#) with identical behavior. Unsupported functions include aggregate functions: `json_agg`, `json_agg_strict`, `json_arrayagg`, `json_objectagg`, `json_object_agg`, `json_object_agg_strict`, `json_object_agg_unique`.

Functions that return JSON or JSONB types may require additional casting to text for proper display.

```
SELECT json_build_array(1, 2, 'foo', 4, 5)::text;
```

The function returns a response similar to the following:

```
json_build_array
-----
[1, 2, "foo", 4, 5]
(1 row)
```

Supported SQL for Aurora DSQL

Aurora DSQL supports a wide range of core PostgreSQL SQL features. In the following sections, you can learn about general PostgreSQL expression support. This list is not exhaustive.

SELECT command

Aurora DSQL supports the following clauses of the SELECT command.

Primary clause	Supported clauses
FROM	
GROUP BY	ALL, DISTINCT
ORDER BY	ASC, DESC, NULLS
LIMIT	
DISTINCT	
HAVING	
USING	
WITH (common table expressions)	
INNER JOIN	ON
OUTER JOIN	LEFT, RIGHT, FULL, ON
CROSS JOIN	ON
UNION	ALL
INTERSECT	ALL
EXCEPT	ALL
OVER	RANK (), PARTITION BY
FOR UPDATE	

Data Definition Language (DDL)

Aurora DSQL supports the following PostgreSQL DDL commands.

Command	Primary Clause	Supported Clauses
CREATE	TABLE	For information about the supported syntax of the CREATE TABLE command, see CREATE TABLE .
ALTER	TABLE	For information about the supported syntax of the ALTER TABLE command, see ALTER TABLE .
DROP	TABLE	
CREATE	[UNIQUE] INDEX ASYNC	You can use this command with the following parameters: ON, NULLS FIRST, NULLS LAST. For information about the supported syntax of the CREATE INDEX ASYNC command, see Asynchronous indexes in Aurora DSQL.
DROP	INDEX	
CREATE	VIEW	For more information about the supported syntax of the CREATE VIEW command, see CREATE VIEW .
ALTER	VIEW	For information about the supported syntax of the ALTER VIEW command, see ALTER VIEW .
DROP	VIEW	For information about the supported syntax of the DROP VIEW command, see DROP VIEW .
CREATE	ROLE, WITH	
CREATE	FUNCTION	LANGUAGE SQL

Command	Primary Clause	Supported Clauses
CREATE	DOMAIN	

Data Manipulation Language (DML)

Aurora DSQL supports the following PostgreSQL DML commands.

Command	Primary clause	Supported clauses
INSERT	INTO	VALUES SELECT
UPDATE	SET	WHERE (SELECT) FROM, WITH
DELETE	FROM	USING, WHERE

Data Control Language (DCL)

Aurora DSQL supports the following PostgreSQL DCL commands.

Command	Supported clauses
GRANT	ON, TO
REVOKE	ON, FROM, CASCADE, RESTRICT

Transaction Control Language (TCL)

Aurora DSQL supports the following PostgreSQL TCL commands.

Command	Supported clauses
COMMIT	

Command	Supported clauses
BEGIN	[WORK TRANSACTION] [READ ONLY READ WRITE]

Utility commands

Aurora DSQL supports the following PostgreSQL utility commands:

- EXPLAIN
- ANALYZE (relation name only)

Supported subsets of SQL commands in Aurora DSQL

This PostgreSQL section provides detailed information about supported expressions, focusing on commands with extensive parameter sets and subcommands. For example, CREATE TABLE in PostgreSQL offers many clauses and parameters. This section describes all the PostgreSQL syntax elements that Aurora DSQL supports for these commands.

Topics

- [CREATE TABLE](#)
- [ALTER TABLE](#)
- [CREATE VIEW](#)
- [ALTER VIEW](#)
- [DROP VIEW](#)

CREATE TABLE

CREATE TABLE defines a new table.

```
CREATE TABLE [ IF NOT EXISTS ] table_name ( [
    { column_name data_type [ column_constraint [ ... ] ]
      | table_constraint
      | LIKE source_table [ like_option ... ] }
    [, ... ]
] )
```

where column_constraint is:

```
[ CONSTRAINT constraint_name ]
{ NOT NULL |
  NULL |
  CHECK ( expression )|
  DEFAULT default_expr |
  GENERATED ALWAYS AS ( generation_expr ) STORED |
  UNIQUE [ NULLS [ NOT ] DISTINCT ] index_parameters |
  PRIMARY KEY index_parameters |
```

and table_constraint is:

```
[ CONSTRAINT constraint_name ]
{ CHECK ( expression ) |
  UNIQUE [ NULLS [ NOT ] DISTINCT ] ( column_name [, ... ] ) index_parameters |
  PRIMARY KEY ( column_name [, ... ] ) index_parameters |
```

and like_option is:

```
{ INCLUDING | EXCLUDING } { COMMENTS | CONSTRAINTS | DEFAULTS | GENERATED | IDENTITY |
INDEXES | STATISTICS | ALL }
```

index_parameters in UNIQUE, and PRIMARY KEY constraints are:

```
[ INCLUDE ( column_name [, ... ] ) ]
```

ALTER TABLE

ALTER TABLE changes the definition of a table.

```
ALTER TABLE [ IF EXISTS ] [ ONLY ] name [ * ]
  action [, ... ]
ALTER TABLE [ IF EXISTS ] [ ONLY ] name [ * ]
  RENAME [ COLUMN ] column_name TO new_column_name
ALTER TABLE [ IF EXISTS ] [ ONLY ] name [ * ]
  RENAME CONSTRAINT constraint_name TO new_constraint_name
ALTER TABLE [ IF EXISTS ] name
  RENAME TO new_name
ALTER TABLE [ IF EXISTS ] name
  SET SCHEMA new_schema
```

where action is one of:

```
ADD [ COLUMN ] [ IF NOT EXISTS ] column_name data_type  
OWNER TO { new_owner | CURRENT_ROLE | CURRENT_USER | SESSION_USER }
```

CREATE VIEW

CREATE VIEW defines a new persistent view. Aurora DSQL does not support temporary views; only permanent views are supported.

Supported syntax

```
CREATE [ OR REPLACE ] [ RECURSIVE ] VIEW name [ ( column_name [, ...] ) ]  
[ WITH ( view_option_name [= view_option_value] [, ...] ) ]  
AS query  
[ WITH [ CASCADED | LOCAL ] CHECK OPTION ]
```

Description

CREATE VIEW defines a view of a query. The view is not physically materialized. Instead, the query is run every time the view is referenced in a query.

CREATE or REPLACE VIEW is similar, but if a view of the same name already exists, it is replaced. The new query must generate the same columns that were generated by the existing view query (that is, the same column names in the same order and with the same data types), but it may add additional columns to the end of the list. The calculations giving rise to the output columns may be different.

If a schema name is given, such as CREATE VIEW myschema.myview ...) then the view is created in the specified schema. Otherwise, it is created in the current schema.

The name of the view must be distinct from the name of any other relation (table, index, view) in the same schema.

Parameters

CREATE VIEW supports various parameters to control the behavior of automatically updatable views.

RECURSIVE

Creates a recursive view. The syntax: CREATE RECURSIVE VIEW [schema .] view_name (column_names) AS SELECT ...; is equivalent to CREATE VIEW [schema .]

```
view_name AS WITH RECURSIVE view_name (column_names) AS (SELECT ...)
SELECT column_names FROM view_name;
```

A view column name list must be specified for a recursive view.

name

The name of the view to be created, which may be optionally schema-qualified. A column name list must be specified for a recursive view.

column_name

An optional list of names to be used for columns of the view. If not given, the column names are deduced from the query.

WITH (view_option_name [= view_option_value] [, ...])

This clause specifies optional parameters for a view; the following parameters are supported.

- **check_option** (enum) — This parameter may be either `local` or `cascaded`, and is equivalent to specifying `WITH [ CASCADED | LOCAL ] CHECK OPTION`.
- **security_barrier** (boolean)—This should be used if the view is intended to provide row-level security. Aurora DSQL does not currently support row-level security, but this option will still force the view's `WHERE` conditions (and any conditions using operators which are marked as `LEAKPROOF`) to be evaluated first.
- **security_invoker** (boolean)—This option causes the underlying base relations to be checked against the privileges of the user of the view rather than the view owner. See the notes below for full details.

All of the above options can be changed on existing views using `ALTER VIEW`.

query

A `SELECT` or `VALUES` command which will provide the columns and rows of the view.

- **WITH [CASCADED | LOCAL] CHECK OPTION**— This option controls the behavior of automatically updatable views. When this option is specified, `INSERT` and `UPDATE` commands on the view will be checked to ensure that new rows satisfy the view-defining condition (that is, the new rows are checked to ensure that they are visible through the view). If they are not, the update will be rejected. If the `CHECK OPTION` is not specified, `INSERT` and `UPDATE` commands on the view are allowed to create rows that are not visible through the view. The following check options are supported.

- **LOCAL**—New rows are only checked against the conditions defined directly in the view itself. Any conditions defined on underlying base views are not checked (unless they also specify the `CHECK OPTION`).
- **CASCADED**—New rows are checked against the conditions of the view and all underlying base views. If the `CHECK OPTION` is specified, and neither **LOCAL** nor **CASCADED** are specified, then **CASCADED** is assumed.

Note

The `CHECK OPTION` may not be used with **RECURSIVE** views. The `CHECK OPTION` is only supported on views that are automatically updatable.

Notes

Use the `DROP VIEW` statement to drop views.

The names and data types of the view's columns should be carefully considered. For example, `CREATE VIEW vista AS SELECT 'Hello World';` is not recommended because the column name defaults to `?column?;`. Also, the column data type defaults to `text`, which might not be what you wanted.

A better approach is to explicitly specify the column name and data type, such as: `CREATE VIEW vista AS SELECT text 'Hello World' AS hello;`

By default, access to the underlying base relations referenced in the view is determined by the permissions of the view owner. In some cases, this can be used to provide secure but restricted access to the underlying tables. However, not all views are secure against tampering.

- If the view has the `security_invoker` property set to `true`, access to the underlying base relations is determined by the permissions of the user executing the query, rather than the view owner. Thus, the user of a security invoker view must have the relevant permissions on the view and its underlying base relations.
- If any of the underlying base relations is a security invoker view, it will be treated as if it had been accessed directly from the original query. Thus, a security invoker view will always check its underlying base relations using the permissions of the current user, even if it is accessed from a view without the `security_invoker` property.

- Functions called in the view are treated the same as if they had been called directly from the query using the view. Therefore, the user of a view must have permissions to call all functions used by the view. Functions in the view are executed with the privileges of the user executing the query or the function owner, depending on whether the functions are defined as `SECURITY INVOKER` or `SECURITY DEFINER`. For example, calling `CURRENT_USER` directly in a view will always return the invoking user, not the view owner. This is not affected by the view's `security_invoker` setting, and so a view with `security_invoker` set to `false` is not equivalent to a `SECURITY DEFINER` function.
- The user creating or replacing a view must have `USAGE` privileges on any schemas referred to in the view query, in order to look up the referenced objects in those schemas. Note, however, that this lookup only happens when the view is created or replaced. Therefore, the user of the view only requires the `USAGE` privilege on the schema containing the view, not on the schemas referred to in the view query, even for a security invoker view.
- When `CREATE OR REPLACE VIEW` is used on an existing view, only the view's defining `SELECT` rule, plus any `WITH ( ... )` parameters and its `CHECK OPTION` are changed. Other view properties, including ownership, permissions, and non-`SELECT` rules, remain unchanged. You must own the view to replace it (this includes being a member of the owning role).

Updatable views

Simple views are automatically updatable: the system will allow `INSERT`, `UPDATE`, and `DELETE` statements to be used on the view in the same way as on a regular table. A view is automatically updatable if it satisfies all of the following conditions:

- The view must have exactly one entry in its `FROM` list, which must be a table or another updatable view.
- The view definition must not contain `WITH`, `DISTINCT`, `GROUP BY`, `HAVING`, `LIMIT`, or `OFFSET` clauses at the top level.
- The view definition must not contain set operations (`UNION`, `INTERSECT`, or `EXCEPT`) at the top level.
- The view's select list must not contain any aggregates, window functions, or set-returning functions.

An automatically updatable view may contain a mix of updatable and non-updatable columns. A column is updatable if it's a simple reference to an updatable column of the underlying base

relation. Otherwise, the column is read-only, and an error occurs if an `INSERT` or `UPDATE` statement attempts to assign a value to it.

For automatically updatable views, the system converts any `INSERT`, `UPDATE`, or `DELETE` statement on the view into the corresponding statement on the underlying base relation. `INSERT` statements with an `ON CONFLICT UPDATE` clause are fully supported.

If an automatically updatable view contains a `WHERE` condition, the condition restricts which rows of the base relation are available for modification by `UPDATE` and `DELETE` statements on the view. However, an `UPDATE` can change a row so that it no longer satisfies the `WHERE` condition, making it invisible through the view. Similarly, an `INSERT` command can potentially insert base-relation rows that don't satisfy the `WHERE` condition, making them invisible through the view. `ON CONFLICT UPDATE` may similarly affect an existing row not visible through the view.

You can use the `CHECK OPTION` to prevent `INSERT` and `UPDATE` commands from creating rows that aren't visible through the view.

If an automatically updatable view is marked with the `security_barrier` property, all the view's `WHERE` conditions (and any conditions using operators marked as `LEAKPROOF`) are always evaluated before any conditions that a user of the view has added. Note that due to this, rows that aren't ultimately returned (because they don't pass the user's `WHERE` conditions) may still end up being locked. You can use `EXPLAIN` to see which conditions are applied at the relation level (and therefore don't lock rows) and which aren't.

A more complex view that doesn't satisfy all these conditions is read-only by default: the system doesn't allow an insert, update, or delete on the view.

Note

The user performing the insert, update, or delete on the view must have the corresponding insert, update, or delete privilege on the view. By default, the view's owner must have the relevant privileges on the underlying base relations, while the user performing the update doesn't need any permissions on the underlying base relations. However, if the view has `security_invoker` set to `true`, the user performing the update, rather than the view owner, must have the relevant privileges on the underlying base relations.

Examples

To create a view consisting of all comedy films.


```
CREATE VIEW comedies AS
  SELECT *
  FROM films
  WHERE kind = 'Comedy';
```

This will create a view containing the columns that are in the `film` table at the time of view creation. Though `*` was used to create the view, columns added later to the table will not be part of the view.

Create a view with `LOCAL CHECK OPTION`.

```
CREATE VIEW pg_comedies AS
  SELECT *
  FROM comedies
  WHERE classification = 'PG'
  WITH CASCADED CHECK OPTION;
```

This will create a view that checks both the `kind` and `classification` of new rows.

Create a view with a mix of updatable and non-updatable columns.

```
CREATE VIEW comedies AS
  SELECT f.*,
         country_code_to_name(f.country_code) AS country,
         (SELECT avg(r.rating)
          FROM user_ratings r
          WHERE r.film_id = f.id) AS avg_rating
  FROM films f
  WHERE f.kind = 'Comedy';
```

This view will support `INSERT`, `UPDATE`, and `DELETE`. All the columns from the `films` table will be updatable, whereas the computed columns `country` and `avg_rating` will be read-only.

```
CREATE RECURSIVE VIEW public.nums_1_100 (n) AS
  VALUES (1)
  UNION ALL
  SELECT n+1 FROM nums_1_100 WHERE n < 100;
```

Note

Although the recursive view's name is schema-qualified in this CREATE, its internal self-reference is not schema-qualified. This is because the implicitly-created Common Table Expression's (CTE's) name cannot be schema-qualified.

Compatibility

CREATE OR REPLACE VIEW is a PostgreSQL language extension. The WITH (...) clause is an extension as well, as are security barrier views and security invoker views. Aurora DSQL supports these language extensions.

ALTER VIEW

The ALTER VIEW statement allows changing various properties of an existing view, and Aurora DSQL supports all the PostgreSQL syntax for this command.

Supported syntax

```
ALTER VIEW [ IF EXISTS ] name ALTER [ COLUMN ] column_name SET DEFAULT expression
ALTER VIEW [ IF EXISTS ] name ALTER [ COLUMN ] column_name DROP DEFAULT
ALTER VIEW [ IF EXISTS ] name OWNER TO { new_owner | CURRENT_ROLE | CURRENT_USER |
SESSION_USER }
ALTER VIEW [ IF EXISTS ] name RENAME [ COLUMN ] column_name TO new_column_name
ALTER VIEW [ IF EXISTS ] name RENAME TO new_name
ALTER VIEW [ IF EXISTS ] name SET SCHEMA new_schema
ALTER VIEW [ IF EXISTS ] name SET ( view_option_name [= view_option_value] [, ... ] )
ALTER VIEW [ IF EXISTS ] name RESET ( view_option_name [, ... ] )
```

Description

ALTER VIEW changes various auxiliary properties of a view. (If you want to modify the view's defining query, use CREATE OR REPLACE VIEW.) You must own the view to use ALTER VIEW. To change a view's schema, you must also have CREATE privilege on the new schema. To alter the owner, you must be able to SET ROLE to the new owning role, and that role must have CREATE privilege on the view's schema. These restrictions enforce that altering the owner doesn't do anything you couldn't do by dropping and recreating the view.)

Parameters

ALTER VIEW parameters

name

The name (optionally schema-qualified) of an existing view.

column_name

New name for an existing column.

IF EXISTS

Don't throw an error if the view doesn't exist. A notice is issued in this case.

SET/DROP DEFAULT

These forms set or remove the default value for a column. The default value for a view column is substituted into any INSERT or UPDATE command where the target is the view. The default for the view will take precedence over any default values from underlying relations.

new_owner

The user name of the new owner of the view.

new_name

The new name for the view.

new_schema

The new schema for the view.

**SET (view_option_name [= view_option_value] [, ...]), RESET
(view_option_name [, ...])**

Sets or resets a view option. The following are supported options.

- **check_option** (enum) - Changes the check option of the view. The value must be `local` or `cascaded`.
- **security_barrier** (boolean) - Changes the security-barrier property of the view. The value must be a Boolean value, such as `true` or `false`.
- **security_invoker** (boolean) - Changes the security-barrier property of the view. The value must be a Boolean value, such as `true` or `false`.

Notes

For historical PostgreSQL reasons, `ALTER TABLE` can be used with views too; but the only variants of `ALTER TABLE` that are allowed with views are equivalent to the ones shown previously.

Examples

Renaming the view `foo` to `bar`.

```
ALTER VIEW foo RENAME TO bar;
```

Attaching a default column value to an updatable view.

```
CREATE TABLE base_table (id int, ts timestampz);
CREATE VIEW a_view AS SELECT * FROM base_table;
ALTER VIEW a_view ALTER COLUMN ts SET DEFAULT now();
INSERT INTO base_table(id) VALUES(1); -- ts will receive a NULL
INSERT INTO a_view(id) VALUES(2); -- ts will receive the current time
```

Compatibility

`ALTER VIEW` is a PostgreSQL extension of the SQL standard that Aurora DSQL supports.

DROP VIEW

The `DROP VIEW` statement removes an existing view. Aurora DSQL supports the full PostgreSQL syntax for this command.

Supported syntax

```
DROP VIEW [ IF EXISTS ] name [, ...] [ CASCADE | RESTRICT ]
```

Description

`DROP VIEW` drops an existing view. To execute this command you must be the owner of the view.

Parameters

IF EXISTS

Don't throw an error if the view doesn't exist. A notice is issued in this case.

name

The name (optionally schema-qualified) of the view to remove.

CASCADE

Automatically drop objects that depend on the view (such as other views), and in turn all objects that depend on those objects.

RESTRICT

Refuse to drop the view if any objects depend on it. This is the default.

Examples

```
DROP VIEW kinds;
```

Compatibility

This command conforms to the SQL standard, except that the standard only allows one view to be dropped per command, and apart from the `IF EXISTS` option, which is a PostgreSQL extension that Aurora DSQL supports.

Unsupported PostgreSQL features in Aurora DSQL

Aurora DSQL is [PostgreSQL compatible](#). This means that Aurora DSQL supports core relational features such as ACID transactions, secondary indexes, joins, insert, and updates. For an overview of supported SQL features, see [Supported SQL expressions](#).

The following sections highlight which PostgreSQL features are currently unsupported in Aurora DSQL.

Unsupported objects

Objects unsupported by Aurora DSQL include the following:

- Multiple databases on a single Aurora DSQL cluster
- Temporary Tables
- Triggers
- Types (partial support)
- Tablespaces

- Functions written in languages other than SQL
- Sequences
- Partitions

Unsupported constraints

- Foreign keys
- Exclusion constraints

Unsupported commands

- ALTER SYSTEM
- TRUNCATE
- SAVEPOINT
- VACUUM

Note

Aurora DSQL doesn't require vacuuming. The system maintains statistics and manages storage optimization automatically without manual vacuum commands.

Unsupported extensions

Aurora DSQL doesn't support PostgreSQL extensions. The following table shows extensions that are unsupported:

- PL/pgSQL
- PostGIS
- PGVector
- PGAudit
- Postgres_FDW
- PGCron
- pg_stat_statements

Unsupported SQL expressions

The following table describes clauses that aren't supported in Aurora DSQL.

Category	Primary Clause	Unsupported Clause
CREATE	INDEX ASYNC	ASC DESC
CREATE	INDEX ¹	
TRUNCATE		
ALTER	SYSTEM	All ALTER SYSTEM commands are blocked.
CREATE	TABLE	COLLATE, AS SELECT, INHERITS, PARTITION
CREATE	FUNCTION	LANGUAGE <i>non-sql-lang</i> , where <i>non-sql-lang</i> is any language other than SQL
CREATE	TEMPORARY	TABLES
CREATE	EXTENSION	
CREATE	SEQUENCE	
CREATE	MATERIALIZED	VIEW
CREATE	TABLESPACE	
CREATE	TRIGGER	
CREATE	TYPE	
CREATE	DATABASE	You can't create additional databases.

¹ See [Asynchronous indexes in Aurora DSQL](#) to create an index on a column of a specified table.

Aurora DSQL considerations for PostgreSQL compatibility

Consider the following compatibility limitations when using Aurora DSQL. For general considerations, see [Considerations for working with Amazon Aurora DSQL](#). For quotas and limits, see [Cluster quotas and database limits in Amazon Aurora DSQL](#).

- Aurora DSQL uses a single built-in database named `postgres`. You can't create additional databases or rename or drop the `postgres` database.
- The `postgres` database uses UTF-8 character encoding. You can't change the server encoding.
- The database uses the C collation only.
- Aurora DSQL uses UTC as the system timezone. `Postgres` stores all timezone-aware dates and times internally in UTC. You can set the `TimeZone` configuration parameter to convert how it is displayed to the client and serve as the default for client input that the server will use to convert to UTC internally.
- The transaction isolation level is fixed at PostgreSQL Repeatable Read.
- Transactions have the following constraints:
 - A transaction can't mix DDL and DML operations
 - A transaction can include only 1 DDL statement
 - A transaction can modify up to 3,000 rows, regardless of the number of secondary indexes
 - The 3,000-row limit applies to all DML statements (INSERT, UPDATE, DELETE)
- Database connections time out after 1 hour.
- Aurora DSQL doesn't currently let you run `GRANT [permission] ON DATABASE`. If you attempt to run that statement, Aurora DSQL returns the error message `ERROR: unsupported object type in GRANT`.
- Aurora DSQL doesn't let non-admin user roles to run the `CREATE SCHEMA` command. You can't run the `GRANT [permission] on DATABASE` command and grant `CREATE` permissions on the database. If a non-admin user role tries to create a schema, Aurora DSQL returns with the error message `ERROR: permission denied for database postgres`.
- Non-admin users can't create objects in the public schema. Only admin users can create objects in the public schema. The admin user role has permissions to grant read, write, and modify access to these objects to non-admin users, but it cannot grant `CREATE` permissions to the public schema itself. Non-admin users must use different, user-created schemas for object creation.
- Aurora DSQL doesn't support the command `ALTER ROLE [] CONNECTION LIMIT`. Contact AWS support if you need a connection limit increase.

- Aurora DSQL doesn't support `asyncpg`, the asynchronous PostgreSQL database driver for Python.

Concurrency control in Aurora DSQL

Concurrency allows multiple sessions to access and modify data simultaneously without compromising data integrity and consistency. Aurora DSQL provides [PostgreSQL compatibility](#) while implementing a modern, lock-free concurrency control mechanism. It maintains full ACID compliance through snapshot isolation, ensuring data consistency and reliability.

A key advantage of Aurora DSQL is its lock-free architecture, which eliminates common database performance bottlenecks. Aurora DSQL prevents slow transactions from blocking other operations and eliminates the risk of deadlocks. This approach makes Aurora DSQL particularly valuable for high-throughput applications where performance and scalability are critical.

Transaction conflicts

Aurora DSQL uses optimistic concurrency control (OCC), which works differently from traditional lock-based systems. Instead of using locks, OCC evaluates conflicts at commit time. When multiple transactions conflict while updating the same row, Aurora DSQL manages transactions as follows:

- The transaction with the earliest commit time is processed by Aurora DSQL.
- Conflicting transactions receive a PostgreSQL serialization error, indicating the need to be retried.

Design your applications to implement retry logic to handle conflicts. The ideal design pattern is idempotent, enabling transaction retry as a first recourse whenever possible. The recommended logic is similar to the abort and retry logic in a standard PostgreSQL lock timeout or deadlock situation. However, OCC requires your applications to exercise this logic more frequently.

Guidelines for optimizing transaction performance

To optimize performance, minimize high contention on single keys or small key ranges. To achieve this goal, design your schema to spread updates over your cluster key range by using the following guidelines:

- Choose a random primary key for your tables.
- Avoid patterns that increase contention on single keys. This approach ensures optimal performance even as transaction volume grows.

DDL and distributed transactions in Aurora DSQL

Data definition language (DDL) behaves differently in Aurora DSQL from PostgreSQL. Aurora DSQL features a Multi-AZ distributed and shared-nothing database layer built on top of multi-tenant compute and storage fleets. Because no single primary database node or leader exists, the database catalog is distributed. Thus, Aurora DSQL manages DDL schema changes as distributed transactions.

Specifically, DDL behaves differently in Aurora DSQL as follows:

Concurrency control errors

Aurora DSQL returns a concurrency control violation error if you run one transaction while another transaction updates a resource. For example, consider the following sequence of actions:

1. In session 1, a user adds a column to the table mytable.
2. In session 2, a user attempts to insert a row into mytable.

Aurora DSQL returns the error `SQL Error [40001]: ERROR: schema has been updated by another transaction, please retry: (0C001)`.

DDL and DML in the same transaction

Transactions in Aurora DSQL can contain only one DDL statement and can't have both DDL and DML statements. This restriction means that you can't create a table and insert data into the same table within the same transaction. For example, Aurora DSQL supports the following sequential transactions.

```
BEGIN;  
  CREATE TABLE mytable (ID_col integer);  
COMMIT;  
  
BEGIN;  
  INSERT into F00 VALUES (1);  
COMMIT;
```

Aurora DSQL doesn't support the following transaction, which includes both CREATE and INSERT statements.

```
BEGIN;  
  CREATE TABLE F00 (ID_col integer);  
  INSERT into F00 VALUES (1);  
COMMIT;
```

Asynchronous DDL

In standard PostgreSQL, DDL operations such as `CREATE INDEX` lock the affected table, making it unavailable for reads and writes from other sessions. In Aurora DSQL, these DDL statements run asynchronously using a background manager. Access to the affected table isn't blocked. Thus, DDL on large tables can run without downtime or performance impact. For more information about the asynchronous job manager in Aurora DSQL, see [Asynchronous indexes in Aurora DSQL](#).

Primary keys in Aurora DSQL

In Aurora DSQL, a *primary key* is a feature that physically organizes table data. It's similar to the `CLUSTER` operation in PostgreSQL or a clustered index in other databases. When you define a primary key, Aurora DSQL creates an index that includes all columns in the table. The primary key structure in Aurora DSQL ensures efficient data access and management.

Data structure and storage

When you define a primary key, Aurora DSQL stores table data in primary key order. This index-organized structure allows a primary key lookup to retrieve all column values directly, instead of following a pointer to the data as in a traditional B-tree index. Unlike the `CLUSTER` operation in PostgreSQL, which reorganizes data only once, Aurora DSQL maintains this order automatically and continuously. This approach improves the performance of queries that rely on primary key access.

Aurora DSQL also uses the primary key to generate a cluster-wide unique key for each row in tables and indexes. This unique key also underpins distributed data management. It enables automatic partitioning of data across multiple nodes, supporting scalable storage and high concurrency. As a result, the primary key structure helps Aurora DSQL scale automatically and manage concurrent workloads efficiently.

Guidelines for choosing a primary key

When choosing and using a primary key in Aurora DSQL, consider the following guidelines:

- Define a primary key when you create a table. You can't change this key or add a new primary key later. The primary key becomes part of the cluster-wide key used for data partitioning and automatic scaling of write throughput. If you don't specify a primary key, Aurora DSQL assigns a synthetic hidden ID.
- For tables with high write volumes, avoid using monotonically increasing integers as primary keys. This can lead to performance issues by directing all new inserts to a single partition. Instead, use primary keys with random distribution to ensure even distribution of writes across storage partitions.
- For tables that change infrequently or are read-only, you can use an ascending key. Examples of ascending keys are timestamps or sequence numbers. A dense key has many closely spaced or duplicate values. You can use an ascending key even if it is dense because write performance is less critical.
- If a full table scan doesn't meet your performance requirements, choose a more efficient access method. In most cases, this means using a primary key that matches your most common join and lookup key in queries.
- The maximum combined size of columns in a primary key is 1 kibibyte. For more information, see [Database limits in Aurora DSQL](#) and [Supported data types in Aurora DSQL](#).
- You can include up to 8 columns in a primary key or a secondary index. For more information, see [Database limits in Aurora DSQL](#) and [Supported data types in Aurora DSQL](#).

Asynchronous indexes in Aurora DSQL

The `CREATE INDEX ASYNC` command creates an index on one or more columns of a specified table. This command is an asynchronous DDL operation that doesn't block other transactions. When you run `CREATE INDEX ASYNC`, Aurora DSQL immediately returns a `job_id`.

You can monitor the status of this asynchronous job using the `sys.jobs` system view. While the index creation job is in progress, you can use these procedures and commands:

`sys.wait_for_job(job_id) 'your_index_creation_job_id'`

Blocks the current session until the specified job completes or fails. Returns a Boolean value indicating success or failure.

DROP INDEX

Cancels an in-progress index build job.

When the asynchronous index creation completes, Aurora DSQL updates the system catalog to mark the index as active.

 Note

Note that concurrent transactions accessing objects in the same namespace during this update might encounter concurrency errors.

When Aurora DSQL finishes an asynchronous index task, it updates the system catalog to show that the index is active. If other transactions reference the objects in the same namespace at this time, you might see a concurrency error.

Syntax

CREATE INDEX ASYNC uses the following syntax.

```
CREATE [ UNIQUE ] INDEX ASYNC [ IF NOT EXISTS ] name ON table_name
    ( { column_name } [ NULLS { FIRST | LAST } ] )
    [ INCLUDE ( column_name [, ...] ) ]
    [ NULLS [ NOT ] DISTINCT ]
```

Parameters

UNIQUE

Indicates to Aurora DSQL to check for duplicate values in the table when it creates the index and each time you add data. If you specify this parameter, insert and update operations that would result in duplicate entries generate an error.

IF NOT EXISTS

Indicates that Aurora DSQL shouldn't throw an exception if an index with the same name already exists. In this situation, Aurora DSQL doesn't create the new index. Note that the index you're trying to create could have a very different structure from the index that exists. If you specify this parameter, the index name is required.

name

The name of the index. You can't include the name of your schema in this parameter.

Aurora DSQL creates the index in the same schema as its parent table. The name of the index must be distinct from the name of any other object, such as a table or index, in the schema.

If you don't specify a name, Aurora DSQL generates a name automatically based on the name of the parent table and indexed column. For example, if you run `CREATE INDEX ASYNC ON table1 (col1, col2)`, Aurora DSQL automatically names the index `table1_col1_col2_idx`.

NULLS FIRST | LAST

The sort order of null and non-null columns. `FIRST` indicates that Aurora DSQL should sort null columns before non-null columns. `LAST` indicates that Aurora DSQL should sort null columns after non-null columns.

INCLUDE

A list of columns to include in the index as non-key columns. You can't use a non-key column in an index scan search qualification. Aurora DSQL ignores the column in terms of uniqueness for an index.

NULLS DISTINCT | NULLS NOT DISTINCT

Specifies whether Aurora DSQL should consider null values as distinct in a unique index. The default is `DISTINCT`, meaning that a unique index can contain multiple null values in a column. `NOT DISTINCT` indicates that an index can't contain multiple null values in a column.

Usage notes

Consider the following guidelines:

- The `CREATE INDEX ASYNC` command doesn't introduce locks. It also doesn't affect the base table that Aurora DSQL uses to create the index.
- During schema migration operations, the `sys.wait_for_job(job_id) 'your_index_creation_job_id'` procedure is useful. It ensures that subsequent DDL and DML operations target the newly created index.
- Every time Aurora DSQL runs a new asynchronous task, it checks the `sys.jobs` view and deletes tasks that have a status of `completed` or `failed` for more than 30 minutes. Thus, `sys.jobs` primarily shows in-progress tasks and doesn't contain information about old tasks.

- If Aurora DSQL fails to build an asynchronous index, the index stays INVALID. For unique indexes, DML operations are subject to uniqueness constraints until you drop the index. We recommend that you drop invalid indexes and recreate them.

Creating an index: example

The following example demonstrates how to create a schema, a table, and then an index.

1. Create a table named `test.departments`.

```
CREATE SCHEMA test;

CREATE TABLE test.departments (name varchar(255) primary key NOT null,
                                manager varchar(255),
                                size varchar(4));
```

2. Insert a row into the table.

```
INSERT INTO test.departments VALUES ('Human Resources', 'John Doe', '10')
```

3. Create an asynchronous index.

```
CREATE INDEX ASYNC test_index on test.departments(name, manager, size);
```

The `CREATE INDEX` command returns a job ID, as shown below.

```
job_id
-----
jh2gbtx4mzhgfkbtgwn5j45y
```

The `job_id` indicates that Aurora DSQL has submitted a new job to create the index. You can use the procedure `sys.wait_for_job(job_id)` *'your_index_creation_job_id'* to block other work on the session until the job finishes or times out.

Querying the status of index creation: example

Query the `sys.jobs` system view to check the creation status of your index, as shown in the following example.

```
SELECT * FROM sys.jobs
```

Aurora DSQL returns a response similar to the following.

job_id	status	details
vs3kcl3rt5ddpk3a6xcq57cmcy	completed	
ihbyw2aoirfnrdfoc4ojnlamoq	processing	

The status column can be one of the following values.

submitted	processing	failed	completed
The task is submitted , but Aurora DSQL hasn't started to process it yet.	Aurora DSQL is processing the task.	The task failed. See the details column for more information. If Aurora DSQL failed to build the index, Aurora DSQL doesn't automatically remove the index definition. You must manually remove the index with the DROP INDEX command.	Aurora DSQL

You can also query the state of the index via the catalog tables `pg_index` and `pg_class`. Specifically, the attributes `indisvalid` and `indisimmediate` can tell you what state your index is in. While Aurora DSQL creates your index, it has an initial status of `INVALID`. The `indisvalid` flag for the index returns `FALSE` or `f`, which indicates that the index isn't valid. If the flag returns `TRUE` or `t`, the index is ready.

```
SELECT relname AS index_name, indisvalid as is_valid, pg_get_indexdef(indexrelid) AS
  index_definition
from pg_index, pg_class
WHERE pg_class.oid = indexrelid AND indrelid = 'test.departments'::regclass;
```



```

    index_name      | is_valid |
index_definition
-----+-----
+-----+-----
department_pkey   |      t   | CREATE UNIQUE INDEX department_pkey ON test.departments
USING btree_index (title) INCLUDE (name, manager, size)
test_index1       |      t   | CREATE INDEX test_index1 ON test.departments USING
btree_index (name, manager, size)

```

Unique index build failures

If your asynchronous unique index build job shows a failed state with the detail Found duplicate key while validating index for UCVs, this indicates that a unique index could not be built due to uniqueness constraint violations.

To resolve unique index build failures

1. Remove any rows in your primary table that have duplicate entries for the keys specified in your unique secondary index.
2. Drop the failed index.
3. Issue a new create index command.

Detecting uniqueness violations in primary tables

The following SQL query helps you identify duplicate values in a specified column of your table. This is particularly useful when you need to enforce uniqueness on a column that isn't currently set as a primary key or doesn't have a unique constraint, such as email addresses in a user table.

The examples below demonstrate how to create a sample users table, populate it with test data containing known duplicates, and then run the detection query.

Define table schema

```

-- Drop the table if it exists
DROP TABLE IF EXISTS users;

-- Create the users table with a simple integer primary key
CREATE TABLE users (
    user_id INTEGER PRIMARY KEY,

```

```

email VARCHAR(255),
first_name VARCHAR(100),
last_name VARCHAR(100),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Insert sample data that includes sets of duplicate email addresses

```

-- Insert sample data with explicit IDs
INSERT INTO users (user_id, email, first_name, last_name) VALUES
  (1, 'john.doe@example.com', 'John', 'Doe'),
  (2, 'jane.smith@example.com', 'Jane', 'Smith'),
  (3, 'john.doe@example.com', 'Johnny', 'Doe'),
  (4, 'alice.wong@example.com', 'Alice', 'Wong'),
  (5, 'bob.jones@example.com', 'Bob', 'Jones'),
  (6, 'alice.wong@example.com', 'Alicia', 'Wong'),
  (7, 'bob.jones@example.com', 'Robert', 'Jones');

```

Run duplicate detection query

```

-- Query to find duplicates
WITH duplicates AS (
  SELECT email, COUNT(*) as duplicate_count
  FROM users
  GROUP BY email
  HAVING COUNT(*) > 1
)
SELECT u.*, d.duplicate_count
FROM users u
INNER JOIN duplicates d ON u.email = d.email
ORDER BY u.email, u.user_id;

```

View all records with duplicate email addresses

user_id	email	first_name	last_name	created_at
duplicate_count				
4	akua.mansa@example.com	Akua	Mansa	2025-05-21 20:55:53.714432
2				
6	akua.mansa@example.com	Akua	Mansa	2025-05-21 20:55:53.714432
2				

1		john.doe@example.com		John		Doe		2025-05-21 20:55:53.714432
		2						
3		john.doe@example.com		Johnny		Doe		2025-05-21 20:55:53.714432
		2						
(4 rows)								

If we were to try the index creation statement now, it would fail:

```
postgres=> CREATE UNIQUE INDEX ASYNC idx_users_email ON users(email);
           job_id
-----
ve32upmjz5dgdknpbleeca5tri
(1 row)

postgres=> select * from sys.jobs;
      job_id      | status |      details      |
+-----+-----+-----+-----+-----+-----+
| job_type | class_id | object_id | object_name | start_time |
| update_time |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
qpn6aqlkijgmzilyidcpwrpova | completed |
| DROP | 1259 | 26384 | | 2025-05-20
00:47:10+00 | 2025-05-20 00:47:32+00
ve32upmjz5dgdknpbleeca5tri | failed | Found duplicate key while validating index
for UCVs | INDEX_BUILD | 1259 | 26396 | public.idx_users_email | 2025-05-20
00:49:49+00 | 2025-05-20 00:49:56+00
(2 rows)
```

System tables and commands in Aurora DSQL

See the following sections to learn about the supported system tables and catalogs in Aurora DSQL.

System tables

Aurora DSQL is compatible with PostgreSQL, so many [system catalog tables](#) and [views](#) from PostgreSQL also exist in Aurora DSQL.

Important PostgreSQL catalog tables and views

The following table describes the most common tables and views you might use in Aurora DSQL.

Name	Description
pg_namespace	Information on all schemas
pg_tables	Information on the all tables
pg_attribute	Information on all attributes
pg_views	Information on (pre-)defined views
pg_class	Describes all tables, column, indices, and similar objects
pg_stats	A view on the planner statistics
pg_user	Information on users
pg_roles	Information on users and groups
pg_indexes	Lists all indexes
pg_constraint	Lists constraints on tables

Supported and unsupported catalog tables

The following table indicates which tables are supported and unsupported in Aurora DSQL.

Name	Applicable to Aurora DSQL
pg_aggregate	No
pg_am	Yes
pg_amop	No
pg_amproc	No
pg_attrdef	Yes
pg_attribute	Yes

Name	Applicable to Aurora DSQL
pg_authid	No (use pg_roles)
pg_auth_members	Yes
pg_cast	Yes
pg_class	Yes
pg_collation	Yes
pg_constraint	Yes
pg_conversion	No
pg_database	No
pg_db_role_setting	Yes
pg_default_acl	Yes
pg_depend	Yes
pg_description	Yes
pg_enum	No
pg_event_trigger	No
pg_extension	No
pg_foreign_data_wrapper	No
pg_foreign_server	No
pg_foreign_table	No
pg_index	Yes
pg_inherits	Yes

Name	Applicable to Aurora DSQL
pg_init_privs	No
pg_language	No
pg_largeobject	No
pg_largeobject_metadata	Yes
pg_namespace	Yes
pg_opclass	No
pg_operator	Yes
pg_opfamily	No
pg_parameter_acl	Yes
pg_partitioned_table	No
pg_policy	No
pg_proc	No
pg_publication	No
pg_publication_namespace	No
pg_publication_rel	No
pg_range	Yes
pg_replication_origin	No
pg_rewrite	No
pg_seclabel	No
pg_sequence	No

Name	Applicable to Aurora DSQL
pg_shdepend	Yes
pg_shdescription	Yes
pg_shseclabel	No
pg_statistic	Yes
pg_statistic_ext	No
pg_statistic_ext_data	No
pg_subscription	No
pg_subscription_rel	No
pg_tablespace	No
pg_transform	No
pg_trigger	No
pg_ts_config	Yes
pg_ts_config_map	Yes
pg_ts_dict	Yes
pg_ts_parser	Yes
pg_ts_template	Yes
pg_type	Yes
pg_user_mapping	No

Supported and unsupported system views

The following table indicates which views are supported and unsupported in Aurora DSQL.

Name	Applicable to Aurora DSQL
pg_available_extensions	No
pg_available_extension_versions	No
pg_backend_memory_contexts	Yes
pg_config	No
pg_cursors	No
pg_file_settings	No
pg_group	Yes
pg_hba_file_rules	No
pg_ident_file_mappings	No
pg_indexes	Yes
pg_locks	No
pg_matviews	No
pg_policies	No
pg_prepared_statements	No
pg_prepared_xacts	No
pg_publication_tables	No
pg_replication_origin_status	No
pg_replication_slots	No
pg_roles	Yes
pg_rules	No

Name	Applicable to Aurora DSQL
pg_seclabels	No
pg_sequences	No
pg_settings	Yes
pg_shadow	Yes
pg_shmem_allocations	Yes
pg_stats	Yes
pg_stats_ext	No
pg_stats_ext_exprs	No
pg_tables	Yes
pg_timezone_abbrevs	Yes
pg_timezone_names	Yes
pg_user	Yes
pg_user_mappings	No
pg_views	Yes
pg_stat_activity	No
pg_stat_replication	No
pg_stat_replication_slots	No
pg_stat_wal_receiver	No
pg_stat_recovery_prefetch	No
pg_stat_subscription	No

Name	Applicable to Aurora DSQL
pg_stat_subscription_stats	No
pg_stat_ssl	Yes
pg_stat_gssapi	No
pg_stat_archiver	No
pg_stat_io	No
pg_stat_bgwriter	No
pg_stat_wal	No
pg_stat_database	No
pg_stat_database_conflicts	No
pg_stat_all_tables	No
pg_stat_all_indexes	No
pg_statio_all_tables	No
pg_statio_all_indexes	No
pg_statio_all_sequences	No
pg_stat_slru	No
pg_statio_user_tables	No
pg_statio_user_sequences	No
pg_stat_user_functions	No
pg_stat_user_indexes	No
pg_stat_progress_analyze	No

Name	Applicable to Aurora DSQL
pg_stat_progress_basebackup	No
pg_stat_progress_cluster	No
pg_stat_progress_create_index	No
pg_stat_progress_vacuum	No
pg_stat_sys_indexes	No
pg_stat_sys_tables	No
pg_stat_xact_all_tables	No
pg_stat_xact_sys_tables	No
pg_stat_xact_user_functions	No
pg_stat_xact_user_tables	No
pg_statio_sys_indexes	No
pg_statio_sys_sequences	No
pg_statio_sys_tables	No
pg_statio_user_indexes	No

The sys.jobs and sys.iam_pg_role_mappings views

Aurora DSQL supports the following system views:

sys.jobs

sys.jobs provides status information about asynchronous jobs. For example, after you [create an asynchronous index](#), Aurora DSQL returns a job_uuid. You can use this job_uuid with sys.jobs to look up the status of the job.

```
SELECT * FROM sys.jobs WHERE job_id = 'example_job_uuid';
```

```

      job_id      | status | details
-----+-----+-----
example_job_uuid | processing |
(1 row)

```

sys.iam_pg_role_mappings

The view `sys.iam_pg_role_mappings` provides information about the permissions granted to IAM users. For example, if `DQSLDBConnect` is an IAM role that gives Aurora DSQL access to non-admins and a user named `testuser` is granted the `DQSLDBConnect` role and corresponding permissions, you can query the `sys.iam_pg_role_mappings` view to see which users are granted which permissions.

```
SELECT * FROM sys.iam_pg_role_mappings;
```

Useful system metadata queries

Use these queries to get table statistics and metadata without performing expensive operations like full table scans.

Get estimated row count for a table

To get the approximate count of rows in a table without performing a full table scan, use the following query:

```
SELECT reltuples FROM pg_class WHERE relname = 'table_name';
```

The command returns output similar to the following:

```

reltuples
-----
9.993836e+08

```

This approach is more efficient than `SELECT COUNT(*)` for large tables in Aurora DSQL.

The ANALYZE command

The `ANALYZE` command collects statistics about the contents of tables in the database and stores the results in the `pg_stats` system view. Subsequently, the query planner uses these statistics to help determine the most efficient execution plans for queries.

In Aurora DSQL, you can't run the `ANALYZE` command within an explicit transaction. `ANALYZE` isn't subject to the database transaction timeout limit.

To reduce the need for manual intervention and keep statistics consistently up to date, Aurora DSQL automatically runs `ANALYZE` as a background process. This background job is triggered automatically based on the observed rate of change in the table. It is linked to the number of rows (tuples) that have been inserted, updated, or deleted since the last analyze.

`ANALYZE` runs asynchronously in the background and its activity can be monitored in the system view `sys.jobs` with the following query:

```
SELECT * FROM sys.jobs WHERE job_type = 'ANALYZE';
```

Key considerations

Note

`ANALYZE` jobs are billed like other asynchronous jobs in Aurora DSQL. When you modify a table, this may indirectly trigger an automatic background statistics collection job, which can result in metering charges due to the associated system-level activity.

Background `ANALYZE` jobs, triggered automatically, collect the same types of statistics as a manual `ANALYZE` and apply them by default to user tables. System and catalog tables are excluded from this automated process.

Working with Aurora DSQL EXPLAIN plans

Aurora DSQL uses a similar `EXPLAIN` plan structure to PostgreSQL, but with key additions that reflect its distributed architecture and execution model.

In this documentation, we'll provide an overview of Aurora DSQL EXPLAIN plans, highlighting the similarities and differences compared to PostgreSQL. We'll cover the various types of scan operations available in Aurora DSQL and help you understand the cost of running your queries.

PostgreSQL VS Aurora DSQL EXPLAIN plans

Aurora DSQL is built on top of the PostgreSQL database and shares most plan structures with PostgreSQL, but has key architectural differences that affect query execution and optimization:

Feature	PostgreSQL	Aurora DSQL
Data Storage	Heap Storage	No heap, all rows are indexed by a unique identifier
Primary Key	Primary key index is separate from table data	Primary key index is the table with all extra columns as INCLUDE columns
Secondary Indexes	Standard secondary indexes	Work the same as PostgreSQL, with ability to include non-key columns
Filtering Capabilities	Index Condition, Heap Filter	Index Condition, Storage Filter, Query Processor Filter
Scan Types	Sequential Scan, Index Scan, Index Only Scan	Full Scan, Index Only Scan, Index Scan
Query Execution	Local to the Database	Distributed (compute and storage are separate)

Aurora DSQL stores table data directly in primary key order rather than in a separate heap. Each row is identified by a unique key, typically the primary key, which allows the database to optimize lookups more efficiently. The architectural difference explains why Aurora DSQL often uses Index Only Scans in cases where PostgreSQL might choose a sequential scan.

Another key distinction is that Aurora DSQL separates compute from storage, enabling filters to be applied earlier in the execution path to reduce data movement and improve performance.

For more using EXPLAIN plans with PostgreSQL, see the [PostgreSQL EXPLAIN documentation](#).

Key elements in Aurora DSQL EXPLAIN plans

Aurora DSQL EXPLAIN plans provide detailed information about how queries are executed, including where filtering occurs and which columns are retrieved from storage. Understanding this output helps you optimize query performance.

Index Cond

Conditions used to navigate the index. Most efficient filtering that reduces data scanned. In Aurora DSQL, index conditions can be applied at multiple layers of the execution plan.

Projections

Columns retrieved from storage. Fewer projections mean better performance.

Storage Filter

Conditions applied at storage level. More efficient than query processor filters.

Query Processor Filter

Conditions applied at the query processor level. Requires transferring all data before filtering, which results in higher data movement and processing overhead.

Filters in Aurora DSQL

Aurora DSQL separates compute from storage, which means that the point where filters are applied during query execution has a significant impact on performance. Filters applied before large volumes of data are transferred reduce latency and improve efficiency. The earlier a filter is applied, the less data needs to be processed, moved, and scanned, resulting in faster queries.

Aurora DSQL can apply filters at multiple stages in the query path. Understanding these stages is key to interpreting query plans and optimizing performance.

Level	Filter Type	Description
1	Index Condition	Applied while scanning the index. Limits how much data is read from storage and reduces the data sent to the compute layer.

Level	Filter Type	Description
2	Storage Filter	Applied after data is read from storage but before it's sent to compute. An example here is a filter on an include column of an index. Reduces data transfer but not the amount read.
3	Query Processor Filter	Applied after data reaches the compute layer. All data must be transferred first, which increases latency and cost. Currently, Aurora DSQL cannot perform all filtering and projection operations on storage, so some queries might be forced to fall back to this type of filtering.

Reading Aurora DSQL EXPLAIN plans

Understanding how to read EXPLAIN plans is key to optimizing query performance. In this section, we'll walk through real examples of Aurora DSQL query plans, show how different scan types behave, explain where filters are applied, and highlight opportunities for optimization.

Full Scan example

Aurora DSQL has both Sequential Scans, which is functionally identical to PostgreSQL, as well as Full Scans. The only difference between these two are that Full Scans can utilize extra filtering on storage. Due to this, it is almost always selected above Sequential Scans. Due to the similarity, we will only cover examples of the more interesting Full Scans.

Full Scans will mostly be used on tables with no primary key. Because Aurora DSQL primary keys are by default full covering indexes, Aurora DSQL will most likely use Index Only Scans on the primary key in many situations where PostgreSQL would use a Sequential Scan. As with most other databases, a table with no indexes on it will scale badly.

```
EXPLAIN SELECT account_id FROM transaction WHERE transaction_date > '2025-01-01' AND
description LIKE '%external%';
```

QUERY PLAN


```

Full Scan (btree-table) on transaction (cost=125100.05..177933.38 rows=33333
width=16)
  Filter: (description ~~ '%external% '::text)
    -> Storage Scan on transaction (cost=12510.05..17793.38 rows=66666 width=16)
        Projections: account_id, description
        Filters: (transaction_date > '2025-01-01 00:00:00'::timestamp without time
zone)
          -> B-Tree Scan on transaction (cost=12510.05..17793.38 rows=100000 width=30)

```

This plan shows two filters applied at different stages. The `transaction_date > '2025-01-01'` condition is applied at the storage layer, reducing how much data is returned. The `description LIKE '%external%'` condition is applied later in the query processor, after data is transferred, making it less efficient. Pushing more selective filters into the storage or index layers generally improves performance.

Index Only Scan example

Index Only Scans are the most optimal scan types in Aurora DSQL as they result in the fewest round trips to the storage layer and can do the most filtering. But just because you see Index Only Scan does not mean that you have the best plan. Because of all the different levels of filtering that can happen, it is essential to still pay attention to the different places filtering can happen.

```

EXPLAIN SELECT balance FROM account
WHERE customer_id = '4b18a761-5870-4d7c-95ce-0a48eca3fceb'
AND balance > 100
AND status = 'pending';

```

QUERY PLAN

```

-----
Index Only Scan using idx1 on account (cost=725.05..1025.08 rows=8 width=18)
  Index Cond: (customer_id = '4b18a761-5870-4d7c-95ce-0a48eca3fceb'::uuid)
  Filter: (balance > '100'::numeric)
    -> Storage Scan on idx1 (cost=12510.05..17793.38 rows=9 width=16)
        Projections: balance
        Filters: ((status)::text = 'pending'::text)
          -> B-Tree Scan on idx1 (cost=12510.05..17793.38 rows=10 width=30)
              Index Cond: (customer_id = '4b18a761-5870-4d7c-95ce-0a48eca3fceb'::uuid)

```

In this plan, the index condition, `customer_id = '4b18a761-5870-4d7c-95ce-0a48eca3fceb'`, is evaluated first during the index scan, which

is the most efficient stage because it limits how much data is read from storage. The storage filter, `status = 'pending'`, is applied after data is read but before it's sent to the compute layer, reducing the amount of data transferred. Finally, the query processor filter, `balance > 100`, runs last, after the data has been moved, making it the least efficient. Of these, the index condition provides the greatest performance because it directly controls how much data is scanned.

Index Scan example

Index Scans are similar to Index Only Scans, except they have the extra step of having to call into the base table. Because Aurora DSQL can specify storage filters, it is able to do so on both the index call as well as the lookup call.

To make this clear, Aurora DSQL presents the plan as two nodes. This way, you can clearly see how much adding an include column will help in terms of rows returned from storage.

```
EXPLAIN SELECT balance FROM account
WHERE customer_id = '4b18a761-5870-4d7c-95ce-0a48eca3fceb'
AND balance > 100
AND status = 'pending'
AND created_at > '2025-01-01';
```

QUERY PLAN

```
-----
Index Scan using idx1 on account (cost=728.18..1132.20 rows=3 width=18)
  Filter: (balance > '100'::numeric)
  Index Cond: (customer_id = '4b18a761-5870-4d7c-95ce-0a48eca3fceb'::uuid)
  -> Storage Scan on idx1 (cost=12510.05..17793.38 rows=8 width=16)
    Projections: balance
    Filters: ((status)::text = 'pending'::text)
    -> B-Tree Scan on account (cost=12510.05..17793.38 rows=10 width=30)
      Index Cond: (customer_id = '4b18a761-5870-4d7c-95ce-0a48eca3fceb'::uuid)
    -> Storage Lookup on account (cost=12510.05..17793.38 rows=4 width=16)
      Filters: (created_at > '2025-01-01 00:00:00'::timestamp without time zone)
      -> B-Tree Lookup on transaction (cost=12510.05..17793.38 rows=8 width=30)
```

This plan shows how filtering happens across multiple stages:

- The index condition on `customer_id` filters data early.
- The storage filter on `status` further narrows results before they're sent to compute.
- The query processor filter on `balance` is applied later, after transfer.

- The lookup filter on `created_at` is evaluated when fetching additional columns from the base table.

Adding frequently used columns as `INCLUDE` fields can often eliminate this lookup and improve performance.

Best Practices

- **Align filters with indexed columns** to push filtering earlier.
- **Use `INCLUDE` columns** to allow Index-Only Scans and avoid lookups.
- **Keep statistics up to date** to ensure cost and row estimates are accurate.
- **Avoid unindexed queries** on large tables to prevent expensive Full Scans.

Managing Aurora DSQL clusters

Aurora DSQL provides several configuration options to help you establish the right database infrastructure for your needs. To set up your Aurora DSQL cluster infrastructure, review the following sections.

Topics

- [Configuring single-Region clusters](#)
- [Configuring multi-Region clusters](#)
- [Configuring Aurora DSQL clusters using AWS CloudFormation](#)

The features and functionality discussed in this guide ensure that your Aurora DSQL environment is more resilient, responsive, and capable of supporting your applications as they grow and evolve.

Configuring single-Region clusters

Configure and manage clusters for an AWS Region using either the AWS CLI or your preferred programming language including Python, C++, JavaScript, Java, Rust, Ruby, .NET, and Golang. The AWS CLI provides quick access through shell commands, while AWS Software Development Kits (SDKs) enable programmatic control through native language support.

Topics

- [Using AWS SDKs](#)
- [Using AWS CLI](#)

Using AWS SDKs

The AWS SDKs provide programmatic access to Aurora DSQL in your preferred programming language. The following sections show how to perform common cluster operations using different programming languages.

Create cluster

The following examples show how to create a single-Region cluster using different programming languages.

Python

To create a cluster in a single AWS Region, use the following example.

```
import boto3

def create_cluster(region):
    try:
        client = boto3.client("dsq1", region_name=region)
        tags = {"Name": "Python single region cluster"}
        cluster = client.create_cluster(tags=tags, deletionProtectionEnabled=True)
        print(f"Initiated creation of cluster: {cluster['identifier']}")

        print(f"Waiting for {cluster['arn']} to become ACTIVE")
        client.get_waiter("cluster_active").wait(
            identifier=cluster["identifier"],
            WaiterConfig={
                'Delay': 10,
                'MaxAttempts': 30
            }
        )

        return cluster
    except:
        print("Unable to create cluster")
        raise

def main():
    region = "us-east-1"
    response = create_cluster(region)
    print(f"Created cluster: {response['arn']}")

if __name__ == "__main__":
    main()
```

C++

The following example lets you create a cluster in a single AWS Region.

```
#include <aws/core/Aws.h>
```

```

#include <aws/core/utils/Outcome.h>
#include <aws/dsql/DSQLClient.h>
#include <aws/dsql/model/CreateClusterRequest.h>
#include <aws/dsql/model/GetClusterRequest.h>
#include <iostream>
#include <thread>
#include <chrono>

using namespace Aws;
using namespace Aws::DSQL;
using namespace Aws::DSQL::Model;

/**
 * Creates a single-region cluster in Amazon Aurora DSQL
 */
CreateClusterResult CreateCluster(const Aws::String& region) {
    // Create client for the specified region
    DSQL::DSQLClientConfiguration clientConfig;
    clientConfig.region = region;
    DSQL::DSQLClient client(clientConfig);

    // Create the cluster
    CreateClusterRequest createClusterRequest;
    createClusterRequest.SetDeletionProtectionEnabled(true);
    createClusterRequest.SetClientToken(Aws::Utils::UUID::RandomUUID());

    // Add tags
    Aws::Map<Aws::String, Aws::String> tags;
    tags["Name"] = "cpp single region cluster";
    createClusterRequest.SetTags(tags);

    auto createOutcome = client.CreateCluster(createClusterRequest);
    if (!createOutcome.IsSuccess()) {
        std::cerr << "Failed to create cluster in " << region << ": "
                  << createOutcome.GetError().GetMessage() << std::endl;
        throw std::runtime_error("Unable to create cluster in " + region);
    }

    auto cluster = createOutcome.GetResult();
    std::cout << "Created " << cluster.GetArn() << std::endl;

    return cluster;
}

```

```

int main() {
    Aws::SDKOptions options;
    Aws::InitAPI(options);
    {
        try {
            // Define region for the single-region setup
            Aws::String region = "us-east-1";

            auto cluster = CreateCluster(region);

            std::cout << "Created single region cluster:" << std::endl;
            std::cout << "Cluster ARN: " << cluster.GetArn() << std::endl;
            std::cout << "Cluster Status: " <<
ClusterStatusMapper::GetNameForClusterStatus(cluster.GetStatus()) << std::endl;
        }
        catch (const std::exception& e) {
            std::cerr << "Error: " << e.what() << std::endl;
        }
    }
    Aws::ShutdownAPI(options);
    return 0;
}

```

JavaScript

To create a cluster in a single AWS Region, use the following example.

```

import { DSQLClient, CreateClusterCommand, waitUntilClusterActive } from "@aws-sdk/
client-dsql";

async function createCluster(region) {

    const client = new DSQLClient({ region });

    try {
        const createClusterCommand = new CreateClusterCommand({
            deletionProtectionEnabled: true,
            tags: {
                Name: "javascript single region cluster"
            },
        });
        const response = await client.send(createClusterCommand);

        console.log(`Waiting for cluster ${response.identifier} to become ACTIVE`);
    }
}

```

```

        await waitUntilClusterActive(
            {
                client: client,
                maxWaitTime: 300 // Wait for 5 minutes
            },
            {
                identifier: response.identifier
            }
        );
        console.log(`Cluster Id ${response.identifier} is now active`);
        return;
    } catch (error) {
        console.error(`Unable to create cluster in ${region}: `, error.message);
        throw error;
    }
}

async function main() {
    const region = "us-east-1";

    await createCluster(region);
}

main();

```

Java

Use the following example to create a cluster in a single AWS Region.

```

package org.example;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.waiters.WaiterResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.retries.api.BackoffStrategy;
import software.amazon.awssdk.services.dsdl.DsdlClient;
import software.amazon.awssdk.services.dsdl.model.CreateClusterRequest;
import software.amazon.awssdk.services.dsdl.model.CreateClusterResponse;
import software.amazon.awssdk.services.dsdl.model.GetClusterResponse;

import java.time.Duration;
import java.util.Map;

public class CreateCluster {

```



```

public static void main(String[] args) {
    Region region = Region.US_EAST_1;

    try (
        DsqlClient client = DsqlClient.builder()
            .region(region)
            .credentialsProvider(DefaultCredentialsProvider.create())
            .build()
    ) {
        CreateClusterRequest request = CreateClusterRequest.builder()
            .deletionProtectionEnabled(true)
            .tags(Map.of("Name", "java single region cluster"))
            .build();
        CreateClusterResponse cluster = client.createCluster(request);
        System.out.println("Created " + cluster.arn());

        // The DSQL SDK offers a built-in waiter to poll for a cluster's
        // transition to ACTIVE.
        System.out.println("Waiting for cluster to become ACTIVE");
        WaiterResponse<GetClusterResponse> waiterResponse =
client.waiter().waitUntilClusterActive(
            getCluster -> getCluster.identifier(cluster.identifier()),
            config -> config.backoffStrategyV2(

BackoffStrategy.fixedDelayWithoutJitter(Duration.ofSeconds(10))
                ).waitTimeout(Duration.ofMinutes(5))
            );
        waiterResponse.matched().response().ifPresent(System.out::println);
    }
}

```

Rust

To create a cluster in a single AWS Region, use the following example.

```

use aws_config::{BehaviorVersion, Region, load_defaults};
use aws_sdk_dsql::client::Waiters;
use aws_sdk_dsql::operation::get_cluster::GetClusterOutput;
use aws_sdk_dsql::{Client, Config};
use std::collections::HashMap;

```

```

/// Create a client. We will use this later for performing operations on the
cluster.
async fn dsql_client(region: &'static str) -> Client {
    let region_provider = Region::new(region);

    let config = load_defaults(BehaviorVersion::latest())
        .region(region_provider)
        .load()
        .await;

    let config = Config::new(&config);

    Client::from_conf(config)
}

/// Create a cluster without delete protection and a name
pub async fn create_cluster(region: &'static str) -> GetClusterOutput {
    let client = dsql_client(region).await;

    let tags = HashMap::from([
        (String::from("Name"), String::from("rust single region cluster")),
    ]);

    println!("Creating cluster in {region}");
    let cluster = client
        .create_cluster()
        .set_tags(Some(tags))
        .deletion_protection_enabled(true)
        .send()
        .await
        .unwrap();

    println!("Created {}", cluster.arn);

    println!("Waiting for {} to become ACTIVE", cluster.arn);
    let cluster_output = client
        .wait_until_cluster_active()
        .identifier(&cluster.identifier)
        .send()
        .await
        .unwrap();

    cluster_output
}

```

```
#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let region = "us-east-1";

    let cluster = create_cluster(region).await;

    println!("Created single region cluster:");
    println!("{:#?}", cluster);

    Ok(())
}
```

Ruby

To create a cluster in a single AWS Region, use the following example.

```
require "aws-sdk-dsql"
require "pp"

def create_cluster(region)
  client = Aws::DSQL::Client.new(region: region)

  puts "Creating cluster in #{region}"
  cluster = client.create_cluster(
    deletion_protection_enabled: true,
    tags: {
      Name: "ruby single region cluster"
    }
  )
  puts "Created #{cluster.arn}"

  puts "Waiting for #{cluster.arn} to become ACTIVE"
  cluster = client.wait_until(:cluster_active, identifier: cluster.identifier) do |
w|
    # Wait for 5 minutes
    w.max_attempts = 30
    w.delay = 10
  end

  cluster
rescue Aws::Errors::ServiceError => e
  abort "Failed to create cluster: #{e.message}"
end
```

```
def main
  region = "us-east-1"

  cluster = create_cluster(region)

  puts "Created single region cluster:"
  pp cluster
end

main if $PROGRAM_NAME == __FILE__
```

.NET

To create a cluster in a single AWS Region, use the following example.

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Amazon;
using Amazon.DSQL;
using Amazon.DSQL.Model;
using Amazon.Runtime;
using Amazon.Runtime.Credentials;
using Amazon.Runtime.Endpoints;

namespace DSQLExamples.examples
{
    public class CreateCluster
    {
        /// <summary>
        /// Create a client. We will use this later for performing operations on the
        cluster.
        /// </summary>
        private static async Task<AmazonDSQLClient> CreateDSQLClient(RegionEndpoint
region)
        {
            var awsCredentials = new DefaultAWSCredentialsChain().GetCredentials();
            var clientConfig = new AmazonDSQLConfig
            {
                RegionEndpoint = region
            };
            return new AmazonDSQLClient(awsCredentials, clientConfig);
        }
    }
}
```

```

    /// <summary>
    /// Create a cluster with deletion protection enabled and a name tag.
    /// </summary>
    public static async Task<CreateClusterResponse> Create(RegionEndpoint
region)
    {
        using (var client = await CreateDSQLClient(region))
        {
            var tags = new Dictionary<string, string>
            {
                { "Name", "csharp single region cluster" }
            };

            var createClusterRequest = new CreateClusterRequest
            {
                DeletionProtectionEnabled = true,
                Tags = tags
            };

            var cluster = await client.CreateClusterAsync(createClusterRequest);
            Console.WriteLine($"Created {cluster.Arn}");

            return cluster;
        }
    }

    public static async Task Main()
    {
        var region = RegionEndpoint.USEast1;

        var cluster = await Create(region);

        Console.WriteLine("Created single region cluster:");
        Console.WriteLine($"Cluster ARN: {cluster.Arn}");
    }
}

```

Golang

To create a cluster in a single AWS Region, use the following example.

```
package main
```

```

import (
    "context"
    "fmt"
    "log"
    "time"

    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/config"
    "github.com/aws/aws-sdk-go-v2/service/dsql"
)

func CreateCluster(ctx context.Context, region string) error {

    cfg, err := config.LoadDefaultConfig(ctx, config.WithRegion(region))
    if err != nil {
        log.Fatalf("Failed to load AWS configuration: %v", err)
    }

    client := dsql.NewFromConfig(cfg)

    deleteProtect := true

    input := &dsql.CreateClusterInput{
        DeletionProtectionEnabled: &deleteProtect,
        Tags: map[string]string{
            "Name": "go single-region cluster",
        },
    }

    clusterProperties, err := client.CreateCluster(context.Background(), input)

    if err != nil {
        return fmt.Errorf("failed to create cluster. %v", err)
    }

    // Create the waiter with our custom options
    waiter := dsql.NewClusterActiveWaiter(client, func(o
    *dsql.ClusterActiveWaiterOptions) {
        o.MaxDelay = 30 * time.Second
        o.MinDelay = 10 * time.Second
        o.LogWaitAttempts = true
    })

```

```
// Create the input for the clusterProperties to monitor
clusterInput := &dsql.GetClusterInput{
    Identifier: clusterProperties.Identifier,
}

fmt.Printf("Waiting for cluster %s to become ACTIVE\n", *clusterProperties.Arn)
err = waiter.Wait(ctx, clusterInput, 5*time.Minute)
if err != nil {
    return fmt.Errorf("error waiting for cluster to become active: %w", err)
}

fmt.Printf("Created single region cluster: %s\n", *clusterProperties.Arn)
return nil
}

func main() {
    // Set up context with timeout
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Minute)
    defer cancel()

    err := CreateCluster(ctx, "us-east-1")
    if err != nil {
        fmt.Printf("failed to create cluster: %v", err)
        panic(err)
    }
}
}
```

Get cluster

The following examples show how to get information about a single-Region cluster using different programming languages.

Python

To get information about a single-Region cluster, use the following example.

```
import boto3
from datetime import datetime
import json
```

```
def get_cluster(region, identifier):
    try:
        client = boto3.client("dsql", region_name=region)
        return client.get_cluster(identifier=identifier)
    except:
        print(f"Unable to get cluster {identifier} in region {region}")
        raise

def main():
    region = "us-east-1"
    cluster_id = "<your cluster id>"
    response = get_cluster(region, cluster_id)

    print(json.dumps(response, indent=2, default=lambda obj: obj.isoformat() if
    isinstance(obj, datetime) else None))

if __name__ == "__main__":
    main()
```

C++

Use the following example to get information about a single-Region cluster.

```
#include <aws/core/Aws.h>
#include <aws/core/utils/Outcome.h>
#include <aws/dsql/DSQLClient.h>
#include <aws/dsql/model/GetClusterRequest.h>
#include <iostream>

using namespace Aws;
using namespace Aws::DSQL;
using namespace Aws::DSQL::Model;

/**
 * Retrieves information about a cluster in Amazon Aurora DSQL
 */
GetClusterResult GetCluster(const Aws::String& region, const Aws::String&
identifier) {
    // Create client for the specified region
    DSQL::DSQLClientConfiguration clientConfig;
    clientConfig.region = region;
```



```

    DSQL::DSQLClient client(clientConfig);

    // Get the cluster
    GetClusterRequest getClusterRequest;
    getClusterRequest.SetIdentifier(identifier);

    auto getOutcome = client.GetCluster(getClusterRequest);
    if (!getOutcome.IsSuccess()) {
        std::cerr << "Failed to retrieve cluster " << identifier << " in " << region
        << ": "
            << getOutcome.GetError().GetMessage() << std::endl;
        throw std::runtime_error("Unable to retrieve cluster " + identifier + " in
region " + region);
    }

    return getOutcome.GetResult();
}

int main() {
    Aws::SDKOptions options;
    Aws::InitAPI(options);
    {
        try {
            // Define region and cluster ID
            Aws::String region = "us-east-1";
            Aws::String clusterId = "<your cluster id>";

            auto cluster = GetCluster(region, clusterId);

            // Print cluster details
            std::cout << "Cluster Details:" << std::endl;
            std::cout << "ARN: " << cluster.GetArn() << std::endl;
            std::cout << "Status: " <<
ClusterStatusMapper::GetNameForClusterStatus(cluster.GetStatus()) << std::endl;
        }
        catch (const std::exception& e) {
            std::cerr << "Error: " << e.what() << std::endl;
        }
    }
    Aws::ShutdownAPI(options);
    return 0;
}

```

JavaScript

To get information about a single-Region cluster, use the following example.

```
import { DSQLClient, GetClusterCommand } from "@aws-sdk/client-dsql";

async function getCluster(region, clusterId) {

  const client = new DSQLClient({ region });

  const getClusterCommand = new GetClusterCommand({
    identifier: clusterId,
  });

  try {
    return await client.send(getClusterCommand);
  } catch (error) {
    if (error.name === "ResourceNotFoundException") {
      console.log("Cluster ID not found or deleted");
    }
    throw error;
  }
}

async function main() {
  const region = "us-east-1";
  const clusterId = "<CLUSTER_ID>";

  const response = await getCluster(region, clusterId);
  console.log("Cluster: ", response);
}

main();
```

Java

The following example lets you get information about a single-Region cluster.

```
package org.example;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dsql.DsqlClient;
```

```

import software.amazon.awssdk.services.dsql.model.GetClusterResponse;
import software.amazon.awssdk.services.dsql.model.ResourceNotFoundException;

public class GetCluster {

    public static void main(String[] args) {
        Region region = Region.US_EAST_1;
        String clusterId = "<your cluster id>";

        try (
            DsqlClient client = DsqlClient.builder()
                .region(region)
                .credentialsProvider(DefaultCredentialsProvider.create())
                .build()
        ) {
            GetClusterResponse cluster = client.getCluster(r ->
r.identifier(clusterId));
            System.out.println(cluster);
        } catch (ResourceNotFoundException e) {
            System.out.printf("Cluster %s not found in %s\n", clusterId, region);
        }
    }
}

```

Rust

The following example lets you get information about a single-Region cluster.

```

use aws_config::load_defaults;
use aws_sdk_dsql::operation::get_cluster::GetClusterOutput;
use aws_sdk_dsql::{
    Client, Config,
    config::{BehaviorVersion, Region},
};

/// Create a client. We will use this later for performing operations on the
/// cluster.
async fn dsql_client(region: &'static str) -> Client {
    // Load default SDK configuration
    let sdk_defaults = load_defaults(BehaviorVersion::latest()).await;

    // You can set your own credentials by following this guide
    // https://docs.aws.amazon.com/sdk-for-rust/latest/dg/credproviders.html

```

```

    let credentials = sdk_defaults.credentials_provider().unwrap();

    let config = Config::builder()
        .behavior_version(BehaviorVersion::latest())
        .credentials_provider(credentials)
        .region(Region::new(region))
        .build();

    Client::from_conf(config)
}

/// Get a ClusterResource from DSQL cluster identifier
pub async fn get_cluster(region: &'static str, identifier: &'static str) ->
    GetClusterOutput {
    let client = dsql_client(region).await;
    client
        .get_cluster()
        .identifier(identifier)
        .send()
        .await
        .unwrap()
}

#[tokio::main(flavor = "current_thread")]
pub async fn main() -> anyhow::Result<()> {
    let region = "us-east-1";

    let cluster = get_cluster(region, "<your cluster id>").await;
    println!("{:?}", cluster);

    Ok(())
}

```

Ruby

The following example lets you get information about a single-Region cluster.

```

require "aws-sdk-dsql"
require "pp"

def get_cluster(region, identifier)
  client = Aws::DSQL::Client.new(region: region)
  client.get_cluster(identifier: identifier)

```

```

rescue Aws::Errors::ServiceError => e
  abort "Unable to retrieve cluster #{identifier} in region #{region}: #{e.message}"
end

def main
  region = "us-east-1"
  cluster_id = "<your cluster id>"
  cluster = get_cluster(region, cluster_id)
  pp cluster
end

main if $PROGRAM_NAME == __FILE__

```

.NET

The following example lets you get information about a single-Region cluster.

```

using System;
using System.Threading.Tasks;
using Amazon;
using Amazon.DSQL;
using Amazon.DSQL.Model;
using Amazon.Runtime.Credentials;

namespace DSQLExamples.examples
{
    public class GetCluster
    {
        /// <summary>
        /// Create a client. We will use this later for performing operations on the
        cluster.
        /// </summary>
        private static async Task<AmazonDSQIClient> CreateDSQIClient(RegionEndpoint
region)
        {
            var awsCredentials = await
DefaultAWSCredentialsIdentityResolver.GetCredentialsAsync();
            var clientConfig = new AmazonDSQLConfig
            {
                RegionEndpoint = region
            };
            return new AmazonDSQIClient(awsCredentials, clientConfig);
        }
    }
}

```

```

    /// <summary>
    /// Get information about a DSQL cluster.
    /// </summary>
    public static async Task<GetClusterResponse> Get(RegionEndpoint region,
string identifier)
    {
        using (var client = await CreateDSQLClient(region))
        {
            var getClusterRequest = new GetClusterRequest
            {
                Identifier = identifier
            };

            return await client.GetClusterAsync(getClusterRequest);
        }
    }

    private static async Task Main()
    {
        var region = RegionEndpoint.USEast1;
        var clusterId = "<your cluster id>";

        var response = await Get(region, clusterId);
        Console.WriteLine($"Cluster ARN: {response.Arn}");
    }
}

```

Golang

The following example lets you get information about a single-Region cluster.

```

package main

import (
    "context"
    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/config"
    "log"
    "time"

    "github.com/aws/aws-sdk-go-v2/service/dsql"

```

```

)

func GetCluster(ctx context.Context, region, identifier string) (clusterStatus
    *dsql.GetClusterOutput, err error) {

    cfg, err := config.LoadDefaultConfig(ctx, config.WithRegion(region))
    if err != nil {
        log.Fatalf("Failed to load AWS configuration: %v", err)
    }

    // Initialize the DSQL client
    client := dsql.NewFromConfig(cfg)

    input := &dsql.GetClusterInput{
        Identifier: aws.String(identifier),
    }
    clusterStatus, err = client.GetCluster(context.Background(), input)

    if err != nil {
        log.Fatalf("Failed to get cluster: %v", err)
    }

    log.Printf("Cluster ARN: %s", *clusterStatus.Arn)

    return clusterStatus, nil
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 6*time.Minute)
    defer cancel()

    // Example cluster identifier
    identifier := "<CLUSTER_ID>"
    region := "us-east-1"

    _, err := GetCluster(ctx, region, identifier)
    if err != nil {
        log.Fatalf("Failed to get cluster: %v", err)
    }
}

```

Update cluster

The following examples show how to update a single-Region cluster using different programming languages.

Python

To update a single-Region cluster, use the following example.

```
import boto3

def update_cluster(region, cluster_id, deletion_protection_enabled):
    try:
        client = boto3.client("dsql", region_name=region)
        return client.update_cluster(identifier=cluster_id,
        deletionProtectionEnabled=deletion_protection_enabled)
    except:
        print("Unable to update cluster")
        raise

def main():
    region = "us-east-1"
    cluster_id = "<your cluster id>"
    deletion_protection_enabled = False
    response = update_cluster(region, cluster_id, deletion_protection_enabled)
    print(f"Updated {response['arn']} with deletion_protection_enabled:
    {deletion_protection_enabled}")

if __name__ == "__main__":
    main()
```

C++

Use the following example to update a single-Region cluster.

```
#include <aws/core/Aws.h>
#include <aws/core/Utils/Outcome.h>
#include <aws/dsql/DSQLClient.h>
#include <aws/dsql/model/UpdateClusterRequest.h>
```



```

#include <iostream>

using namespace Aws;
using namespace Aws::DSQL;
using namespace Aws::DSQL::Model;

/**
 * Updates a cluster in Amazon Aurora DSQL
 */
UpdateClusterResult UpdateCluster(const Aws::String& region, const
    Aws::Map<Aws::String, Aws::String>& updateParams) {
    // Create client for the specified region
    DSQL::DSQLClientConfiguration clientConfig;
    clientConfig.region = region;
    DSQL::DSQLClient client(clientConfig);

    // Create update request
    UpdateClusterRequest updateRequest;
    updateRequest.SetClientToken(Aws::Utils::UUID::RandomUUID());

    // Set identifier (required)
    if (updateParams.find("identifier") != updateParams.end()) {
        updateRequest.SetIdentifier(updateParams.at("identifier"));
    } else {
        throw std::runtime_error("Cluster identifier is required for update
operation");
    }

    // Set deletion protection if specified
    if (updateParams.find("deletion_protection_enabled") != updateParams.end()) {
        bool deletionProtection = (updateParams.at("deletion_protection_enabled") ==
"true");
        updateRequest.SetDeletionProtectionEnabled(deletionProtection);
    }

    // Execute the update
    auto updateOutcome = client.UpdateCluster(updateRequest);
    if (!updateOutcome.IsSuccess()) {
        std::cerr << "Failed to update cluster: " <<
updateOutcome.GetError().GetMessage() << std::endl;
        throw std::runtime_error("Unable to update cluster");
    }

    return updateOutcome.GetResult();
}

```

```

}

int main() {
    Aws::SDKOptions options;
    Aws::InitAPI(options);
    {
        try {
            // Define region and update parameters
            Aws::String region = "us-east-1";
            Aws::String clusterId = "<your cluster id>";

            // Create parameter map
            Aws::Map<Aws::String, Aws::String> updateParams;
            updateParams["identifier"] = clusterId;
            updateParams["deletion_protection_enabled"] = "false";

            auto updatedCluster = UpdateCluster(region, updateParams);

            std::cout << "Updated " << updatedCluster.GetArn() << std::endl;
        }
        catch (const std::exception& e) {
            std::cerr << "Error: " << e.what() << std::endl;
        }
    }
    Aws::ShutdownAPI(options);
    return 0;
}

```

JavaScript

To update a single-Region cluster, use the following example.

```

import { DSQLClient, UpdateClusterCommand } from "@aws-sdk/client-dsql";

export async function updateCluster(region, clusterId, deletionProtectionEnabled) {

    const client = new DSQLClient({ region });

    const updateClusterCommand = new UpdateClusterCommand({
        identifier: clusterId,
        deletionProtectionEnabled: deletionProtectionEnabled
    });
}

```

```

    try {
        return await client.send(updateClusterCommand);
    } catch (error) {
        console.error("Unable to update cluster", error.message);
        throw error;
    }
}

async function main() {
    const region = "us-east-1";
    const clusterId = "<CLUSTER_ID>";
    const deletionProtectionEnabled = false;

    const response = await updateCluster(region, clusterId,
    deletionProtectionEnabled);
    console.log(`Updated ${response.arn}`);
}

main();

```

Java

Use the following example to update a single-Region cluster.

```

package org.example;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dsqli.DsqliClient;
import software.amazon.awssdk.services.dsqli.model.UpdateClusterRequest;
import software.amazon.awssdk.services.dsqli.model.UpdateClusterResponse;

public class UpdateCluster {

    public static void main(String[] args) {
        Region region = Region.US_EAST_1;
        String clusterId = "<your cluster id>";

        try (
            DsqliClient client = DsqliClient.builder()
                .region(region)
                .credentialsProvider(DefaultCredentialsProvider.create())
                .build()
        ) {

```

```

        UpdateClusterRequest request = UpdateClusterRequest.builder()
            .identifier(clusterId)
            .deletionProtectionEnabled(false)
            .build();
        UpdateClusterResponse cluster = client.updateCluster(request);
        System.out.println("Updated " + cluster.arn());
    }
}
}

```

Rust

Use the following example to update a single-Region cluster.

```

use aws_config::load_defaults;
use aws_sdk_dsql::operation::update_cluster::UpdateClusterOutput;
use aws_sdk_dsql::{
    Client, Config,
    config::{BehaviorVersion, Region},
};

/// Create a client. We will use this later for performing operations on the
/// cluster.
async fn dsq_client(region: &'static str) -> Client {
    // Load default SDK configuration
    let sdk_defaults = load_defaults(BehaviorVersion::latest()).await;

    // You can set your own credentials by following this guide
    // https://docs.aws.amazon.com/sdk-for-rust/latest/dg/credproviders.html
    let credentials = sdk_defaults.credentials_provider().unwrap();

    let config = Config::builder()
        .behavior_version(BehaviorVersion::latest())
        .credentials_provider(credentials)
        .region(Region::new(region))
        .build();

    Client::from_conf(config)
}

/// Update a DSQL cluster and set delete protection to false. Also add new tags.
pub async fn update_cluster(region: &'static str, identifier: &'static str) ->
    UpdateClusterOutput {
    let client = dsq_client(region).await;

```

```
// Update delete protection
let update_response = client
    .update_cluster()
    .identifier(identifier)
    .deletion_protection_enabled(false)
    .send()
    .await
    .unwrap();

update_response
}

#[tokio::main(flavor = "current_thread")]
pub async fn main() -> anyhow::Result<()> {
    let region = "us-east-1";

    let cluster = update_cluster(region, "<your cluster id>").await;
    println!("{}", cluster);

    Ok(())
}
```

Ruby

Use the following example to update a single-Region cluster.

```
require "aws-sdk-dsql"

def update_cluster(region, update_params)
  client = Aws::DSQL::Client.new(region: region)
  client.update_cluster(update_params)
rescue Aws::Errors::ServiceError => e
  abort "Unable to update cluster: #{e.message}"
end

def main
  region = "us-east-1"
  cluster_id = "<your cluster id>"
  updated_cluster = update_cluster(region, {
    identifier: cluster_id,
    deletion_protection_enabled: false
  })
  puts "Updated #{updated_cluster.arn}"
end
```

```
main if $PROGRAM_NAME == __FILE__
```

.NET

Use the following example to update a single-Region cluster.

```
using System;
using System.Threading.Tasks;
using Amazon;
using Amazon.DSQL;
using Amazon.DSQL.Model;
using Amazon.Runtime.Credentials;

namespace DSQLExamples.examples
{
    public class UpdateCluster
    {
        /// <summary>
        /// Create a client. We will use this later for performing operations on the
        cluster.
        /// </summary>
        private static async Task<AmazonDSQLClient> CreateDSQLClient(RegionEndpoint
region)
        {
            var awsCredentials = await
DefaultAWSCredentialsIdentityResolver.GetCredentialsAsync();
            var clientConfig = new AmazonDSQLConfig
            {
                RegionEndpoint = region
            };
            return new AmazonDSQLClient(awsCredentials, clientConfig);
        }

        /// <summary>
        /// Update a DSQL cluster and set delete protection to false.
        /// </summary>
        public static async Task<UpdateClusterResponse> Update(RegionEndpoint
region, string identifier)
        {
            using (var client = await CreateDSQLClient(region))
            {
                var updateClusterRequest = new UpdateClusterRequest
```

```

        {
            Identifier = identifier,
            DeletionProtectionEnabled = false
        };

        UpdateClusterResponse response = await
client.UpdateClusterAsync(updateClusterRequest);
        Console.WriteLine($"Updated {response.Arn}");

        return response;
    }
}

private static async Task Main()
{
    var region = RegionEndpoint.USEast1;
    var clusterId = "<your cluster id>";

    await Update(region, clusterId);
}
}
}

```

Golang

Use the following example to update a single-Region cluster.

```

package main

import (
    "context"
    "github.com/aws/aws-sdk-go-v2/config"
    "log"
    "time"

    "github.com/aws/aws-sdk-go-v2/service/dsql"
)

func UpdateCluster(ctx context.Context, region, id string, deleteProtection bool)
(clusterStatus *dsql.UpdateClusterOutput, err error) {

    cfg, err := config.LoadDefaultConfig(ctx, config.WithRegion(region))
    if err != nil {
        log.Fatalf("Failed to load AWS configuration: %v", err)
    }
}

```

```

}

// Initialize the DSQL client
client := dsql.NewFromConfig(cfg)

input := dsql.UpdateClusterInput{
    Identifier:          &id,
    DeletionProtectionEnabled: &deleteProtection,
}

clusterStatus, err = client.UpdateCluster(context.Background(), &input)

if err != nil {
    log.Fatalf("Failed to update cluster: %v", err)
}

log.Printf("Cluster updated successfully: %v", clusterStatus.Status)
return clusterStatus, nil
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 6*time.Minute)
    defer cancel()

    // Example cluster identifier
    identifier := "<CLUSTER_ID>"
    region := "us-east-1"
    deleteProtection := false

    _, err := UpdateCluster(ctx, region, identifier, deleteProtection)
    if err != nil {
        log.Fatalf("Failed to update cluster: %v", err)
    }
}

```

Delete cluster

The following examples show how to delete a single-Region cluster using different programming languages.

Python

To delete a cluster in a single AWS Region, use the following example.

```
import boto3

def delete_cluster(region, identifier):
    try:
        client = boto3.client("dsql", region_name=region)
        cluster = client.delete_cluster(identifier=identifier)
        print(f"Initiated delete of {cluster['arn']}")

        print("Waiting for cluster to finish deletion")
        client.get_waiter("cluster_not_exists").wait(
            identifier=cluster["identifier"],
            WaiterConfig={
                'Delay': 10,
                'MaxAttempts': 30
            }
        )
    except:
        print("Unable to delete cluster " + identifier)
        raise

def main():
    region = "us-east-1"
    cluster_id = "<cluster id>" # Use a placeholder in docs
    delete_cluster(region, cluster_id)
    print(f"Deleted {cluster_id}")

if __name__ == "__main__":
    main()
```

C++

To delete a cluster in a single AWS Region, use the following example.

```
#include <aws/core/Aws.h>
#include <aws/core/Utils/Outcome.h>
#include <aws/dsql/DSQLClient.h>
```

```

#include <aws/dsql/model/DeleteClusterRequest.h>
#include <aws/dsql/model/GetClusterRequest.h>
#include <iostream>
#include <thread>
#include <chrono>

using namespace Aws;
using namespace Aws::DSQL;
using namespace Aws::DSQL::Model;

/**
 * Deletes a single-region cluster in Amazon Aurora DSQL
 */
void DeleteCluster(const Aws::String& region, const Aws::String& identifier) {
    // Create client for the specified region
    DSQL::DSQLClientConfiguration clientConfig;
    clientConfig.region = region;
    DSQL::DSQLClient client(clientConfig);

    // Delete the cluster
    DeleteClusterRequest deleteRequest;
    deleteRequest.SetIdentifier(identifier);
    deleteRequest.SetClientToken(Aws::Utils::UUID::RandomUUID());

    auto deleteOutcome = client.DeleteCluster(deleteRequest);
    if (!deleteOutcome.IsSuccess()) {
        std::cerr << "Failed to delete cluster " << identifier << " in " << region
        << ": "
            << deleteOutcome.GetError().GetMessage() << std::endl;
        throw std::runtime_error("Unable to delete cluster " + identifier + " in " +
region);
    }

    auto cluster = deleteOutcome.GetResult();
    std::cout << "Initiated delete of " << cluster.GetArn() << std::endl;
}

int main() {
    Aws::SDKOptions options;
    Aws::InitAPI(options);
    {
        try {
            // Define region and cluster ID
            Aws::String region = "us-east-1";

```

```

        Aws::String clusterId = "<your cluster id>";

        DeleteCluster(region, clusterId);

        std::cout << "Deleted " << clusterId << std::endl;
    }
    catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
    }
}
Aws::ShutdownAPI(options);
return 0;
}

```

JavaScript

To delete a cluster in a single AWS Region, use the following example.

```

import { DSQLClient, DeleteClusterCommand, waitUntilClusterNotExists } from "@aws-
sdk/client-dsql";

async function deleteCluster(region, clusterId) {

    const client = new DSQLClient({ region });

    try {
        const deleteClusterCommand = new DeleteClusterCommand({
            identifier: clusterId,
        });
        const response = await client.send(deleteClusterCommand);

        console.log(`Waiting for cluster ${response.identifier} to finish deletion`);

        await waitUntilClusterNotExists(
            {
                client: client,
                maxWaitTime: 300 // Wait for 5 minutes
            },
            {
                identifier: response.identifier
            }
        );
        console.log(`Cluster Id ${response.identifier} is now deleted`);
        return;
    }
}

```

```

    } catch (error) {
      if (error.name === "ResourceNotFoundException") {
        console.log("Cluster ID not found or already deleted");
      } else {
        console.error("Unable to delete cluster: ", error.message);
      }
      throw error;
    }
  }

  async function main() {
    const region = "us-east-1";
    const clusterId = "<CLUSTER_ID>";

    await deleteCluster(region, clusterId);
  }

  main();

```

Java

To delete a cluster in a single AWS Region, use the following example.

```

package org.example;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.retries.api.BackoffStrategy;
import software.amazon.awssdk.services.dsdl.DsdlClient;
import software.amazon.awssdk.services.dsdl.model.DeleteClusterResponse;
import software.amazon.awssdk.services.dsdl.model.ResourceNotFoundException;

import java.time.Duration;

public class DeleteCluster {

    public static void main(String[] args) {
        Region region = Region.US_EAST_1;
        String clusterId = "<your cluster id>";

        try (
            DsdlClient client = DsdlClient.builder()
                .region(region)

```

```

        .credentialsProvider(DefaultCredentialsProvider.create())
        .build()
    ) {
        DeleteClusterResponse cluster = client.deleteCluster(r ->
r.identifier(clusterId));
        System.out.println("Initiated delete of " + cluster.arn());

        // The DSQL SDK offers a built-in waiter to poll for deletion.
        System.out.println("Waiting for cluster to finish deletion");
        client.waiter().waitUntilClusterNotExists(
            getCluster -> getCluster.identifier(clusterId),
            config -> config.backoffStrategyV2(
BackoffStrategy.fixedDelayWithoutJitter(Duration.ofSeconds(10))
            ).waitTimeout(Duration.ofMinutes(5))
        );
        System.out.println("Deleted " + cluster.arn());
    } catch (ResourceNotFoundException e) {
        System.out.printf("Cluster %s not found in %s\n", clusterId, region);
    }
}
}

```

Rust

To delete a cluster in a single AWS Region, use the following example.

```

use aws_config::load_defaults;
use aws_sdk_dsql::client::Waiters;
use aws_sdk_dsql::{
    Client, Config,
    config::{BehaviorVersion, Region},
};

/// Create a client. We will use this later for performing operations on the
/// cluster.
async fn dsql_client(region: &'static str) -> Client {
    // Load default SDK configuration
    let sdk_defaults = load_defaults(BehaviorVersion::latest()).await;

    // You can set your own credentials by following this guide
    // https://docs.aws.amazon.com/sdk-for-rust/latest/dg/credproviders.html
    let credentials = sdk_defaults.credentials_provider().unwrap();

```

```

    let config = Config::builder()
      .behavior_version(BehaviorVersion::latest())
      .credentials_provider(credentials)
      .region(Region::new(region))
      .build();

    Client::from_conf(config)
  }

  /// Delete a DSQL cluster
  pub async fn delete_cluster(region: &'static str, identifier: &'static str) {
    let client = dsql_client(region).await;
    let delete_response = client
      .delete_cluster()
      .identifier(identifier)
      .send()
      .await
      .unwrap();
    println!("Initiated delete of {}", delete_response.arn);

    println!("Waiting for cluster to finish deletion");
    client
      .wait_until_cluster_not_exists()
      .identifier(identifier)
      .wait(std::time::Duration::from_secs(300)) // Wait up to 5 minutes
      .await
      .unwrap();
  }

  #[tokio::main(flavor = "current_thread")]
  pub async fn main() -> anyhow::Result<()> {
    let region = "us-east-1";
    let cluster_id = "<cluster to be deleted>";

    delete_cluster(region, cluster_id).await;
    println!("Deleted {cluster_id}");

    Ok(())
  }

```

Ruby

To delete a cluster in a single AWS Region, use the following example.

```
require "aws-sdk-dsql"

def delete_cluster(region, identifier)
  client = Aws::DSQL::Client.new(region: region)
  cluster = client.delete_cluster(identifier: identifier)
  puts "Initiated delete of #{cluster.arn}"

  # The DSQL SDK offers built-in waiters to poll for deletion.
  puts "Waiting for cluster to finish deletion"
  client.wait_until(:cluster_not_exists, identifier: cluster.identifier) do |w|
    # Wait for 5 minutes
    w.max_attempts = 30
    w.delay = 10
  end
rescue Aws::Errors::ServiceError => e
  abort "Unable to delete cluster #{identifier} in #{region}: #{e.message}"
end

def main
  region = "us-east-1"
  cluster_id = "<your cluster id>"
  delete_cluster(region, cluster_id)
  puts "Deleted #{cluster_id}"
end

main if $PROGRAM_NAME == __FILE__
```

.NET

To delete a cluster in a single AWS Region, use the following example.

```
using System;
using System.Threading.Tasks;
using Amazon;
using Amazon.DSQL;
using Amazon.DSQL.Model;
using Amazon.Runtime.Credentials;

namespace DSQLExamples.examples
{
    public class DeleteSingleRegionCluster
    {
        /// <summary>
```

```

        /// Create a client. We will use this later for performing operations on the
        cluster.
        /// </summary>
        private static async Task<AmazonDSQLErrorClient> CreateDSQLErrorClient(RegionEndpoint
region)
        {
            var awsCredentials = await
DefaultAWSCredentialsIdentityResolver.GetCredentialsAsync();
            var clientConfig = new AmazonDSQLErrorConfig
            {
                RegionEndpoint = region
            };
            return new AmazonDSQLErrorClient(awsCredentials, clientConfig);
        }

        /// <summary>
        /// Delete a DSQL cluster.
        /// </summary>
        public static async Task Delete(RegionEndpoint region, string identifier)
        {
            using (var client = await CreateDSQLErrorClient(region))
            {
                var deleteRequest = new DeleteClusterRequest
                {
                    Identifier = identifier
                };

                var deleteResponse = await client.DeleteClusterAsync(deleteRequest);
                Console.WriteLine($"Initiated deletion of {deleteResponse.Arn}");
            }
        }

        private static async Task Main()
        {
            var region = RegionEndpoint.USEast1;
            var clusterId = "<cluster to be deleted>";

            await Delete(region, clusterId);
        }
    }
}

```


Golang

To delete a cluster in a single AWS Region, use the following example.

```
package main

import (
    "context"
    "fmt"
    "log"
    "time"

    "github.com/aws/aws-sdk-go-v2/config"
    "github.com/aws/aws-sdk-go-v2/service/dsql"
)

func DeleteSingleRegion(ctx context.Context, identifier, region string) error {

    cfg, err := config.LoadDefaultConfig(ctx, config.WithRegion(region))
    if err != nil {
        log.Fatalf("Failed to load AWS configuration: %v", err)
    }

    // Initialize the DSQL client
    client := dsql.NewFromConfig(cfg)

    // Create delete cluster input
    deleteInput := &dsql.DeleteClusterInput{
        Identifier: &identifier,
    }

    // Delete the cluster
    result, err := client.DeleteCluster(ctx, deleteInput)
    if err != nil {
        return fmt.Errorf("failed to delete cluster: %w", err)
    }

    fmt.Printf("Initiated deletion of cluster: %s\n", *result.Arn)

    // Create waiter to check cluster deletion
    waiter := dsql.NewClusterNotExistsWaiter(client, func(options
    *dsql.ClusterNotExistsWaiterOptions) {
        options.MinDelay = 10 * time.Second
        options.MaxDelay = 30 * time.Second
    })
}
```

```

    options.LogWaitAttempts = true
}))

// Create the input for checking cluster status
getInput := &dsql.GetClusterInput{
    Identifier: &identifier,
}

// Wait for the cluster to be deleted
fmt.Printf("Waiting for cluster %s to be deleted...\n", identifier)
err = waiter.Wait(ctx, getInput, 5*time.Minute)
if err != nil {
    return fmt.Errorf("error waiting for cluster to be deleted: %w", err)
}

fmt.Printf("Cluster %s has been successfully deleted\n", identifier)
return nil
}

func DeleteCluster(ctx context.Context) {
}

// Example usage in main function
func main() {
    // Your existing setup code for client configuration...

    ctx, cancel := context.WithTimeout(context.Background(), 6*time.Minute)
    defer cancel()

    // Example cluster identifier
    // Need to make sure that cluster does not have delete protection enabled
    identifier := "<CLUSTER_ID>"
    region := "us-east-1"

    err := DeleteSingleRegion(ctx, identifier, region)
    if err != nil {
        log.Fatalf("Failed to delete cluster: %v", err)
    }
}

```

For more code samples and examples, visit the [Aurora DSQL Samples GitHub repository](#).

Using AWS CLI

The AWS CLI provides a command-line interface for managing your Aurora DSQL clusters. The following examples demonstrate common cluster management operations.

Create cluster

Create a cluster using the **create-cluster** command.

Note

Cluster creation is an asynchronous operation. Call the **GetCluster** API until the status changes to ACTIVE. You can connect to your cluster after it becomes active.

Example Command

```
aws dsq1 create-cluster --region us-east-1
```

Note

To disable deletion protection during creation, include the `--no-deletion-protection-enabled` flag.

Example Response

```
{
  "identifier": "abc0def1baz2quux3quuux4",
  "arn": "arn:aws:dsq1:us-east-1:111122223333:cluster/abc0def1baz2quux3quuux4",
  "status": "CREATING",
  "creationTime": "2024-05-25T16:56:49.784000-07:00",
  "deletionProtectionEnabled": true,
  "tag": {},
  "encryptionDetails": {
    "encryptionType": "AWS_OWNED_KMS_KEY",
    "encryptionStatus": "ENABLED"
  }
}
```

Describing a cluster

Get information about a cluster using the **get-cluster** command.

Example Command

```
aws dsq1 get-cluster \  
  --region us-east-1 \  
  --identifier your_cluster_id
```

Example Response

```
{  
  "identifier": "abc0def1baz2quux3quux4",  
  "arn": "arn:aws:dsq1:us-east-1:111122223333:cluster/abc0def1baz2quux3quux4",  
  "status": "ACTIVE",  
  "creationTime": "2024-11-27T00:32:14.434000-08:00",  
  "deletionProtectionEnabled": false,  
  "encryptionDetails": {  
    "encryptionType": "CUSTOMER_MANAGED_KMS_KEY",  
    "kmsKeyArn": "arn:aws:kms:us-east-1:111122223333:key/123a456b-c789-01de-2f34-g5hi6j7k8lm9",  
    "encryptionStatus": "ENABLED"  
  }  
}
```

Updating a cluster

Update an existing cluster using the **update-cluster** command.

Note

Updates are asynchronous operations. Call the **GetCluster** API until the status changes to ACTIVE to see your changes.

Example Command

```
aws dsq1 update-cluster \  
  --region us-east-1 \  
  --identifier your_cluster_id
```

```
--no-deletion-protection-enabled \  
--identifier your_cluster_id
```

Example Response

```
{  
  "identifier": "abc0def1baz2quux3quux4",  
  "arn": "arn:aws:dsq1:us-east-1:111122223333:cluster/abc0def1baz2quux3quux4",  
  "status": "UPDATING",  
  "creationTime": "2024-05-24T09:15:32.708000-07:00"  
}
```

Deleting a cluster

Delete an existing cluster using the **delete-cluster** command.

Note

You can only delete clusters that have deletion protection disabled. By default, deletion protection is enabled when you create new clusters.

Example Command

```
aws dsq1 delete-cluster \  
  --region us-east-1 \  
  --identifier your_cluster_id
```

Example Response

```
{  
  "identifier": "abc0def1baz2quux3quux4",  
  "arn": "arn:aws:dsq1:us-east-1:111122223333:cluster/abc0def1baz2quux3quux4",  
  "status": "DELETING",  
  "creationTime": "2024-05-24T09:16:43.778000-07:00"  
}
```

Listing clusters

List your clusters using the **list-clusters** command.

Example Command

```
aws dsq1 list-clusters --region us-east-1
```

Example Response

```
{
  "clusters": [
    {
      "identifier": "abc0def1baz2quux3quux4quuux",
      "arn": "arn:aws:dsq1:us-east-1:111122223333:cluster/abc0def1baz2quux3quux4quuux"
    },
    {
      "identifier": "abc0def1baz2quux3quux5quuuux",
      "arn": "arn:aws:dsq1:us-east-1:111122223333:cluster/abc0def1baz2quux3quux5quuuux"
    }
  ]
}
```

Configuring multi-Region clusters

Configure and manage clusters across multiple AWS Regions using either the AWS CLI or your preferred programming language including Python, C++, JavaScript, Java, Rust, Ruby, .NET, and Golang. The AWS CLI provides quick access through shell commands, while AWS SDKs enable programmatic control through native language support.

Topics

- [Using AWS SDKs](#)
- [Using AWS CLI](#)

Using AWS SDKs

The AWS SDKs provide programmatic access to Aurora DSQL in your preferred programming language. The following sections show how to perform common cluster operations using different programming languages.

Create cluster

The following examples show how to create a multi-Region cluster using different programming languages.

Python

To create a multi-Region cluster, use the following example. Creating a multi-Region cluster might take some time.

```
import boto3

def create_multi_region_clusters(region_1, region_2, witness_region):
    try:
        client_1 = boto3.client("dsql", region_name=region_1)
        client_2 = boto3.client("dsql", region_name=region_2)

        # We can only set the witness region for the first cluster
        cluster_1 = client_1.create_cluster(
            deletionProtectionEnabled=True,
            multiRegionProperties={"witnessRegion": witness_region},
            tags={"Name": "Python multi region cluster"}
        )
        print(f"Created {cluster_1["arn"]}")

        # For the second cluster we can set witness region and designate cluster_1
        # as a peer
        cluster_2 = client_2.create_cluster(
            deletionProtectionEnabled=True,
            multiRegionProperties={"witnessRegion": witness_region, "clusters":
[cluster_1["arn"]]},
            tags={"Name": "Python multi region cluster"}
        )

        print(f"Created {cluster_2["arn"]}")
        # Now that we know the cluster_2 arn we can set it as a peer of cluster_1
        client_1.update_cluster(
            identifier=cluster_1["identifier"],
            multiRegionProperties={"witnessRegion": witness_region, "clusters":
[cluster_2["arn"]]}
        )
        print(f"Added {cluster_2["arn"]} as a peer of {cluster_1["arn"]}")
```

```

    # Now that multiRegionProperties is fully defined for both clusters
    # they'll begin the transition to ACTIVE
    print(f"Waiting for {cluster_1["arn"]} to become ACTIVE")
    client_1.get_waiter("cluster_active").wait(
        identifier=cluster_1["identifier"],
        WaiterConfig={
            'Delay': 10,
            'MaxAttempts': 30
        }
    )

    print(f"Waiting for {cluster_2["arn"]} to become ACTIVE")
    client_2.get_waiter("cluster_active").wait(
        identifier=cluster_2["identifier"],
        WaiterConfig={
            'Delay': 10,
            'MaxAttempts': 30
        }
    )

    return (cluster_1, cluster_2)

except:
    print("Unable to create cluster")
    raise

def main():
    region_1 = "us-east-1"
    region_2 = "us-east-2"
    witness_region = "us-west-2"
    (cluster_1, cluster_2) = create_multi_region_clusters(region_1, region_2,
    witness_region)
    print("Created multi region clusters:")
    print("Cluster id: " + cluster_1['arn'])
    print("Cluster id: " + cluster_2['arn'])

if __name__ == "__main__":
    main()

```


C++

To create a multi-Region cluster, use the following example. Creating a multi-Region cluster might take some time.

```
#include <aws/core/Aws.h>
#include <aws/core/utils/Outcome.h>
#include <aws/dsql/DSQLClient.h>
#include <aws/dsql/model/CreateClusterRequest.h>
#include <aws/dsql/model/UpdateClusterRequest.h>
#include <aws/dsql/model/MultiRegionProperties.h>
#include <aws/dsql/model/GetClusterRequest.h>
#include <iostream>
#include <thread>
#include <chrono>

using namespace Aws;
using namespace Aws::DSQL;
using namespace Aws::DSQL::Model;

/**
 * Creates multi-region clusters in Amazon Aurora DSQL
 */
std::pair<CreateClusterResult, CreateClusterResult> CreateMultiRegionClusters(
    const Aws::String& region1,
    const Aws::String& region2,
    const Aws::String& witnessRegion) {

    // Create clients for each region
    DSQL::DSQLClientConfiguration clientConfig1;
    clientConfig1.region = region1;
    DSQL::DSQLClient client1(clientConfig1);

    DSQL::DSQLClientConfiguration clientConfig2;
    clientConfig2.region = region2;
    DSQL::DSQLClient client2(clientConfig2);

    std::cout << "Creating cluster in " << region1 << std::endl;

    CreateClusterRequest createClusterRequest1;
    createClusterRequest1.SetDeletionProtectionEnabled(true);

    // Set multi-region properties with witness region
    MultiRegionProperties multiRegionProps1;
```

```

multiRegionProps1.SetWitnessRegion(witnessRegion);
createClusterRequest1.SetMultiRegionProperties(multiRegionProps1);

// Add tags
Aws::Map<Aws::String, Aws::String> tags;
tags["Name"] = "cpp multi region cluster 1";
createClusterRequest1.SetTags(tags);
createClusterRequest1.SetClientToken(Aws::Utils::UUID::RandomUUID());

auto createOutcome1 = client1.CreateCluster(createClusterRequest1);
if (!createOutcome1.IsSuccess()) {
    std::cerr << "Failed to create cluster in " << region1 << ": "
               << createOutcome1.GetError().GetMessage() << std::endl;
    throw std::runtime_error("Failed to create multi-region clusters");
}

auto cluster1 = createOutcome1.GetResult();
std::cout << "Created " << cluster1.GetArn() << std::endl;

// Create second cluster
std::cout << "Creating cluster in " << region2 << std::endl;

CreateClusterRequest createClusterRequest2;
createClusterRequest2.SetDeletionProtectionEnabled(true);

// Set multi-region properties with witness region and cluster1 as peer
MultiRegionProperties multiRegionProps2;
multiRegionProps2.SetWitnessRegion(witnessRegion);

Aws::Vector<Aws::String> clusters;
clusters.push_back(cluster1.GetArn());
multiRegionProps2.SetClusters(clusters);

tags["Name"] = "cpp multi region cluster 2";
createClusterRequest2.SetMultiRegionProperties(multiRegionProps2);
createClusterRequest2.SetTags(tags);
createClusterRequest2.SetClientToken(Aws::Utils::UUID::RandomUUID());

auto createOutcome2 = client2.CreateCluster(createClusterRequest2);
if (!createOutcome2.IsSuccess()) {
    std::cerr << "Failed to create cluster in " << region2 << ": "
               << createOutcome2.GetError().GetMessage() << std::endl;
    throw std::runtime_error("Failed to create multi-region clusters");
}

```

```

auto cluster2 = createOutcome2.GetResult();
std::cout << "Created " << cluster2.GetArn() << std::endl;

// Now that we know the cluster2 arn we can set it as a peer of cluster1
UpdateClusterRequest updateClusterRequest;
updateClusterRequest.SetIdentifier(cluster1.GetIdentifier());

MultiRegionProperties updatedProps;
updatedProps.SetWitnessRegion(witnessRegion);

Aws::Vector<Aws::String> updatedClusters;
updatedClusters.push_back(cluster2.GetArn());
updatedProps.SetClusters(updatedClusters);

updateClusterRequest.SetMultiRegionProperties(updatedProps);
updateClusterRequest.SetClientToken(Aws::Utils::UUID::RandomUUID());

auto updateOutcome = client1.UpdateCluster(updateClusterRequest);
if (!updateOutcome.IsSuccess()) {
    std::cerr << "Failed to update cluster in " << region1 << ": "
               << updateOutcome.GetError().GetMessage() << std::endl;
    throw std::runtime_error("Failed to update multi-region clusters");
}

std::cout << "Added " << cluster2.GetArn() << " as a peer of " <<
cluster1.GetArn() << std::endl;

return std::make_pair(cluster1, cluster2);
}

int main() {
    Aws::SDKOptions options;
    Aws::InitAPI(options);
    {
        try {
            // Define regions for the multi-region setup
            Aws::String region1 = "us-east-1";
            Aws::String region2 = "us-east-2";
            Aws::String witnessRegion = "us-west-2";

            auto [cluster1, cluster2] = CreateMultiRegionClusters(region1, region2,
witnessRegion);

```

```

        std::cout << "Created multi region clusters:" << std::endl;
        std::cout << "Cluster 1 ARN: " << cluster1.GetArn() << std::endl;
        std::cout << "Cluster 2 ARN: " << cluster2.GetArn() << std::endl;
    }
    catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
    }
}
Aws::ShutdownAPI(options);
return 0;
}

```

JavaScript

To create a multi-Region cluster, use the following example. Creating a multi-Region cluster might take some time.

```

import { DSQLClient, CreateClusterCommand, UpdateClusterCommand,
  waitUntilClusterActive } from "@aws-sdk/client-dsql";

async function createMultiRegionCluster(region1, region2, witnessRegion) {

  const client1 = new DSQLClient({ region: region1 });
  const client2 = new DSQLClient({ region: region2 });

  try {
    // We can only set the witness region for the first cluster
    console.log(`Creating cluster in ${region1}`);
    const createClusterCommand1 = new CreateClusterCommand({
      deletionProtectionEnabled: true,
      tags: {
        Name: "javascript multi region cluster 1"
      },
      multiRegionProperties: {
        witnessRegion: witnessRegion
      }
    });
    const response1 = await client1.send(createClusterCommand1);
    console.log(`Created ${response1.arn}`);

    // For the second cluster we can set witness region and designate the first
    cluster as a peer
    console.log(`Creating cluster in ${region2}`);
  }
}

```

```

const createClusterCommand2 = new CreateClusterCommand({
  deletionProtectionEnabled: true,
  tags: {
    Name: "javascript multi region cluster 2"
  },
  multiRegionProperties: {
    witnessRegion: witnessRegion,
    clusters: [response1.arn]
  }
});
const response2 = await client2.send(createClusterCommand2);
console.log(`Created ${response2.arn}`);

// Now that we know the second cluster arn we can set it as a peer of the
first cluster
const updateClusterCommand = new UpdateClusterCommand({
  identifier: response1.identifier,
  multiRegionProperties: {
    witnessRegion: witnessRegion,
    clusters: [response2.arn]
  }
});
await client1.send(updateClusterCommand);
console.log(`Added ${response2.arn} as a peer of ${response1.arn}`);

// Now that multiRegionProperties is fully defined for both clusters they'll
begin the transition to ACTIVE
console.log(`Waiting for cluster ${response1.identifier} to become ACTIVE`);
await waitUntilClusterActive(
  {
    client: client1,
    maxWaitTime: 300 // Wait for 5 minutes
  },
  {
    identifier: response1.identifier
  }
);
console.log(`Cluster 1 is now active`);

console.log(`Waiting for cluster ${response2.identifier} to become ACTIVE`);
await waitUntilClusterActive(
  {
    client: client2,
    maxWaitTime: 300 // Wait for 5 minutes
  },
  {
    identifier: response2.identifier
  }
);

```

```

        },
        {
            identifier: response2.identifier
        }
    );
    console.log(`Cluster 2 is now active`);
    console.log("The multi region clusters are now active");
    return;
} catch (error) {
    console.error("Failed to create cluster: ", error.message);
    throw error;
}
}

async function main() {
    const region1 = "us-east-1";
    const region2 = "us-east-2";
    const witnessRegion = "us-west-2";

    await createMultiRegionCluster(region1, region2, witnessRegion);
}

main();

```

Java

To create a multi-Region cluster, use the following example. Creating a multi-Region cluster might take some time.

```

package org.example;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.retries.api.BackoffStrategy;
import software.amazon.awssdk.services.dsqli.DsqliClient;
import software.amazon.awssdk.services.dsqli.DsqliClientBuilder;
import software.amazon.awssdk.services.dsqli.model.CreateClusterRequest;
import software.amazon.awssdk.services.dsqli.model.CreateClusterResponse;
import software.amazon.awssdk.services.dsqli.model.GetClusterResponse;
import software.amazon.awssdk.services.dsqli.model.UpdateClusterRequest;

import java.time.Duration;
import java.util.Map;

```

```

public class CreateMultiRegionCluster {

    public static void main(String[] args) {
        Region region1 = Region.US_EAST_1;
        Region region2 = Region.US_EAST_2;
        Region witnessRegion = Region.US_WEST_2;

        DsqlClientBuilder clientBuilder = DsqlClient.builder()
            .credentialsProvider(DefaultCredentialsProvider.create());

        try {
            DsqlClient client1 = clientBuilder.region(region1).build();
            DsqlClient client2 = clientBuilder.region(region2).build()
        ) {
            // We can only set the witness region for the first cluster
            System.out.println("Creating cluster in " + region1);
            CreateClusterRequest request1 = CreateClusterRequest.builder()
                .deletionProtectionEnabled(true)
                .multiRegionProperties(mrp ->
mrp.witnessRegion(witnessRegion.toString()))
                .tags(Map.of("Name", "java multi region cluster"))
                .build();
            CreateClusterResponse cluster1 = client1.createCluster(request1);
            System.out.println("Created " + cluster1.arn());

            // For the second cluster we can set the witness region and designate
            // cluster1 as a peer.
            System.out.println("Creating cluster in " + region2);
            CreateClusterRequest request2 = CreateClusterRequest.builder()
                .deletionProtectionEnabled(true)
                .multiRegionProperties(mrp ->

mrp.witnessRegion(witnessRegion.toString()).clusters(cluster1.arn())
                )
                .tags(Map.of("Name", "java multi region cluster"))
                .build();
            CreateClusterResponse cluster2 = client2.createCluster(request2);
            System.out.println("Created " + cluster2.arn());

            // Now that we know the cluster2 ARN we can set it as a peer of cluster1
            UpdateClusterRequest updateReq = UpdateClusterRequest.builder()
                .identifier(cluster1.identifier())
                .multiRegionProperties(mrp ->

```

```

mrp.witnessRegion(witnessRegion.toString()).clusters(cluster2.arn())
    )
    .build();
    client1.updateCluster(updateReq);
    System.out.printf("Added %s as a peer of %s\n", cluster2.arn(),
cluster1.arn());

    // Now that MultiRegionProperties is fully defined for both clusters
they'll begin
    // the transition to ACTIVE.
    System.out.printf("Waiting for cluster %s to become ACTIVE\n",
cluster1.arn());
    GetClusterResponse activeCluster1 =
client1.waiter().waitUntilClusterActive(
        getCluster -> getCluster.identifier(cluster1.identifier()),
        config -> config.backoffStrategyV2(

BackoffStrategy.fixedDelayWithoutJitter(Duration.ofSeconds(10))
            ).waitTimeout(Duration.ofMinutes(5))
        ).matched().response().orElseThrow();

    System.out.printf("Waiting for cluster %s to become ACTIVE\n",
cluster2.arn());
    GetClusterResponse activeCluster2 =
client2.waiter().waitUntilClusterActive(
        getCluster -> getCluster.identifier(cluster2.identifier()),
        config -> config.backoffStrategyV2(

BackoffStrategy.fixedDelayWithoutJitter(Duration.ofSeconds(10))
            ).waitTimeout(Duration.ofMinutes(5))
        ).matched().response().orElseThrow();

    System.out.println("Created multi region clusters:");
    System.out.println(activeCluster1);
    System.out.println(activeCluster2);
    }
}
}

```


Rust

To create a multi-Region cluster, use the following example. Creating a multi-Region cluster might take some time.

```
use aws_config::{BehaviorVersion, Region, load_defaults};
use aws_sdk_dsql::client::Waiters;
use aws_sdk_dsql::operation::get_cluster::GetClusterOutput;
use aws_sdk_dsql::types::MultiRegionProperties;
use aws_sdk_dsql::{Client, Config};
use std::collections::HashMap;

/// Create a client. We will use this later for performing operations on the
/// cluster.
async fn dsql_client(region: &'static str) -> Client {
    // Load default SDK configuration
    let sdk_defaults = load_defaults(BehaviorVersion::latest()).await;

    // You can set your own credentials by following this guide
    // https://docs.aws.amazon.com/sdk-for-rust/latest/dg/credproviders.html
    let credentials = sdk_defaults.credentials_provider().unwrap();

    let config = Config::builder()
        .behavior_version(BehaviorVersion::latest())
        .credentials_provider(credentials)
        .region(Region::new(region))
        .build();

    Client::from_conf(config)
}

/// Create a cluster without delete protection and a name
pub async fn create_multi_region_clusters(
    region_1: &'static str,
    region_2: &'static str,
    witness_region: &'static str,
) -> (GetClusterOutput, GetClusterOutput) {
    let client_1 = dsql_client(region_1).await;
    let client_2 = dsql_client(region_2).await;

    let tags = HashMap::from([
        String::from("Name"),
        String::from("rust multi region cluster"),
    ]);
}
```

```
// We can only set the witness region for the first cluster
println!("Creating cluster in {region_1}");
let cluster_1 = client_1
    .create_cluster()
    .set_tags(Some(tags.clone()))
    .deletion_protection_enabled(true)
    .multi_region_properties(
        MultiRegionProperties::builder()
            .witness_region(witness_region)
            .build(),
    )
    .send()
    .await
    .unwrap();
let cluster_1_arn = &cluster_1.arn;
println!("Created {cluster_1_arn}");

// For the second cluster we can set witness region and designate cluster_1 as a
peer
println!("Creating cluster in {region_2}");
let cluster_2 = client_2
    .create_cluster()
    .set_tags(Some(tags))
    .deletion_protection_enabled(true)
    .multi_region_properties(
        MultiRegionProperties::builder()
            .witness_region(witness_region)
            .clusters(&cluster_1.arn)
            .build(),
    )
    .send()
    .await
    .unwrap();
let cluster_2_arn = &cluster_2.arn;
println!("Created {cluster_2_arn}");

// Now that we know the cluster_2 arn we can set it as a peer of cluster_1
client_1
    .update_cluster()
    .identifier(&cluster_1.identifier)
    .multi_region_properties(
        MultiRegionProperties::builder()
            .witness_region(witness_region)
```

```

        .clusters(&cluster_2.arn)
        .build(),
    )
    .send()
    .await
    .unwrap();
println!("Added {cluster_2_arn} as a peer of {cluster_1_arn}");

// Now that the multi-region properties are fully defined for both clusters
// they'll begin the transition to ACTIVE
println!("Waiting for {cluster_1_arn} to become ACTIVE");
let cluster_1_output = client_1
    .wait_until_cluster_active()
    .identifier(&cluster_1.identifier)
    .wait(std::time::Duration::from_secs(300)) // Wait up to 5 minutes
    .await
    .unwrap()
    .into_result()
    .unwrap();

println!("Waiting for {cluster_2_arn} to become ACTIVE");
let cluster_2_output = client_2
    .wait_until_cluster_active()
    .identifier(&cluster_2.identifier)
    .wait(std::time::Duration::from_secs(300)) // Wait up to 5 minutes
    .await
    .unwrap()
    .into_result()
    .unwrap();

    (cluster_1_output, cluster_2_output)
}

#[tokio::main(flavor = "current_thread")]
pub async fn main() -> anyhow::Result<()> {
    let region_1 = "us-east-1";
    let region_2 = "us-east-2";
    let witness_region = "us-west-2";

    let (cluster_1, cluster_2) =
        create_multi_region_clusters(region_1, region_2, witness_region).await;

    println!("Created multi region clusters:");
    println!("{:?}", cluster_1);

```

```
println!("{:#?}", cluster_2);

Ok(())
}
```

Ruby

To create a multi-Region cluster, use the following example. Creating a multi-Region cluster might take some time.

```
require "aws-sdk-dsql"
require "pp"

def create_multi_region_clusters(region_1, region_2, witness_region)
  client_1 = Aws::DSQL::Client.new(region: region_1)
  client_2 = Aws::DSQL::Client.new(region: region_2)

  # We can only set the witness region for the first cluster
  puts "Creating cluster in #{region_1}"
  cluster_1 = client_1.create_cluster(
    deletion_protection_enabled: true,
    multi_region_properties: {
      witness_region: witness_region
    },
    tags: {
      Name: "ruby multi region cluster"
    }
  )
  puts "Created #{cluster_1.arn}"

  # For the second cluster we can set witness region and designate cluster_1 as a
  peer
  puts "Creating cluster in #{region_2}"
  cluster_2 = client_2.create_cluster(
    deletion_protection_enabled: true,
    multi_region_properties: {
      witness_region: witness_region,
      clusters: [ cluster_1.arn ]
    },
    tags: {
      Name: "ruby multi region cluster"
    }
  )
end
```

```

)
puts "Created #{cluster_2.arn}"

# Now that we know the cluster_2 arn we can set it as a peer of cluster_1
client_1.update_cluster(
  identifier: cluster_1.identifier,
  multi_region_properties: {
    witness_region: witness_region,
    clusters: [ cluster_2.arn ]
  }
)
puts "Added #{cluster_2.arn} as a peer of #{cluster_1.arn}"

# Now that multi_region_properties is fully defined for both clusters
# they'll begin the transition to ACTIVE
puts "Waiting for #{cluster_1.arn} to become ACTIVE"
cluster_1 = client_1.wait_until(:cluster_active, identifier: cluster_1.identifier)
do |w|
  # Wait for 5 minutes
  w.max_attempts = 30
  w.delay = 10
end

puts "Waiting for #{cluster_2.arn} to become ACTIVE"
cluster_2 = client_2.wait_until(:cluster_active, identifier: cluster_2.identifier)
do |w|
  w.max_attempts = 30
  w.delay = 10
end

[ cluster_1, cluster_2 ]
rescue Aws::Errors::ServiceError => e
  abort "Failed to create multi-region clusters: #{e.message}"
end

def main
  region_1 = "us-east-1"
  region_2 = "us-east-2"
  witness_region = "us-west-2"

  cluster_1, cluster_2 = create_multi_region_clusters(region_1, region_2,
witness_region)

  puts "Created multi region clusters:"

```

```

    pp cluster_1
    pp cluster_2
end

main if $PROGRAM_NAME == __FILE__

```

Golang

To create a multi-Region cluster, use the following example. Creating a multi-Region cluster might take some time.

```

package main

import (
    "context"
    "fmt"
    "log"
    "time"

    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/config"
    "github.com/aws/aws-sdk-go-v2/service/dsql"
    dtypes "github.com/aws/aws-sdk-go-v2/service/dsql/types"
)

func CreateMultiRegionClusters(ctx context.Context, witness, region1, region2
    string) error {

    cfg, err := config.LoadDefaultConfig(ctx, config.WithRegion(region1))
    if err != nil {
        log.Fatalf("Failed to load AWS configuration: %v", err)
    }

    // Create a DSQL region 1 client
    client := dsql.NewFromConfig(cfg)

    cfg2, err := config.LoadDefaultConfig(ctx, config.WithRegion(region2))
    if err != nil {
        log.Fatalf("Failed to load AWS configuration: %v", err)
    }

    // Create a DSQL region 2 client

```

```

client2 := dsql.NewFromConfig(cfg2, func(o *dsql.Options) {
    o.Region = region2
})

// Create cluster
deleteProtect := true

// We can only set the witness region for the first cluster
input := &dsql.CreateClusterInput{
    DeletionProtectionEnabled: &deleteProtect,
    MultiRegionProperties: &dtypes.MultiRegionProperties{
        WitnessRegion: aws.String(witness),
    },
    Tags: map[string]string{
        "Name": "go multi-region cluster",
    },
}

clusterProperties, err := client.CreateCluster(context.Background(), input)

if err != nil {
    return fmt.Errorf("failed to create first cluster: %v", err)
}

// create second cluster
cluster2Arns := []string{*clusterProperties.Arn}

// For the second cluster we can set witness region and designate the first cluster
as a peer
input2 := &dsql.CreateClusterInput{
    DeletionProtectionEnabled: &deleteProtect,
    MultiRegionProperties: &dtypes.MultiRegionProperties{
        WitnessRegion: aws.String("us-west-2"),
        Clusters:      cluster2Arns,
    },
    Tags: map[string]string{
        "Name": "go multi-region cluster",
    },
}

clusterProperties2, err := client2.CreateCluster(context.Background(), input2)

if err != nil {
    return fmt.Errorf("failed to create second cluster: %v", err)
}

```

```

}

// link initial cluster to second cluster
cluster1Arns := []string{*clusterProperties2.Arn}

// Now that we know the second cluster arn we can set it as a peer of the first
cluster
input3 := dsql.UpdateClusterInput{
    Identifier: clusterProperties.Identifier,
    MultiRegionProperties: &dtypes.MultiRegionProperties{
        WitnessRegion: aws.String("us-west-2"),
        Clusters:      cluster1Arns,
    }}

_, err = client.UpdateCluster(context.Background(), &input3)

if err != nil {
    return fmt.Errorf("failed to update cluster to associate with first cluster. %v",
err)
}

// Create the waiter with our custom options for first cluster
waiter := dsql.NewClusterActiveWaiter(client, func(o
*dsql.ClusterActiveWaiterOptions) {
    o.MaxDelay = 30 * time.Second // Creating a multi-region cluster can take a few
minutes
    o.MinDelay = 10 * time.Second
    o.LogWaitAttempts = true
})

// Now that multiRegionProperties is fully defined for both clusters
// they'll begin the transition to ACTIVE

// Create the input for the clusterProperties to monitor for first cluster
getInput := &dsql.GetClusterInput{
    Identifier: clusterProperties.Identifier,
}

// Wait for the first cluster to become active
fmt.Printf("Waiting for first cluster %s to become active...\n",
*clusterProperties.Identifier)
err = waiter.Wait(ctx, getInput, 5*time.Minute)
if err != nil {
    return fmt.Errorf("error waiting for first cluster to become active: %w", err)
}

```



```

}

// Create the waiter with our custom options
waiter2 := dsql.NewClusterActiveWaiter(client2, func(o
*dsql.ClusterActiveWaiterOptions) {
    o.MaxDelay = 30 * time.Second // Creating a multi-region cluster can take a few
minutes
    o.MinDelay = 10 * time.Second
    o.LogWaitAttempts = true
})

// Create the input for the clusterProperties to monitor for second
getInput2 := &dsql.GetClusterInput{
    Identifier: clusterProperties2.Identifier,
}

// Wait for the second cluster to become active
fmt.Printf("Waiting for second cluster %s to become active...\n",
*clusterProperties2.Identifier)
err = waiter2.Wait(ctx, getInput2, 5*time.Minute)
if err != nil {
    return fmt.Errorf("error waiting for second cluster to become active: %w", err)
}

fmt.Printf("Cluster %s is now active\n", *clusterProperties.Identifier)
fmt.Printf("Cluster %s is now active\n", *clusterProperties2.Identifier)
return nil
}

// Example usage in main function
func main() {
    // Set up context with timeout
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Minute)
    defer cancel()

    err := CreateMultiRegionClusters(ctx, "us-west-2", "us-east-1", "us-east-2")
    if err != nil {
        fmt.Printf("failed to create multi-region clusters: %v", err)
        panic(err)
    }
}

```

.NET

To create a multi-Region cluster, use the following example. Creating a multi-Region cluster might take some time.

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Amazon;
using Amazon.DSQL;
using Amazon.DSQL.Model;
using Amazon.Runtime.Credentials;
using Amazon.Runtime.Endpoints;

namespace DSQLExamples.examples
{
    public class CreateMultiRegionClusters
    {
        /// <summary>
        /// Create a client. We will use this later for performing operations on the
        cluster.
        /// </summary>
        private static async Task<AmazonDSQIClient> CreateDSQIClient(RegionEndpoint
region)
        {
            var awsCredentials = await
DefaultAWSCredentialsIdentityResolver.GetCredentialsAsync();
            var clientConfig = new AmazonDSQLConfig
            {
                RegionEndpoint = region,
            };
            return new AmazonDSQIClient(awsCredentials, clientConfig);
        }

        /// <summary>
        /// Create multi-region clusters with a witness region.
        /// </summary>
        public static async Task<(CreateClusterResponse, CreateClusterResponse)>
Create(
            RegionEndpoint region1,
            RegionEndpoint region2,
            RegionEndpoint witnessRegion)
        {
            using (var client1 = await CreateDSQIClient(region1))
```

```

using (var client2 = await CreateDSQLClient(region2))
{
    var tags = new Dictionary<string, string>
    {
        { "Name", "csharp multi region cluster" }
    };

    // We can only set the witness region for the first cluster
    var createClusterRequest1 = new CreateClusterRequest
    {
        DeletionProtectionEnabled = true,
        Tags = tags,
        MultiRegionProperties = new MultiRegionProperties
        {
            WitnessRegion = witnessRegion.SystemName
        }
    };

    var cluster1 = await
client1.CreateClusterAsync(createClusterRequest1);
    var cluster1Arn = cluster1.Arn;
    Console.WriteLine($"Initiated creation of {cluster1Arn}");

    // For the second cluster we can set witness region and designate
cluster1 as a peer
    var createClusterRequest2 = new CreateClusterRequest
    {
        DeletionProtectionEnabled = true,
        Tags = tags,
        MultiRegionProperties = new MultiRegionProperties
        {
            WitnessRegion = witnessRegion.SystemName,
            Clusters = new List<string> { cluster1.Arn }
        }
    };

    var cluster2 = await
client2.CreateClusterAsync(createClusterRequest2);
    var cluster2Arn = cluster2.Arn;
    Console.WriteLine($"Initiated creation of {cluster2Arn}");

    // Now that we know the cluster2 arn we can set it as a peer of
cluster1
    var updateClusterRequest = new UpdateClusterRequest

```

```

        {
            Identifier = cluster1.Identifier,
            MultiRegionProperties = new MultiRegionProperties
            {
                WitnessRegion = witnessRegion.SystemName,
                Clusters = new List<string> { cluster2.Arn }
            }
        };

        await client1.UpdateClusterAsync(updateClusterRequest);
        Console.WriteLine($"Added {cluster2Arn} as a peer of
{cluster1Arn}");

        return (cluster1, cluster2);
    }
}

private static async Task Main()
{
    var region1 = RegionEndpoint.USEast1;
    var region2 = RegionEndpoint.USEast2;
    var witnessRegion = RegionEndpoint.USWest2;

    var (cluster1, cluster2) = await Create(region1, region2,
witnessRegion);

    Console.WriteLine("Created multi region clusters:");
    Console.WriteLine($"Cluster 1: {cluster1.Arn}");
    Console.WriteLine($"Cluster 2: {cluster2.Arn}");
}
}
}

```

Get cluster

The following examples show how to get information about a multi-Region cluster using different programming languages.

Python

To get information about a multi-Region cluster, use the following example.

```

import boto3
from datetime import datetime
import json

def get_cluster(region, identifier):
    try:
        client = boto3.client("dsql", region_name=region)
        return client.get_cluster(identifier=identifier)
    except:
        print(f"Unable to get cluster {identifier} in region {region}")
        raise

def main():
    region = "us-east-1"
    cluster_id = "<your cluster id>"
    response = get_cluster(region, cluster_id)

    print(json.dumps(response, indent=2, default=lambda obj: obj.isoformat() if
    isinstance(obj, datetime) else None))

if __name__ == "__main__":
    main()

```

C++

Use the following example to get information about a multi-Region cluster.

```

#include <aws/core/Aws.h>
#include <aws/core/utils/Outcome.h>
#include <aws/dsql/DSQLClient.h>
#include <aws/dsql/model/GetClusterRequest.h>
#include <iostream>

using namespace Aws;
using namespace Aws::DSQL;
using namespace Aws::DSQL::Model;

/**
 * Retrieves information about a cluster in Amazon Aurora DSQL

```

```

*/
GetClusterResult GetCluster(const Aws::String& region, const Aws::String&
    identifier) {
    // Create client for the specified region
    DSQL::DSQLClientConfiguration clientConfig;
    clientConfig.region = region;
    DSQL::DSQLClient client(clientConfig);

    // Get the cluster
    GetClusterRequest getClusterRequest;
    getClusterRequest.SetIdentifier(identifier);

    auto getOutcome = client.GetCluster(getClusterRequest);
    if (!getOutcome.IsSuccess()) {
        std::cerr << "Failed to retrieve cluster " << identifier << " in " << region
        << ": "
            << getOutcome.GetError().GetMessage() << std::endl;
        throw std::runtime_error("Unable to retrieve cluster " + identifier + " in
    region " + region);
    }

    return getOutcome.GetResult();
}

int main() {
    Aws::SDKOptions options;
    Aws::InitAPI(options);
    {
        try {
            // Define region and cluster ID
            Aws::String region = "us-east-1";
            Aws::String clusterId = "<your cluster id>";

            auto cluster = GetCluster(region, clusterId);

            // Print cluster details
            std::cout << "Cluster Details:" << std::endl;
            std::cout << "ARN: " << cluster.GetArn() << std::endl;
            std::cout << "Status: " <<
ClusterStatusMapper::GetNameForClusterStatus(cluster.GetStatus()) << std::endl;
        }
        catch (const std::exception& e) {
            std::cerr << "Error: " << e.what() << std::endl;
        }
    }
}

```

```

    }
    Aws::ShutdownAPI(options);
    return 0;
}

```

JavaScript

To get information about a multi-Region cluster, use the following example.

```

import { DSQLClient, GetClusterCommand } from "@aws-sdk/client-dsql";

async function getCluster(region, clusterId) {

    const client = new DSQLClient({ region });

    const getClusterCommand = new GetClusterCommand({
        identifier: clusterId,
    });

    try {
        return await client.send(getClusterCommand);
    } catch (error) {
        if (error.name === "ResourceNotFoundException") {
            console.log("Cluster ID not found or deleted");
        }
        throw error;
    }
}

async function main() {
    const region = "us-east-1";
    const clusterId = "<CLUSTER_ID>";

    const response = await getCluster(region, clusterId);
    console.log("Cluster: ", response);
}

main();

```

Java

The following example lets you get information about a multi-Region cluster.

```

package org.example;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dsql.DsqlClient;
import software.amazon.awssdk.services.dsql.model.GetClusterResponse;
import software.amazon.awssdk.services.dsql.model.ResourceNotFoundException;

public class GetCluster {

    public static void main(String[] args) {
        Region region = Region.US_EAST_1;
        String clusterId = "<your cluster id>";

        try (
            DsqlClient client = DsqlClient.builder()
                .region(region)
                .credentialsProvider(DefaultCredentialsProvider.create())
                .build()
        ) {
            GetClusterResponse cluster = client.getCluster(r ->
r.identifier(clusterId));
            System.out.println(cluster);
        } catch (ResourceNotFoundException e) {
            System.out.printf("Cluster %s not found in %s%n", clusterId, region);
        }
    }
}

```

Rust

The following example lets you get information about a multi-Region cluster.

```

use aws_config::load_defaults;
use aws_sdk_dsql::operation::get_cluster::GetClusterOutput;
use aws_sdk_dsql::{
    Client, Config,
    config::{BehaviorVersion, Region},
};

/// Create a client. We will use this later for performing operations on the
cluster.

```



```

async fn dsql_client(region: &'static str) -> Client {
    // Load default SDK configuration
    let sdk_defaults = load_defaults(BehaviorVersion::latest()).await;

    // You can set your own credentials by following this guide
    // https://docs.aws.amazon.com/sdk-for-rust/latest/dg/credproviders.html
    let credentials = sdk_defaults.credentials_provider().unwrap();

    let config = Config::builder()
        .behavior_version(BehaviorVersion::latest())
        .credentials_provider(credentials)
        .region(Region::new(region))
        .build();

    Client::from_conf(config)
}

/// Get a ClusterResource from DSQL cluster identifier
pub async fn get_cluster(region: &'static str, identifier: &'static str) ->
    GetClusterOutput {
    let client = dsql_client(region).await;
    client
        .get_cluster()
        .identifier(identifier)
        .send()
        .await
        .unwrap()
}

#[tokio::main(flavor = "current_thread")]
pub async fn main() -> anyhow::Result<()> {
    let region = "us-east-1";

    let cluster = get_cluster(region, "<your cluster id>").await;
    println!("{:#?}", cluster);

    Ok(())
}

```

Ruby

The following example lets you get information about a multi-Region cluster.

```
require "aws-sdk-dsql"
require "pp"

def get_cluster(region, identifier)
  client = Aws::DSQL::Client.new(region: region)
  client.get_cluster(identifier: identifier)
rescue Aws::Errors::ServiceError => e
  abort "Unable to retrieve cluster #{identifier} in region #{region}: #{e.message}"
end

def main
  region = "us-east-1"
  cluster_id = "<your cluster id>"
  cluster = get_cluster(region, cluster_id)
  pp cluster
end

main if $PROGRAM_NAME == __FILE__
```

.NET

The following example lets you get information about a multi-Region cluster.

```
using System;
using System.Threading.Tasks;
using Amazon;
using Amazon.DSQL;
using Amazon.DSQL.Model;
using Amazon.Runtime.Credentials;

namespace DSQLExamples.examples
{
    public class GetCluster
    {
        /// <summary>
        /// Create a client. We will use this later for performing operations on the
        cluster.
        /// </summary>
        private static async Task<AmazonDSQLClient> CreateDSQLClient(RegionEndpoint
region)
        {
```

```

        var awsCredentials = await
DefaultAWSCredentialsIdentityResolver.GetCredentialsAsync();
        var clientConfig = new AmazonDSQLConfig
        {
            RegionEndpoint = region
        };
        return new AmazonDSQLClient(awsCredentials, clientConfig);
    }

    /// <summary>
    /// Get information about a DSQL cluster.
    /// </summary>
    public static async Task<GetClusterResponse> Get(RegionEndpoint region,
string identifier)
    {
        using (var client = await CreateDSQLClient(region))
        {
            var getClusterRequest = new GetClusterRequest
            {
                Identifier = identifier
            };

            return await client.GetClusterAsync(getClusterRequest);
        }
    }

    private static async Task Main()
    {
        var region = RegionEndpoint.USEast1;
        var clusterId = "<your cluster id>";

        var response = await Get(region, clusterId);
        Console.WriteLine($"Cluster ARN: {response.Arn}");
    }
}

```

Golang

The following example lets you get information about a multi-Region cluster.

```
package main
```

```

import (
    "context"
    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/config"
    "log"
    "time"

    "github.com/aws/aws-sdk-go-v2/service/dsql"
)

func GetCluster(ctx context.Context, region, identifier string) (clusterStatus
    *dsql.GetClusterOutput, err error) {

    cfg, err := config.LoadDefaultConfig(ctx, config.WithRegion(region))
    if err != nil {
        log.Fatalf("Failed to load AWS configuration: %v", err)
    }

    // Initialize the DSQL client
    client := dsql.NewFromConfig(cfg)

    input := &dsql.GetClusterInput{
        Identifier: aws.String(identifier),
    }
    clusterStatus, err = client.GetCluster(context.Background(), input)

    if err != nil {
        log.Fatalf("Failed to get cluster: %v", err)
    }

    log.Printf("Cluster ARN: %s", *clusterStatus.Arn)

    return clusterStatus, nil
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 6*time.Minute)
    defer cancel()

    // Example cluster identifier
    identifier := "<CLUSTER_ID>"
    region := "us-east-1"

    _, err := GetCluster(ctx, region, identifier)

```

```
if err != nil {  
    log.Fatalf("Failed to get cluster: %v", err)  
}  
}
```

Update cluster

The following examples show how to update a multi-Region cluster using different programming languages.

Python

To update a multi-Region cluster, use the following example.

```
import boto3  
  
def update_cluster(region, cluster_id, deletion_protection_enabled):  
    try:  
        client = boto3.client("dsq1", region_name=region)  
        return client.update_cluster(identifier=cluster_id,  
deletionProtectionEnabled=deletion_protection_enabled)  
    except:  
        print("Unable to update cluster")  
        raise  
  
def main():  
    region = "us-east-1"  
    cluster_id = "<your cluster id>"  
    deletion_protection_enabled = False  
    response = update_cluster(region, cluster_id, deletion_protection_enabled)  
    print(f"Updated {response["arn"]} with deletion_protection_enabled:  
{deletion_protection_enabled}")  
  
if __name__ == "__main__":  
    main()
```

C++

Use the following example to update a multi-Region cluster.

```
#include <aws/core/Aws.h>
#include <aws/core/utils/Outcome.h>
#include <aws/dsql/DSQLClient.h>
#include <aws/dsql/model/UpdateClusterRequest.h>
#include <iostream>

using namespace Aws;
using namespace Aws::DSQL;
using namespace Aws::DSQL::Model;

/**
 * Updates a cluster in Amazon Aurora DSQL
 */
UpdateClusterResult UpdateCluster(const Aws::String& region, const
    Aws::Map<Aws::String, Aws::String>& updateParams) {
    // Create client for the specified region
    DSQL::DSQLClientConfiguration clientConfig;
    clientConfig.region = region;
    DSQL::DSQLClient client(clientConfig);

    // Create update request
    UpdateClusterRequest updateRequest;
    updateRequest.SetClientToken(Aws::Utils::UUID::RandomUUID());

    // Set identifier (required)
    if (updateParams.find("identifier") != updateParams.end()) {
        updateRequest.SetIdentifier(updateParams.at("identifier"));
    } else {
        throw std::runtime_error("Cluster identifier is required for update
operation");
    }

    // Set deletion protection if specified
    if (updateParams.find("deletion_protection_enabled") != updateParams.end()) {
        bool deletionProtection = (updateParams.at("deletion_protection_enabled") ==
"true");
        updateRequest.SetDeletionProtectionEnabled(deletionProtection);
    }

    // Execute the update
```

```

    auto updateOutcome = client.UpdateCluster(updateRequest);
    if (!updateOutcome.IsSuccess()) {
        std::cerr << "Failed to update cluster: " <<
updateOutcome.GetError().GetMessage() << std::endl;
        throw std::runtime_error("Unable to update cluster");
    }

    return updateOutcome.GetResult();
}

int main() {
    Aws::SDKOptions options;
    Aws::InitAPI(options);
    {
        try {
            // Define region and update parameters
            Aws::String region = "us-east-1";
            Aws::String clusterId = "<your cluster id>";

            // Create parameter map
            Aws::Map<Aws::String, Aws::String> updateParams;
            updateParams["identifier"] = clusterId;
            updateParams["deletion_protection_enabled"] = "false";

            auto updatedCluster = UpdateCluster(region, updateParams);

            std::cout << "Updated " << updatedCluster.GetArn() << std::endl;
        }
        catch (const std::exception& e) {
            std::cerr << "Error: " << e.what() << std::endl;
        }
    }
    Aws::ShutdownAPI(options);
    return 0;
}

```

JavaScript

To update a multi-Region cluster, use the following example.

```

import { DSQLClient, UpdateClusterCommand } from "@aws-sdk/client-dsql";

export async function updateCluster(region, clusterId, deletionProtectionEnabled) {

```

```

const client = new DSQLClient({ region });

const updateClusterCommand = new UpdateClusterCommand({
  identifier: clusterId,
  deletionProtectionEnabled: deletionProtectionEnabled
});

try {
  return await client.send(updateClusterCommand);
} catch (error) {
  console.error("Unable to update cluster", error.message);
  throw error;
}
}

async function main() {
  const region = "us-east-1";
  const clusterId = "<CLUSTER_ID>";
  const deletionProtectionEnabled = false;

  const response = await updateCluster(region, clusterId,
  deletionProtectionEnabled);
  console.log(`Updated ${response.arn}`);
}

main();

```

Java

Use the following example to update a multi-Region cluster.

```

package org.example;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dsdl.DsdlClient;
import software.amazon.awssdk.services.dsdl.model.UpdateClusterRequest;
import software.amazon.awssdk.services.dsdl.model.UpdateClusterResponse;

public class UpdateCluster {

    public static void main(String[] args) {
        Region region = Region.US_EAST_1;

```



```

String clusterId = "<your cluster id>";

try (
    DsqlClient client = DsqlClient.builder()
        .region(region)
        .credentialsProvider(DefaultCredentialsProvider.create())
        .build()
    ) {
    UpdateClusterRequest request = UpdateClusterRequest.builder()
        .identifier(clusterId)
        .deletionProtectionEnabled(false)
        .build();
    UpdateClusterResponse cluster = client.updateCluster(request);
    System.out.println("Updated " + cluster.arn());
}
}

```

Rust

Use the following example to update a multi-Region cluster.

```

use aws_config::load_defaults;
use aws_sdk_dsql::operation::update_cluster::UpdateClusterOutput;
use aws_sdk_dsql::{
    Client, Config,
    config::{BehaviorVersion, Region},
};

/// Create a client. We will use this later for performing operations on the
/// cluster.
async fn dsqldb_client(region: &'static str) -> Client {
    // Load default SDK configuration
    let sdk_defaults = load_defaults(BehaviorVersion::latest()).await;

    // You can set your own credentials by following this guide
    // https://docs.aws.amazon.com/sdk-for-rust/latest/dg/credproviders.html
    let credentials = sdk_defaults.credentials_provider().unwrap();

    let config = Config::builder()
        .behavior_version(BehaviorVersion::latest())
        .credentials_provider(credentials)
        .region(Region::new(region))
        .build();
}

```

```

    Client::from_conf(config)
}

/// Update a DSQL cluster and set delete protection to false. Also add new tags.
pub async fn update_cluster(region: &'static str, identifier: &'static str) ->
UpdateClusterOutput {
    let client = dsql_client(region).await;
    // Update delete protection
    let update_response = client
        .update_cluster()
        .identifier(identifier)
        .deletion_protection_enabled(false)
        .send()
        .await
        .unwrap();

    update_response
}

#[tokio::main(flavor = "current_thread")]
pub async fn main() -> anyhow::Result<()> {
    let region = "us-east-1";

    let cluster = update_cluster(region, "<your cluster id>").await;
    println!("{:?}", cluster);

    Ok(())
}

```

Ruby

Use the following example to update a multi-Region cluster.

```

require "aws-sdk-dsql"

def update_cluster(region, update_params)
  client = Aws::DSQL::Client.new(region: region)
  client.update_cluster(update_params)
rescue Aws::Errors::ServiceError => e
  abort "Unable to update cluster: #{e.message}"
end

def main

```

```

region = "us-east-1"
cluster_id = "<your cluster id>"
updated_cluster = update_cluster(region, {
  identifier: cluster_id,
  deletion_protection_enabled: false
})
puts "Updated #{updated_cluster.arn}"
end

main if $PROGRAM_NAME == __FILE__

```

.NET

Use the following example to update a multi-Region cluster.

```

using System;
using System.Threading.Tasks;
using Amazon;
using Amazon.DSQL;
using Amazon.DSQL.Model;
using Amazon.Runtime.Credentials;

namespace DSQLExamples.examples
{
    public class UpdateCluster
    {
        /// <summary>
        /// Create a client. We will use this later for performing operations on the
        cluster.
        /// </summary>
        private static async Task<AmazonDSQLClient> CreateDSQLClient(RegionEndpoint
region)
        {
            var awsCredentials = await
DefaultAWSCredentialsIdentityResolver.GetCredentialsAsync();
            var clientConfig = new AmazonDSQLConfig
            {
                RegionEndpoint = region
            };
            return new AmazonDSQLClient(awsCredentials, clientConfig);
        }

        /// <summary>

```

```

    /// Update a DSQL cluster and set delete protection to false.
    /// </summary>
    public static async Task<UpdateClusterResponse> Update(RegionEndpoint
region, string identifier)
    {
        using (var client = await CreateDSQLClient(region))
        {
            var updateClusterRequest = new UpdateClusterRequest
            {
                Identifier = identifier,
                DeletionProtectionEnabled = false
            };

            UpdateClusterResponse response = await
client.UpdateClusterAsync(updateClusterRequest);
            Console.WriteLine($"Updated {response.Arn}");

            return response;
        }
    }

    private static async Task Main()
    {
        var region = RegionEndpoint.USEast1;
        var clusterId = "<your cluster id>";

        await Update(region, clusterId);
    }
}

```

Golang

Use the following example to update a multi-Region cluster.

```

package main

import (
    "context"
    "github.com/aws/aws-sdk-go-v2/config"
    "log"
    "time"

    "github.com/aws/aws-sdk-go-v2/service/dsql"

```

```

)

func UpdateCluster(ctx context.Context, region, id string, deleteProtection bool)
(clusterStatus *dsql.UpdateClusterOutput, err error) {

    cfg, err := config.LoadDefaultConfig(ctx, config.WithRegion(region))
    if err != nil {
        log.Fatalf("Failed to load AWS configuration: %v", err)
    }

    // Initialize the DSQL client
    client := dsql.NewFromConfig(cfg)

    input := dsql.UpdateClusterInput{
        Identifier:           &id,
        DeletionProtectionEnabled: &deleteProtection,
    }

    clusterStatus, err = client.UpdateCluster(context.Background(), &input)

    if err != nil {
        log.Fatalf("Failed to update cluster: %v", err)
    }

    log.Printf("Cluster updated successfully: %v", clusterStatus.Status)
    return clusterStatus, nil
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 6*time.Minute)
    defer cancel()

    // Example cluster identifier
    identifier := "<CLUSTER_ID>"
    region := "us-east-1"
    deleteProtection := false

    _, err := UpdateCluster(ctx, region, identifier, deleteProtection)
    if err != nil {
        log.Fatalf("Failed to update cluster: %v", err)
    }
}

```

Delete cluster

The following examples show how to delete a multi-Region cluster using different programming languages.

Python

To delete a multi-Region cluster, use the following example. Deleting a multi-Region cluster might take some time.

```
import boto3

def delete_multi_region_clusters(region_1, cluster_id_1, region_2, cluster_id_2):
    try:

        client_1 = boto3.client("dsql", region_name=region_1)
        client_2 = boto3.client("dsql", region_name=region_2)

        client_1.delete_cluster(identifier=cluster_id_1)
        print(f"Deleting cluster {cluster_id_1} in {region_1}")

        # cluster_1 will stay in PENDING_DELETE state until cluster_2 is deleted

        client_2.delete_cluster(identifier=cluster_id_2)
        print(f"Deleting cluster {cluster_id_2} in {region_2}")

        # Now that both clusters have been marked for deletion they will transition
        # to DELETING state and finalize deletion
        print(f"Waiting for {cluster_id_1} to finish deletion")
        client_1.get_waiter("cluster_not_exists").wait(
            identifier=cluster_id_1,
            WaiterConfig={
                'Delay': 10,
                'MaxAttempts': 30
            }
        )

        print(f"Waiting for {cluster_id_2} to finish deletion")
        client_2.get_waiter("cluster_not_exists").wait(
            identifier=cluster_id_2,
            WaiterConfig={
                'Delay': 10,
```

```

        'MaxAttempts': 30
    }
)

except:
    print("Unable to delete cluster")
    raise

def main():
    region_1 = "us-east-1"
    cluster_id_1 = "<cluster 1 id>"
    region_2 = "us-east-2"
    cluster_id_2 = "<cluster 2 id>"

    delete_multi_region_clusters(region_1, cluster_id_1, region_2, cluster_id_2)
    print(f"Deleted {cluster_id_1} in {region_1} and {cluster_id_2} in {region_2}")

if __name__ == "__main__":
    main()

```

C++

To delete a multi-Region cluster, use the following example. Deleting a multi-Region cluster might take some time.

```

#include <aws/core/Aws.h>
#include <aws/core/Utils/Outcome.h>
#include <aws/dsql/DSQLClient.h>
#include <aws/dsql/model/DeleteClusterRequest.h>
#include <aws/dsql/model/GetClusterRequest.h>
#include <iostream>
#include <thread>
#include <chrono>

using namespace Aws;
using namespace Aws::DSQL;
using namespace Aws::DSQL::Model;

/**
 * Deletes multi-region clusters in Amazon Aurora DSQL
 */

```

```

void DeleteMultiRegionClusters(
    const Aws::String& region1,
    const Aws::String& clusterId1,
    const Aws::String& region2,
    const Aws::String& clusterId2) {

    // Create clients for each region
    DSQL::DSQLClientConfiguration clientConfig1;
    clientConfig1.region = region1;
    DSQL::DSQLClient client1(clientConfig1);

    DSQL::DSQLClientConfiguration clientConfig2;
    clientConfig2.region = region2;
    DSQL::DSQLClient client2(clientConfig2);

    // Delete the first cluster
    std::cout << "Deleting cluster " << clusterId1 << " in " << region1 <<
std::endl;

    DeleteClusterRequest deleteRequest1;
    deleteRequest1.SetIdentifier(clusterId1);
    deleteRequest1.SetClientToken(Aws::Utils::UUID::RandomUUID());

    auto deleteOutcome1 = client1.DeleteCluster(deleteRequest1);
    if (!deleteOutcome1.IsSuccess()) {
        std::cerr << "Failed to delete cluster " << clusterId1 << " in " << region1
<< ": "
                << deleteOutcome1.GetError().GetMessage() << std::endl;
        throw std::runtime_error("Failed to delete multi-region clusters");
    }

    // cluster1 will stay in PENDING_DELETE state until cluster2 is deleted
    std::cout << "Deleting cluster " << clusterId2 << " in " << region2 <<
std::endl;

    DeleteClusterRequest deleteRequest2;
    deleteRequest2.SetIdentifier(clusterId2);
    deleteRequest2.SetClientToken(Aws::Utils::UUID::RandomUUID());

    auto deleteOutcome2 = client2.DeleteCluster(deleteRequest2);
    if (!deleteOutcome2.IsSuccess()) {
        std::cerr << "Failed to delete cluster " << clusterId2 << " in " << region2
<< ": "
                << deleteOutcome2.GetError().GetMessage() << std::endl;
    }
}

```



```

        throw std::runtime_error("Failed to delete multi-region clusters");
    }
}

int main() {
    Aws::SDKOptions options;
    Aws::InitAPI(options);
    {
        try {
            Aws::String region1 = "us-east-1";
            Aws::String clusterId1 = "<your cluster id 1>";
            Aws::String region2 = "us-east-2";
            Aws::String clusterId2 = "<your cluster id 2>";

            DeleteMultiRegionClusters(region1, clusterId1, region2, clusterId2);

            std::cout << "Deleted " << clusterId1 << " in " << region1
                      << " and " << clusterId2 << " in " << region2 << std::endl;
        }
        catch (const std::exception& e) {
            std::cerr << "Error: " << e.what() << std::endl;
        }
    }
    Aws::ShutdownAPI(options);
    return 0;
}

```

JavaScript

To delete a multi-Region cluster, use the following example. Deleting a multi-Region cluster might take some time.

```

import { DSQLClient, DeleteClusterCommand, waitUntilClusterNotExists } from "@aws-sdk/client-dsql";

async function deleteMultiRegionClusters(region1, cluster1_id, region2, cluster2_id)
{
    const client1 = new DSQLClient({ region: region1 });
    const client2 = new DSQLClient({ region: region2 });

    try {
        const deleteClusterCommand1 = new DeleteClusterCommand({

```

```

        identifier: cluster1_id,
    });
    const response1 = await client1.send(deleteClusterCommand1);

    const deleteClusterCommand2 = new DeleteClusterCommand({
        identifier: cluster2_id,
    });
    const response2 = await client2.send(deleteClusterCommand2);

    console.log(`Waiting for cluster1 ${response1.identifier} to finish
deletion`);
    await waitUntilClusterNotExists(
        {
            client: client1,
            maxWaitTime: 300 // Wait for 5 minutes
        },
        {
            identifier: response1.identifier
        }
    );
    console.log(`Cluster1 Id ${response1.identifier} is now deleted`);

    console.log(`Waiting for cluster2 ${response2.identifier} to finish
deletion`);
    await waitUntilClusterNotExists(
        {
            client: client2,
            maxWaitTime: 300 // Wait for 5 minutes
        },
        {
            identifier: response2.identifier
        }
    );
    console.log(`Cluster2 Id ${response2.identifier} is now deleted`);
    return;
} catch (error) {
    if (error.name === "ResourceNotFoundException") {
        console.log("Some or all Cluster ARNs not found or already deleted");
    } else {
        console.error("Unable to delete multi-region clusters: ",
error.message);
    }
    throw error;
}

```

```

}

async function main() {
  const region1 = "us-east-1";
  const cluster1_id = "<CLUSTER_ID_1>";
  const region2 = "us-east-2";
  const cluster2_id = "<CLUSTER_ID_2>";

  const response = await deleteMultiRegionClusters(region1, cluster1_id, region2,
cluster2_id);
  console.log(`Deleted ${cluster1_id} in ${region1} and ${cluster2_id} in
${region2}`);
}

main();

```

Java

To delete a multi-Region cluster, use the following example. Deleting a multi-Region cluster might take some time.

```

package org.example;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.retries.api.BackoffStrategy;
import software.amazon.awssdk.services.dsdl.DsdlClient;
import software.amazon.awssdk.services.dsdl.DsdlClientBuilder;
import software.amazon.awssdk.services.dsdl.model.DeleteClusterRequest;

import java.time.Duration;

public class DeleteMultiRegionClusters {

  public static void main(String[] args) {
    Region region1 = Region.US_EAST_1;
    String clusterId1 = "<your cluster id 1>";
    Region region2 = Region.US_EAST_2;
    String clusterId2 = "<your cluster id 2>";

    DsdlClientBuilder clientBuilder = DsdlClient.builder()
      .credentialsProvider(DefaultCredentialsProvider.create());

```

```

    try (
        DsqlClient client1 = clientBuilder.region(region1).build();
        DsqlClient client2 = clientBuilder.region(region2).build()
    ) {
        System.out.printf("Deleting cluster %s in %s\n", clusterId1, region1);
        DeleteClusterRequest request1 = DeleteClusterRequest.builder()
            .identifier(clusterId1)
            .build();
        client1.deleteCluster(request1);

        // cluster1 will stay in PENDING_DELETE until cluster2 is deleted
        System.out.printf("Deleting cluster %s in %s\n", clusterId2, region2);
        DeleteClusterRequest request2 = DeleteClusterRequest.builder()
            .identifier(clusterId2)
            .build();
        client2.deleteCluster(request2);

        // Now that both clusters have been marked for deletion they will
transition
        // to DELETING state and finalize deletion.
        System.out.printf("Waiting for cluster %s to finish deletion\n",
clusterId1);
        client1.waiter().waitUntilClusterNotExists(
            getCluster -> getCluster.identifier(clusterId1),
            config -> config.backoffStrategyV2(
BackoffStrategy.fixedDelayWithoutJitter(Duration.ofSeconds(10))
                ).waitTimeout(Duration.ofMinutes(5))
            );

        System.out.printf("Waiting for cluster %s to finish deletion\n",
clusterId2);
        client2.waiter().waitUntilClusterNotExists(
            getCluster -> getCluster.identifier(clusterId2),
            config -> config.backoffStrategyV2(
BackoffStrategy.fixedDelayWithoutJitter(Duration.ofSeconds(10))
                ).waitTimeout(Duration.ofMinutes(5))
            );

        System.out.printf("Deleted %s in %s and %s in %s\n", clusterId1,
region1, clusterId2, region2);
    }
}

```

```
}
```

Rust

To delete a multi-Region cluster, use the following example. Deleting a multi-Region cluster might take some time.

```
use aws_config::{BehaviorVersion, Region, load_defaults};
use aws_sdk_dsql::client::Waiters;
use aws_sdk_dsql::{Client, Config};

/// Create a client. We will use this later for performing operations on the
/// cluster.
async fn dsql_client(region: &'static str) -> Client {
    // Load default SDK configuration
    let sdk_defaults = load_defaults(BehaviorVersion::latest()).await;

    // You can set your own credentials by following this guide
    // https://docs.aws.amazon.com/sdk-for-rust/latest/dg/credproviders.html
    let credentials = sdk_defaults.credentials_provider().unwrap();

    let config = Config::builder()
        .behavior_version(BehaviorVersion::latest())
        .credentials_provider(credentials)
        .region(Region::new(region))
        .build();

    Client::from_conf(config)
}

/// Create a cluster without delete protection and a name
pub async fn delete_multi_region_clusters(
    region_1: &'static str,
    cluster_id_1: &'static str,
    region_2: &'static str,
    cluster_id_2: &'static str,
) {
    let client_1 = dsql_client(region_1).await;
    let client_2 = dsql_client(region_2).await;

    println!("Deleting cluster {cluster_id_1} in {region_1}");
    client_1
        .delete_cluster()
```

```

        .identifier(cluster_id_1)
        .send()
        .await
        .unwrap();

// cluster_1 will stay in PENDING_DELETE state until cluster_2 is deleted
println!("Deleting cluster {cluster_id_2} in {region_2}");
client_2
    .delete_cluster()
    .identifier(cluster_id_2)
    .send()
    .await
    .unwrap();

// Now that both clusters have been marked for deletion they will transition
// to DELETING state and finalize deletion
println!("Waiting for {cluster_id_1} to finish deletion");
client_1
    .wait_until_cluster_not_exists()
    .identifier(cluster_id_1)
    .wait(std::time::Duration::from_secs(300)) // Wait up to 5 minutes
    .await
    .unwrap();

println!("Waiting for {cluster_id_2} to finish deletion");
client_2
    .wait_until_cluster_not_exists()
    .identifier(cluster_id_2)
    .wait(std::time::Duration::from_secs(300)) // Wait up to 5 minutes
    .await
    .unwrap();
}

#[tokio::main(flavor = "current_thread")]
pub async fn main() -> anyhow::Result<()> {
    let region_1 = "us-east-1";
    let cluster_id_1 = "<cluster 1 to be deleted>";
    let region_2 = "us-east-2";
    let cluster_id_2 = "<cluster 2 to be deleted>";

    delete_multi_region_clusters(region_1, cluster_id_1, region_2,
cluster_id_2).await;
    println!("Deleted {cluster_id_1} in {region_1} and {cluster_id_2} in
{region_2}");

```

```
    Ok(())  
  }  
}
```

Ruby

To delete a multi-Region cluster, use the following example. Deleting a multi-Region cluster might take some time.

```
require "aws-sdk-dsql"  
  
def delete_multi_region_clusters(region_1, cluster_id_1, region_2, cluster_id_2)  
  client_1 = Aws::DSQL::Client.new(region: region_1)  
  client_2 = Aws::DSQL::Client.new(region: region_2)  
  
  puts "Deleting cluster #{cluster_id_1} in #{region_1}"  
  client_1.delete_cluster(identifier: cluster_id_1)  
  
  # cluster_1 will stay in PENDING_DELETE state until cluster_2 is deleted  
  puts "Deleting #{cluster_id_2} in #{region_2}"  
  client_2.delete_cluster(identifier: cluster_id_2)  
  
  # Now that both clusters have been marked for deletion they will transition  
  # to DELETING state and finalize deletion  
  puts "Waiting for #{cluster_id_1} to finish deletion"  
  client_1.wait_until(:cluster_not_exists, identifier: cluster_id_1) do |w|  
    # Wait for 5 minutes  
    w.max_attempts = 30  
    w.delay = 10  
  end  
  
  puts "Waiting for #{cluster_id_2} to finish deletion"  
  client_2.wait_until(:cluster_not_exists, identifier: cluster_id_2) do |w|  
    # Wait for 5 minutes  
    w.max_attempts = 30  
    w.delay = 10  
  end  
  
  rescue Aws::Errors::ServiceError => e  
    abort "Failed to delete multi-region clusters: #{e.message}"  
  end  
  
  def main  
    region_1 = "us-east-1"
```

```

cluster_id_1 = "<your cluster id 1>"
region_2 = "us-east-2"
cluster_id_2 = "<your cluster id 2>"

delete_multi_region_clusters(region_1, cluster_id_1, region_2, cluster_id_2)
puts "Deleted #{cluster_id_1} in #{region_1} and #{cluster_id_2} in #{region_2}"
end

main if $PROGRAM_NAME == __FILE__

```

.NET

To delete a multi-Region cluster, use the following example. Deleting a multi-Region cluster might take some time.

```

using System;
using System.Threading.Tasks;
using Amazon;
using Amazon.DSQL;
using Amazon.DSQL.Model;
using Amazon.Runtime.Credentials;
using Amazon.Runtime.Endpoints;

namespace DSQLExamples.examples
{
    public class DeleteMultiRegionClusters
    {
        /// <summary>
        /// Create a client. We will use this later for performing operations on the
        cluster.
        /// </summary>
        private static async Task<AmazonDSQLClient> CreateDSQLClient(RegionEndpoint
region)
        {
            var awsCredentials = await
DefaultAWSCredentialsIdentityResolver.GetCredentialsAsync();
            var clientConfig = new AmazonDSQLConfig
            {
                RegionEndpoint = region,
            };
            return new AmazonDSQLClient(awsCredentials, clientConfig);
        }
    }
}

```



```

    /// <summary>
    /// Delete multi-region clusters.
    /// </summary>
    public static async Task Delete(
        RegionEndpoint region1,
        string clusterId1,
        RegionEndpoint region2,
        string clusterId2)
    {
        using (var client1 = await CreateDSQLClient(region1))
        using (var client2 = await CreateDSQLClient(region2))
        {
            var deleteRequest1 = new DeleteClusterRequest
            {
                Identifier = clusterId1
            };

            var deleteResponse1 = await
client1.DeleteClusterAsync(deleteRequest1);
            Console.WriteLine($"Initiated deletion of {deleteResponse1.Arn}");

            // cluster 1 will stay in PENDING_DELETE state until cluster 2 is
deleted
            var deleteRequest2 = new DeleteClusterRequest
            {
                Identifier = clusterId2
            };

            var deleteResponse2 = await
client2.DeleteClusterAsync(deleteRequest2);
            Console.WriteLine($"Initiated deletion of {deleteResponse2.Arn}");
        }
    }

    private static async Task Main()
    {
        var region1 = RegionEndpoint.USEast1;
        var cluster1 = "<cluster 1 to be deleted>";
        var region2 = RegionEndpoint.USEast2;
        var cluster2 = "<cluster 2 to be deleted>";

        await Delete(region1, cluster1, region2, cluster2);
    }

```

```
}
}
```

Golang

To delete a multi-Region cluster, use the following example. Deleting a multi-Region cluster might take some time.

```
package main

import (
    "context"
    "fmt"
    "log"
    "time"

    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/config"
    "github.com/aws/aws-sdk-go-v2/service/dsql"
)

func DeleteMultiRegionClusters(ctx context.Context, region1, clusterId1, region2,
    clusterId2 string) error {
    // Load the AWS configuration for region 1
    cfg1, err := config.LoadDefaultConfig(ctx, config.WithRegion(region1))
    if err != nil {
        return fmt.Errorf("unable to load SDK config for region %s: %w", region1, err)
    }

    // Load the AWS configuration for region 2
    cfg2, err := config.LoadDefaultConfig(ctx, config.WithRegion(region2))
    if err != nil {
        return fmt.Errorf("unable to load SDK config for region %s: %w", region2, err)
    }

    // Create DSQL clients for both regions
    client1 := dsql.NewFromConfig(cfg1)
    client2 := dsql.NewFromConfig(cfg2)

    // Delete cluster in region 1
    fmt.Printf("Deleting cluster %s in %s\n", clusterId1, region1)
    _, err = client1.DeleteCluster(ctx, &dsql.DeleteClusterInput{
        Identifier: aws.String(clusterId1),
```

```

}))
if err != nil {
    return fmt.Errorf("failed to delete cluster in region %s: %w", region1, err)
}

// Delete cluster in region 2
fmt.Printf("Deleting cluster %s in %s\n", clusterId2, region2)
_, err = client2.DeleteCluster(ctx, &dsql.DeleteClusterInput{
    Identifier: aws.String(clusterId2),
})
if err != nil {
    return fmt.Errorf("failed to delete cluster in region %s: %w", region2, err)
}

// Create waiters for both regions
waiter1 := dsql.NewClusterNotExistsWaiter(client1, func(options
*dsql.ClusterNotExistsWaiterOptions) {
    options.MinDelay = 10 * time.Second
    options.MaxDelay = 30 * time.Second
    options.LogWaitAttempts = true
})

waiter2 := dsql.NewClusterNotExistsWaiter(client2, func(options
*dsql.ClusterNotExistsWaiterOptions) {
    options.MinDelay = 10 * time.Second
    options.MaxDelay = 30 * time.Second
    options.LogWaitAttempts = true
})

// Wait for cluster in region 1 to be deleted
fmt.Printf("Waiting for cluster %s to finish deletion\n", clusterId1)
err = waiter1.Wait(ctx, &dsql.GetClusterInput{
    Identifier: aws.String(clusterId1),
}, 5*time.Minute)
if err != nil {
    return fmt.Errorf("error waiting for cluster deletion in region %s: %w", region1,
err)
}

// Wait for cluster in region 2 to be deleted
fmt.Printf("Waiting for cluster %s to finish deletion\n", clusterId2)
err = waiter2.Wait(ctx, &dsql.GetClusterInput{
    Identifier: aws.String(clusterId2),
}, 5*time.Minute)

```

```

if err != nil {
    return fmt.Errorf("error waiting for cluster deletion in region %s: %w", region2,
err)
}

fmt.Printf("Successfully deleted clusters %s in %s and %s in %s\n",
clusterId1, region1, clusterId2, region2)
return nil
}

// Example usage in main function
func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Minute)
    defer cancel()

    err := DeleteMultiRegionClusters(
        ctx,
        "us-east-1",      // region1
        "<CLUSTER_ID_1>", // clusterId1
        "us-east-2",      // region2
        "<CLUSTER_ID_2>", // clusterId2
    )
    if err != nil {
        log.Fatalf("Failed to delete multi-region clusters: %v", err)
    }
}

```

For more code samples and examples, visit the [Aurora DSQL Samples GitHub repository](#).

Using AWS CLI

The AWS CLI provides a command-line interface for managing your multi-Region Aurora DSQL clusters. The following examples demonstrate how to create, configure, and delete multi-Region clusters.

Connect to your multi-Region cluster

Multi-Region peered clusters provide two regional endpoints, one in each peered cluster AWS Region. Both endpoints present a single logical database that supports concurrent read and write operations with strong data consistency. In addition to peered clusters, a multi-Region cluster also has a witness Region that stores a limited window of encrypted transaction logs, which is used

to improve multi-Region durability and availability. Multi-Region witness Regions do not have endpoints.

Create multi-Region clusters

To create multi-Region clusters, you first create a cluster with a witness Region. Then you peer this cluster with a second cluster that shares the same witness Region as your first cluster. The following example shows how to create clusters in US East (N. Virginia) and US East (Ohio) with US West (Oregon) as the witness Region.

Step 1: Create cluster one in US East (N. Virginia)

To create a cluster in the US East (N. Virginia) AWS Region with multi-Region properties, use the command below.

```
aws dsq1 create-cluster \  
--region us-east-1 \  
--multi-region-properties '{"witnessRegion":"us-west-2"}'
```

Example Response:

```
{  
  "identifier": "abc0def1baz2quux3quuux4",  
  "arn": "arn:aws:dsq1:us-east-1:111122223333:cluster/abc0def1baz2quux3quuux4",  
  "status": "UPDATING",  
  "encryptionDetails": {  
    "encryptionType": "AWS_OWNED_KMS_KEY",  
    "encryptionStatus": "ENABLED"  
  }  
  "creationTime": "2024-05-24T09:15:32.708000-07:00"  
}
```

Note

When the API operation succeeds, the cluster enters the `PENDING_SETUP` state. Cluster creation remains in the `PENDING_SETUP` until you update the cluster with the ARN of its peer cluster.

Step 2: Create cluster two in US East (Ohio)

To create a cluster in the US East (Ohio) AWS Region with multi-Region properties, use the command below.

```
aws dsq1 create-cluster \  
--region us-east-2 \  
--multi-region-properties '{"witnessRegion":"us-west-2"}'
```

Example Response:

```
{  
  "identifier": "foo0bar1baz2quux3quuxquux5",  
  "arn": "arn:aws:dsq1:us-east-2:111122223333:cluster/foo0bar1baz2quux3quuxquux5",  
  "status": "PENDING_SETUP",  
  "creationTime": "2025-05-06T06:51:16.145000-07:00",  
  "deletionProtectionEnabled": true,  
  "multiRegionProperties": {  
    "witnessRegion": "us-west-2",  
    "clusters": [  
      "arn:aws:dsq1:us-east-2:111122223333:cluster/foo0bar1baz2quux3quuxquux5"  
    ]  
  }  
}
```

When the API operation succeeds, the cluster transitions to PENDING_SETUP state. The cluster creation remains in the PENDING_SETUP state until you update it with the ARN of another cluster for peering.

Step 3: Peer cluster in US East (N. Virginia) with US East (Ohio)

To peer your US East (N. Virginia) cluster with your US East (Ohio) cluster, use the update-cluster command. Specify your US East (N. Virginia) cluster name and a JSON string with the ARN of the US East (Ohio) cluster.

```
aws dsq1 update-cluster \  
--region us-east-1 \  
--identifier 'foo0bar1baz2quux3quuxquux4' \  
--multi-region-properties '{"witnessRegion": "us-west-2", "clusters": ["arn:aws:dsq1:us-east-2:111122223333:cluster/foo0bar1baz2quux3quuxquux5"]}'
```

Example Response

```
{
  "identifier": "foo0bar1baz2quux3quuxquux4",
  "arn": "arn:aws:dsq1:us-east-1:111122223333:cluster/foo0bar1baz2quux3quuxquux4",
  "status": "UPDATING",
  "creationTime": "2025-05-06T06:46:10.745000-07:00"
}
```

Step 4: Peer cluster in US East (Ohio) with US East (N. Virginia)

To peer your US East (Ohio) cluster with your US East (N. Virginia) cluster, use the `update-cluster` command. Specify your US East (Ohio) cluster name and a JSON string with the ARN of the US East (N. Virginia) cluster.

Example

```
aws dsq1 update-cluster \
--region us-east-2 \
--identifier 'foo0bar1baz2quux3quuxquux5' \
--multi-region-properties '{"witnessRegion": "us-west-2", "clusters":
["arn:aws:dsq1:us-east-1:111122223333:cluster/foo0bar1baz2quux3quuxquux4"]}'
```

Example Response

```
{
  "identifier": "foo0bar1baz2quux3quuxquux5",
  "arn": "arn:aws:dsq1:us-east-2:111122223333:cluster/foo0bar1baz2quux3quuxquux5",
  "status": "UPDATING",
  "creationTime": "2025-05-06T06:51:16.145000-07:00"
}
```

Note

After successful peering, both clusters transition from "PENDING_SETUP" to "CREATING" and finally to "ACTIVE" status when ready for use.

View multi-Region cluster properties

When you describe a cluster, you can view multi-Region properties for clusters in different AWS Regions.

Example

```
aws dsq1 get-cluster \  
--region us-east-1 \  
--identifier 'foo0bar1baz2quux3quuxquux4'
```

Example Response

```
{  
  "identifier": "foo0bar1baz2quux3quuxquux4",  
  "arn": "arn:aws:dsq1:us-east-1:111122223333:cluster/foo0bar1baz2quux3quuxquux4",  
  "status": "PENDING_SETUP",  
  "encryptionDetails": {  
    "encryptionType": "AWS_OWNED_KMS_KEY",  
    "encryptionStatus": "ENABLED"  
  },  
  "creationTime": "2024-11-27T00:32:14.434000-08:00",  
  "deletionProtectionEnabled": false,  
  "multiRegionProperties": {  
    "witnessRegion": "us-west-2",  
    "clusters": [  
      "arn:aws:dsq1:us-east-1:111122223333:cluster/foo0bar1baz2quux3quuxquux4",  
      "arn:aws:dsq1:us-east-2:111122223333:cluster/foo0bar1baz2quux3quuxquux5"  
    ]  
  }  
}
```

Peer clusters during creation

You can reduce the number of steps by including peering information during cluster creation. After creating your first cluster in US East (N. Virginia) (Step 1), you can create your second cluster in US East (Ohio) while simultaneously initiating the peering process by including the ARN of the first cluster.

Example

```
aws dsq1 create-cluster \  

```



```
--region us-east-2 \  
--multi-region-properties '{"witnessRegion":"us-west-2","clusters": ["arn:aws:dsql:us-east-1:111122223333:cluster/foo0bar1baz2quux3quuxquux4"]}'
```

This combines Steps 2 and 4, but you still need to complete Step 3 (updating the first cluster with the ARN of the second cluster) to establish the peering relationship. After all steps are completed, both clusters will transition through the same states as in the standard process: from **PENDING_SETUP** to **CREATING**, and finally to **ACTIVE** when ready for use.

Delete multi-Region clusters

To delete a multi-Region cluster, you need to complete two steps.

1. Turn off deletion protection for each cluster.
2. Delete each peered cluster separately in their respective AWS Region

Update and delete cluster in US East (N. Virginia)

1. Turn off deletion protection using the `update-cluster` command.

```
aws dsql update-cluster \  
  --region us-east-1 \  
  --identifier 'foo0bar1baz2quux3quuxquux4' \  
  --no-deletion-protection-enabled
```

2. Delete the cluster using the `delete-cluster` command.

```
aws dsql delete-cluster \  
  --region us-east-1 \  
  --identifier 'foo0bar1baz2quux3quuxquux4'
```

The command returns the following response.

```
{  
  "identifier": "foo0bar1baz2quux3quuxquux4",  
  "arn": "arn:aws:dsql:us-east-1:111122223333:cluster/  
foo0bar1baz2quux3quuxquux4",  
  "status": "PENDING_DELETE",  
  "creationTime": "2025-05-06T06:46:10.745000-07:00"  
}
```

Note

The cluster transitions to PENDING_DELETE status. The deletion isn't complete until you delete the peered cluster in US East (Ohio).

Update and delete cluster in US East (Ohio)

1. Turn off deletion protection using the `update-cluster` command.

```
aws dsq1 update-cluster \  
--region us-east-2 \  
--identifier 'foo0bar1baz2quux3quux4quux' \  
--no-deletion-protection-enabled
```

2. Delete the cluster using the `delete-cluster` command.

```
aws dsq1 delete-cluster \  
--region us-east-2 \  
--identifier 'foo0bar1baz2quux3quuxquux5'
```

The command returns the following response:

```
{  
  "identifier": "foo0bar1baz2quux3quuxquux5",  
  "arn": "arn:aws:dsq1:us-east-2:111122223333:cluster/  
foo0bar1baz2quux3quuxquux5",  
  "status": "PENDING_DELETE",  
  "creationTime": "2025-05-06T06:46:10.745000-07:00"  
}
```

Note

The cluster transitions to PENDING_DELETE status. After a few seconds, the system automatically transitions both peered clusters to DELETING status after validation.

Configuring Aurora DSQL clusters using AWS CloudFormation

You can use the same AWS CloudFormation resource `AWS::DSQL::Cluster` to deploy and manage single-Region and multi-Region Aurora DSQL clusters.

See the [Amazon Aurora DSQL resource type reference](#) for more on how to create, modify, and manage clusters using the `AWS::DSQL::Cluster` resource.

Creating the Initial Cluster Configuration

First, create an AWS CloudFormation template to define your multi-Region cluster:

```
---
Resources:
  MRCluster:
    Type: AWS::DSQL::Cluster
    Properties:
      DeletionProtectionEnabled: true
      MultiRegionProperties:
        WitnessRegion: us-west-2
```

Create stacks in both Regions using the following AWS CLI commands:

```
aws cloudformation create-stack --region us-east-2 \
  --stack-name MRCluster \
  --template-body file://mr-cluster.yaml
```

```
aws cloudformation create-stack --region us-east-1 \
  --stack-name MRCluster \
  --template-body file://mr-cluster.yaml
```

Finding Cluster Identifiers

Retrieve the physical resource IDs for your clusters:

```
aws cloudformation describe-stack-resources --region us-east-2 \
  --stack-name MRCluster \
  --query 'StackResources[].PhysicalResourceId'
[
  "auabudrks5jwh4mjt6o5xxhr4y"
```

```
]
```

```
aws cloudformation describe-stack-resources --region us-east-1 \  
  --stack-name MRCluster \  
  --query 'StackResources[].PhysicalResourceId'  
[  
  "imabudrfon4p2z3nv2jo4rlajm"  
]
```

Updating the Cluster Configuration

Update your AWS CloudFormation template to include both cluster ARNs:

```
---  
Resources:  
  MRCluster:  
    Type: AWS::DSQL::Cluster  
    Properties:  
      DeletionProtectionEnabled: true  
      MultiRegionProperties:  
        WitnessRegion: us-west-2  
        Clusters:  
          - arn:aws:dsql:us-east-2:123456789012:cluster/auabudrks5jwh4mjt6o5xxhr4y  
          - arn:aws:dsql:us-east-1:123456789012:cluster/imabudrfon4p2z3nv2jo4rlajm
```

Apply the updated configuration to both Regions:

```
aws cloudformation update-stack --region us-east-2 \  
  --stack-name MRCluster \  
  --template-body file://mr-cluster.yaml
```

```
aws cloudformation update-stack --region us-east-1 \  
  --stack-name MRCluster \  
  --template-body file://mr-cluster.yaml
```

Programming with Aurora DSQL

Aurora DSQL provides you with the following tools to manage your Aurora DSQL resources programmatically.

AWS Command Line Interface (AWS CLI)

You can create and manage your resources by using the AWS CLI in a command-line shell. The AWS CLI provides direct access to the APIs for AWS services, such as Aurora DSQL. For syntax and examples for the commands for Aurora DSQL, see [dsql](#) in the *AWS CLI Command Reference*.

AWS software development kits (SDKs)

AWS provides SDKs for many popular technologies and programming languages. They make it easier for you to call AWS services from within your applications in that language or technology. For more information about these SDKs, see [Tools for developing and managing applications on AWS](#).

Aurora DSQL API

This API is another programming interface for Aurora DSQL. When using this API, you must format every HTTPS request correctly and add a valid digital signature to every request. For more information, see [API reference](#).

AWS CloudFormation

The [AWS::DSQL::Cluster](#) is an AWS CloudFormation resource that enables you to create and manage Aurora DSQL clusters as part of your infrastructure as code. AWS CloudFormation helps you define your entire AWS environment in code, making it easier to provision, update, and replicate your infrastructure in a consistent and reliable way.

When you use the `AWS::DSQL::Cluster` resource in your AWS CloudFormation templates, you can declaratively provision Aurora DSQL clusters alongside your other cloud resources. This helps ensure that your data infrastructure deploys and manages alongside the rest of your application stack.

Amazon Aurora DSQL cluster connectivity tools

AWS provides various tools for connecting to and working with Aurora DSQL databases. These include database drivers, ORM libraries, and specialized adapters that make it easier for developers to build applications in their preferred programming language.

Database Drivers

The following table shows the available database drivers for connecting directly to Aurora DSQL.

Programming language	Driver	Sample repository link
C++	libpq	https://github.com/aws-samples/aurora-dsql-samples/tree/main/cpp/libpq
C# (.NET)	Npgsql	https://github.com/aws-samples/aurora-dsql-samples/tree/main/dotnet/npgsql
Go	pgx	https://github.com/aws-samples/aurora-dsql-samples/tree/main/go/pgx
Java	pgJDBC	https://github.com/aws-samples/aurora-dsql-samples/tree/main/java/pgjdbc
Java	Aurora DSQL JDBC Wrapper	https://github.com/aws-labs/aurora-dsql-jdbc-wrapper
JavaScript	node-postgres	https://github.com/aws-samples/aurora-dsql-samples/tree/main/javascript/node-postgres
JavaScript	Postgres.js	https://github.com/aws-samples/aurora-dsql-samples/

Programming language	Driver	Sample repository link
		tree/main/javascript/postgres-js
Python	Psycopg	https://github.com/aws-samples/aurora-dsql-samples/tree/main/python/psycopg
Python	Psycopg2	https://github.com/aws-samples/aurora-dsql-samples/tree/main/python/psycopg2
Ruby	pg	https://github.com/aws-samples/aurora-dsql-samples/tree/main/ruby/ruby-pg
Rust	SQLx	https://github.com/aws-samples/aurora-dsql-samples/tree/main/rust/sqlx

Object-Relational Mapping (ORM) libraries

The following table shows sample code for using standalone ORM libraries with Aurora DSQL.

Programming language	ORM Library	Sample repository link
Java	Hibernate	https://github.com/aws-labs/aurora-dsql-hibernate/tree/main/examples/pet-clinic-app
Python	SQLAlchemy	https://github.com/aws-labs/aurora-dsql-sqlalchemy/tree/main/examples/pet-clinic-app

Programming language	ORM Library	Sample repository link
TypeScript	Sequelize	https://github.com/aws-samples/aurora-dsql-samples/tree/main/typescript/sequelize
TypeScript	TypeORM	https://github.com/aws-samples/aurora-dsql-samples/tree/main/typescript/type-orm

Aurora DSQL adapters and dialects

The following table shows the available adapters and dialects specifically designed for Aurora DSQL.

Programming language	ORM/Framework	Repository link
Java	Hibernate	https://github.com/aws-labs/aurora-dsql-hibernate/
Python	Django	https://github.com/aws-labs/aurora-dsql-django/
Python	SQLAlchemy	https://github.com/aws-labs/aurora-dsql-sqlalchemy/

Connectors for Aurora DSQL

Aurora DSQL provides specialized connectors that extend existing database drivers to enable seamless IAM authentication and integration with AWS services. These connectors are designed to work with popular programming languages and frameworks while maintaining compatibility with existing PostgreSQL workflows.

Additional connectors for Python, Node.js, and other languages are planned for future releases. For the latest information on connector availability, see the [Aurora DSQL samples repository](#).

Connecting to Aurora DSQL clusters with a JDBC connector

The Aurora DSQL JDBC connector is designed as an authentication plugin that extends the functionality of the PostgreSQL JDBC driver to enable applications to authenticate with Aurora DSQL using IAM credentials. The connector does not connect directly to the database, but provides seamless IAM authentication on top of the underlying PostgreSQL JDBC driver.

The Aurora DSQL JDBC connector is built to work with the [PostgreSQL JDBC Driver](#) and provides seamless integration with Aurora DSQL's IAM authentication requirements.

In conjunction with the PostgreSQL JDBC Driver, the Aurora DSQL JDBC connector enables IAM-based authentication for Aurora DSQL. It introduces deep integration with AWS authentication services such as [AWS Identity and Access Management](#) (IAM).

About the connector

Aurora DSQL is a distributed SQL database service that provides high availability and scalability for PostgreSQL-compatible applications. Aurora DSQL requires IAM-based authentication with time-limited tokens that existing JDBC drivers do not natively support.

The main idea behind the Aurora DSQL JDBC connector is to add an authentication layer on top of the PostgreSQL JDBC driver that handles IAM token generation, allowing users to connect to Aurora DSQL without changing their existing JDBC workflows.

What is Aurora DSQL Authentication?

In Aurora DSQL, **authentication** involves:

- **IAM Authentication:** All connections use IAM-based authentication with time-limited tokens
- **Token Generation:** Authentication tokens are generated using AWS credentials and have configurable lifetimes

The Aurora DSQL JDBC connector is designed to understand these requirements and automatically generate IAM authentication tokens when establishing connections.

Benefits of the Aurora DSQL JDBC connector

Although Aurora DSQL provides a PostgreSQL-compatible interface, existing PostgreSQL drivers do not currently support Aurora DSQL's IAM authentication requirements. The Aurora DSQL JDBC

connector allows customers to continue using their existing PostgreSQL workflows while enabling IAM authentication through:

- **Automatic Token Generation:** IAM tokens are generated automatically using AWS credentials
- **Seamless Integration:** Works with existing JDBC connection patterns
- **AWS Credentials Support:** Supports various AWS credential providers (default, profile-based, etc.)

Using the Aurora DSQL JDBC connector with connection pooling

The Aurora DSQL JDBC connector works with connection pooling libraries such as HikariCP. The connector handles IAM token generation during connection establishment, allowing connection pools to operate normally.

Key features

Automatic Token Generation

IAM tokens are generated automatically using AWS credentials.

Seamless Integration

Works with existing JDBC connection patterns without requiring workflow changes.

AWS Credentials Support

Supports various AWS credential providers (default, profile-based, etc.).

Connection Pooling Compatibility

Works seamlessly with connection pooling libraries like HikariCP.

Prerequisites

Before you begin, make sure that you have completed the following prerequisites:

- [Created a cluster in Aurora DSQL.](#)
- Installed the Java Development Kit (JDK). Make sure that you have version 17 or higher.
- Set up appropriate IAM permissions to allow your application to connect to Aurora DSQL.
- AWS credentials configured (via AWS CLI, environment variables, or IAM roles).

Using the JDBC connector

To use the Aurora DSQL JDBC connector in your Java application, follow these steps:

1. Add the following dependencies to your Maven project:

```
<dependencies>
  <!-- Aurora DSQL JDBC Connector -->
  <dependency>
    <groupId>software.amazon.dsqr</groupId>
    <artifactId>aurora-dsqr-jdbc-connector</artifactId>
    <version>1.0.0</version>
  </dependency>
</dependencies>
```

For Gradle projects, add this dependency:

```
implementation("software.amazon.dsqr:aurora-dsqr-jdbc-connector:1.0.0")
```

2. Create a basic connection to your Aurora DSQL cluster using the AWS DSQL PostgreSQL connector format:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class DsqrJdbcConnectorExample {
    public static void main(String[] args) {
        // Using AWS DSQL PostgreSQL Connector prefix
        String jdbcUrl = "jdbc:aws-dsqr:postgresql://your-cluster.dsqr.us-east-1.on.aws/postgres?user=admin";

        try (Connection connection = DriverManager.getConnection(jdbcUrl)) {
            // Use the connection
            try (Statement statement = connection.createStatement()) {
                // Create a table
                statement.execute("CREATE TABLE IF NOT EXISTS test_table (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), name VARCHAR(100))");

                // Insert data
            }
        }
    }
}
```

```
        statement.execute("INSERT INTO test_table (name) VALUES ('Test
Name')");

        // Query data
        try (ResultSet resultSet = statement.executeQuery("SELECT * FROM
test_table")) {
            while (resultSet.next()) {
                System.out.println("ID: " + resultSet.getInt("id") + ",
Name: " + resultSet.getString("name"));
            }
        }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Configuration properties

The Aurora DSQL JDBC connector supports the following connection properties:

user

Determines the user for the connection and the token generation method used. Example:

admin

token-duration-secs

Duration in seconds for token validity. For more information on token limits, see [Generating an authentication token in Amazon Aurora DSQL](#).

profile

Used for instantiating a ProfileCredentialsProvider for token generation with the provided profile name.

region

AWS region for Aurora DSQL connections. It is optional. When provided, it will override the region extracted from the URL.

database

The database name to connect to. Default is postgres.

Logging

Enable logging for troubleshooting any issue you might experience while using the Aurora DSQL JDBC connector.

The connector uses the built-in logging system (java.util.logging) of Java. You can configure logging levels by creating a logging.properties file:

```
# Set root logger level to INFO for clean output
.level = INFO

# Show Aurora DSQL JDBC Connector FINE logs for detailed debugging
software.amazon.dsql.level = FINE

# Console handler configuration
handlers = java.util.logging.ConsoleHandler
java.util.logging.ConsoleHandler.level = FINE
java.util.logging.ConsoleHandler.formatter = java.util.logging.SimpleFormatter

# Detailed formatter pattern with timestamp and logger name
java.util.logging.SimpleFormatter.format = %1$tH:%1$tM:%1$tS.%1$tL [%4$s] %3$s - %5$s%n
```

Examples

For more comprehensive examples and use cases, refer to the [Aurora DSQL samples repository](#), which includes:

Description	Example
Using the Aurora DSQL JDBC connector for basic connections	Basic JDBC Connector Example
Using HikariCP with the Aurora DSQL JDBC connector for connection pooling	HikariCP with DSQL Connector
Using Spring Boot with HikariCP and the Aurora DSQL JDBC connector	Spring Boot HikariCP Example

Backup and restore for Amazon Aurora DSQL

Amazon Aurora DSQL helps you meet your regulatory compliance and business continuity requirements through integration with AWS Backup, a fully managed data protection service that makes it easy to centralize and automate backups across AWS services, in the cloud, and on premises. The service streamlines backup creation, management, and restoration for both single-Region and multi-Region Aurora DSQL clusters.

Key features include the following:

- Centralized backup management through the AWS Management Console, SDK, or AWS CLI
- Full cluster backups
- Automated backup schedules and retention policies
- Cross-Region and cross-account capabilities
- WORM (write-once, read-many) configuration for all the backups you store

For more information on the features of AWS Backup Vault Lock and an extensive list of available AWS Backup features for Aurora DSQL, see [Vault lock benefits](#) and [AWS Backup feature availability](#) in the *AWS Backup Developer Guide*.

Getting started with AWS Backup

AWS Backup creates complete copies of your Aurora DSQL clusters. You can get started using AWS Backup for Aurora DSQL by following the steps in [Getting started with AWS Backup](#):

1. Create on-demand backups for immediate protection.
2. Establish backup plans for automated, scheduled backups.
3. Configure retention periods and cross-Region copying.
4. Set up monitoring and notifications for backup activities.

Restoring your backups

When you restore Aurora DSQL clusters, AWS Backup always creates new clusters to preserve your source data.

Restoring single-Region clusters

To restore an Aurora DSQL single-Region cluster, use the console: <https://console.aws.amazon.com/backup> or CLI to select the recovery point (backup) you wish to restore. Configure the settings for the new cluster that will be created from your backup. For detailed instructions, see [Restore a single-Region Aurora DSQL cluster](#).

Restoring multi-Region clusters

Restoring an Aurora DSQL multi-Region cluster is supported through both the console: <https://console.aws.amazon.com/backup> and the AWS CLI. For detailed instructions, see [Restore a multi-Region Aurora DSQL cluster](#).

To restore to a multi-Region Aurora DSQL cluster, you can use a backup taken in a single AWS Region. However, before you initiate the restore process, you must ensure there is an identical copy of your backup in all AWS Regions for your multi-Region clusters. If you don't yet have those copies, you must first copy the backup to another AWS Region that supports multi-Region clusters.

We recommend creating backup copies in key AWS Regions to enable robust disaster recovery options and meet compliance requirements. To view available AWS Regions for Aurora DSQL, see [the section called "AWS Region availability"](#).

For detailed instructions on these steps, see [Amazon Aurora DSQL restore](#) documentation.

Monitoring and compliance

AWS Backup provides comprehensive visibility into backup and restore operations with the following resources.

- A centralized dashboard for tracking backup and restore jobs
- Integration with CloudWatch and CloudTrail.
- [AWS Backup Audit Manager](#) for compliance reporting and auditing.

See [Logging Aurora DSQL operations using AWS CloudTrail](#) to learn more about logging records of actions taken by a user, role, or an AWS service while using Aurora DSQL.

Additional resources

To learn more about AWS Backup features and using it in tandem with Aurora DSQL, see the following resources:

- [Managed policies for AWS Backup](#)
- [Amazon Aurora DSQL restore](#)
- [Supported services by AWS Region](#)
- [Encryption for backups in AWS Backup](#)

By using AWS Backup for Aurora DSQL, you implement a robust, compliant, and automated backup strategy that protects your critical database resources while minimizing administrative overhead. Whether you manage a single cluster or a complex multi-Region deployment, AWS Backup provides the tools you need to ensure your data remains secure and recoverable.

Monitoring and logging for Aurora DSQL

Monitoring and logging is an important part of maintaining the reliability, availability, and performance of your Amazon Aurora DSQL resources. You should monitor and collect logging data from all parts of your Aurora DSQL resources so you can easily debug a multi-point failure.

- *Amazon CloudWatch* monitors your AWS resources and the applications you run on AWS in real time. You can collect and track metrics, create customized dashboards, and set alarms that notify you or take actions when a specified metric reaches a threshold that you specify. For example, you can have CloudWatch track CPU usage or other metrics of your Amazon EC2 instances and automatically launch new instances when needed. For more information, see the [Amazon CloudWatch User Guide](#).
- *AWS CloudTrail* captures API calls and related events made by or on behalf of your AWS account and delivers the log files to an Amazon S3 bucket that you specify. You can identify which users and accounts called AWS, the source IP address from which the calls were made, and when the calls occurred. For more information, see the [AWS CloudTrail User Guide](#).

Viewing Aurora DSQL cluster status

The Aurora DSQL cluster status provides critical information about cluster health and connectivity. You can view the status of clusters and cluster instances by using the AWS Management Console, AWS CLI, or Aurora DSQL API.

Aurora DSQL cluster statuses and definitions

The following table describes each possible status for a Aurora DSQL cluster and what each status means.

Status	Description
Creating	Aurora DSQL is attempting to create or configure resources for the cluster. Any connection attempts will fail while a cluster is in this state.
Active	The cluster is operational and ready to use.

Status	Description
Idle	A cluster becomes idle when it's idle long enough for Aurora DSQL to reclaim the resources configured for it. When you connect to an idle cluster, Aurora DSQL transitions the cluster back to the Active state.
Inactive	A cluster becomes inactive when there's been no activity on the cluster for a prolonged period. When you attempt to connect to an inactive cluster, Aurora DSQL automatically transitions the cluster back to the Active state.
Updating	A cluster transitions to the Updating status when you make changes to the cluster configuration.
Deleting	A cluster transitions to the Deleting status when you submit a request to delete it.
Deleted	The cluster has been successfully deleted.
Failed	Aurora DSQL could not create the cluster because it encountered an error.
Pending Setup	For multi-Region clusters only. A multi-Region cluster enters the Pending Setup status when you create a multi-Region cluster in your first Region with a witness Region. Cluster creation pauses until you create another cluster in a secondary Region and peer the two clusters together.
Pending Delete	For multi-Region clusters only. A multi-Region cluster enters the Pending Delete status when you delete a cluster from it. The cluster moves to the Deleting state once you delete the last peer cluster.

Viewing your Aurora DSQL cluster status

To view the status of your cluster, use the AWS Management Console, AWS CLI, or Aurora DSQL API.

Console

Follow these steps to view cluster status in the AWS Management Console:

To view cluster status in the console

1. Open the Aurora DSQL console at <https://console.aws.amazon.com/dsql>.
2. Choose **Clusters** in the navigation pane.
3. View the status for each cluster in the dashboard.

AWS CLI

Use the following AWS CLI command to check the status of a single cluster.

```
aws dsq1 get-cluster --identifier cluster-id --query status --output text
```

Run the following command to list the status of all clusters.

```
for id in $(aws dsq1 list-clusters --query 'clusters[*].identifier' --output text); do
    cluster_status=$(aws dsq1 get-cluster --identifier "$id" --query 'status' --output
text)
    echo "$id    $cluster_status"
done
```

This sample output shows two active clusters and one cluster in the process of being deleted.

```
aaabbb2bkx555xa7p42qd5cdef    ACTIVE
abcde123efghi77t35abcdefgh    ACTIVE
12abc6lqasc5bbbbbbbbbbbbbb    DELETING
```

Monitoring Aurora DSQL with Amazon CloudWatch

Monitor Aurora DSQL using CloudWatch, which collects raw data and processes it into readable, near real-time metrics. CloudWatch keeps these statistics for 15 months, helping you gain better perspective on your web application or service performance. Set alarms to watch for specific thresholds and send notifications or take actions when met. Review the following Usage and Observability metrics available for Aurora DSQL.

For more information, see the [Amazon CloudWatch User Guide](#).

Observability and performance

This table outlines observability metrics for Aurora DSQL. It includes metrics for tracking read-only and total transactions to provide overall workload characterization. Actionable metrics like query timeouts and OCC conflict rate are included to help identify performance issues and concurrency conflicts. Session-related metrics, both active and total, offer insights into the current load on the system.

CloudWatch Metric Name	Metric	Unit	Description
ReadOnlyTransactions	Read-only transactions	none	The number of read-only transactions
TotalTransactions	Total transactions	none	The total number of transactions executed on the system, including read-only transactions.
QueryTimeouts	Query timeouts	none	The number of queries which have timed out due to hitting the maximum transaction time
OccConflicts	OCC conflicts	none	The number of transactions aborted due to key level OCC
CommitLatency	Commit Latency	milliseconds	Time spent by commit phase of query execution (P50)
BytesWritten	Bytes Written	bytes	Bytes written to storage

CloudWatch Metric Name	Metric	Unit	Description
BytesRead	Bytes Read	bytes	Bytes read from storage
ComputeTime	QP compute time	milliseconds	QP wall clock time
ClusterStorageSize	Cluster Storage Size	bytes	Cluster size

Usage metrics

Aurora DSQL measures all request-based activity, such as query processing, reads, and writes, using a single normalized billing unit called Distributed Processing Unit (DPU).

CloudWatch Metric Name	Metric	Dimension: ResourceId	Unit	Description
WriteDPU	Write Units	<cluster-id>	DPU	Approximates the write active-use component of your Aurora DSQL cluster DPU usage.
MultiRegionWriteDPU	Multi-Region Write Units	<cluster-id>	DPU	Applicable for Multi-Region clusters: Approximates the multi-Region write active-use component of your Aurora DSQL cluster DPU usage.

CloudWatch Metric Name	Metric	Dimension: ResourceId	Unit	Description
ReadDPU	Read Units	<cluster-id>	DPU	Approximates the read active-use component of your Aurora DSQL cluster DPU usage.
ComputeDPU	Compute Units	<cluster-id>	DPU	Approximates the compute active-use component of your Aurora DSQL cluster DPU usage.
TotalDPU	Total Units	<cluster-id>	DPU	Approximates the total active-use component of your Aurora DSQL cluster DPU usage.

Logging Aurora DSQL operations using AWS CloudTrail

Amazon Aurora DSQL is integrated with [AWS CloudTrail](#), a service that provides a record of actions taken by a user, role, or an AWS service. There are two types of events in CloudTrail: management events and data events. Management events are emitted to audit AWS resource configuration changes. Data events capture the AWS resource usage typically in the service data plane.

CloudTrail captures all API calls for Aurora DSQL as events. Aurora DSQL records console activity as management events. It also captures authenticated connection attempts to clusters as data events.

Using the information collected by CloudTrail, you can determine the request that was made to Aurora DSQL, the IP address from which the request was made, when it was made, the user identity making the request, and additional details.

CloudTrail is enabled by default in your AWS account when you create the account and you have access to the CloudTrail **Event history**. The CloudTrail **Event history** provides a viewable, searchable, downloadable, and immutable record of the past 90 days of recorded management events in an AWS Region. For more information, see [Working with CloudTrail Event history](#) in the *AWS CloudTrail User Guide*. There are no CloudTrail charges for recording the **Event history**.

To create an ongoing record of events in your AWS account, including events for Aurora DSQL, create a trail or an AWS CloudTrail Lake event data store (a centralized storage and analysis solution for AWS CloudTrail events). For more information on creating trails, see [Working with CloudTrail trails](#). To learn about setting up and managing event data stores, see [CloudTrail Lake event data stores](#).

Aurora DSQL management events in CloudTrail

CloudTrail [Management events](#) provide information about management operations that are performed on resources in your AWS account. These are also known as control plane operations. By default, CloudTrail captures management events in the **Event history**.

Amazon Aurora DSQL logs all Aurora DSQL control plane operations as management events. For a list of the Amazon Aurora DSQL control plane operations that Aurora DSQL logs to CloudTrail, see the [Aurora DSQL API reference](#).

Control plane logs

Amazon Aurora DSQL logs the following Aurora DSQL control plane operations to CloudTrail as management events.

- [CreateCluster](#)
- [DeleteCluster](#)
- [GetCluster](#)
- [GetVpcEndpointServiceName](#)
- [ListClusters](#)
- [ListTagsForResource](#)

- [TagResource](#)
- [UntagResource](#)
- [UpdateCluster](#)

Backup and restore logs

Amazon Aurora DSQL logs the following Aurora DSQL backup and restore operations to CloudTrail as management events.

- StartBackupJob
- StopBackupJob
- GetBackupJob
- StartRestoreJob
- StopRestoreJob
- GetRestoreJob

For more on protecting your Aurora DSQL clusters using AWS Backup, see [Backup and restore for Amazon Aurora DSQL](#).

AWS KMS logs

Amazon Aurora DSQL logs the following AWS KMS operations to CloudTrail as management events.

- GenerateDataKey
- Decrypt

To learn more about how CloudTrail logs track requests that Aurora DSQL sends to AWS KMS on your behalf, see [Monitoring Aurora DSQL interaction with AWS KMS](#).

Aurora DSQL data events in CloudTrail

CloudTrail [Data events](#) typically provide information about the resource operations performed on or in a resource. These are also used to capture the service's data plane operations. Data events are often high-volume activities. By default, CloudTrail doesn't log data events. The CloudTrail **Event history** doesn't record data events.

For more information about how to log data events, see [Logging data events with the AWS Management Console](#) and [Logging data events with the AWS Command Line Interface](#) in the *AWS CloudTrail User Guide*.

Additional charges apply for data events. For more information about CloudTrail pricing, see [AWS CloudTrail Pricing](#).

For Aurora DSQL, CloudTrail captures any connection attempt made to an Aurora DSQL cluster as a data event. The following table lists the Aurora DSQL resource types for which you can log data events. The **Resource type (console)** column shows the value to choose from the **Resource type** list on the CloudTrail console. The **resources.type value** column shows the `resources.type` value, which you would specify when configuring advanced event selectors using the AWS CLI or CloudTrail APIs. The **Data APIs logged to CloudTrail** column shows the API calls logged to CloudTrail for the resource type.

Resource type (console)	resources.type value	Data APIs logged to CloudTrail
Amazon Aurora DSQL	<code>AWS::DSQL::Cluster</code>	• <code>DbConnect</code> • <code>DbConnectAdmin</code>

You can configure advanced event selectors to filter on the `eventName` and `resources.ARN` fields to log only filtered events. For more information about these fields, see [AdvancedFieldSelector](#) in the *AWS CloudTrail API Reference*.

The following example shows how to use AWS CLI to configure `dsql-data-events-trail` to receive data events for Aurora DSQL.

```
aws cloudtrail put-event-selectors \
--region us-east-1 \
--trail-name dsql-data-events-trail \
--advanced-event-selectors '[{
  "Name": "Log DSQL Data Events",
  "FieldSelectors": [
    { "Field": "eventCategory", "Equals": ["Data"] },
    { "Field": "resources.type", "Equals": ["AWS::DSQL::Cluster"] } ] ]'
```

Security in Amazon Aurora DSQL

Cloud security at AWS is the highest priority. As an AWS customer, you benefit from data centers and network architectures that are built to meet the requirements of the most security-sensitive organizations.

Security is a shared responsibility between AWS and you. The [shared responsibility model](#) describes this as security *of* the cloud and security *in* the cloud:

- **Security of the cloud** – AWS is responsible for protecting the infrastructure that runs AWS services in the AWS Cloud. AWS also provides you with services that you can use securely. Third-party auditors regularly test and verify the effectiveness of our security as part of the [AWS Compliance Programs](#). To learn about the compliance programs that apply to Amazon Aurora DSQL, see [AWS Services in Scope by Compliance Program](#).
- **Security in the cloud** – Your responsibility is determined by the AWS service that you use. You are also responsible for other factors including the sensitivity of your data, your company's requirements, and applicable laws and regulations.

This documentation helps you understand how to apply the shared responsibility model when using Aurora DSQL. The following topics show you how to configure Aurora DSQL to meet your security and compliance objectives. You also learn how to use other AWS services that help you to monitor and secure your Aurora DSQL resources.

Topics

- [AWS managed policies for Amazon Aurora DSQL](#)
- [Data protection in Amazon Aurora DSQL](#)
- [Data encryption for Amazon Aurora DSQL](#)
- [Identity and access management for Aurora DSQL](#)
- [Resource-based policies for Aurora DSQL](#)
- [Using service-linked roles in Aurora DSQL](#)
- [Using IAM condition keys with Amazon Aurora DSQL](#)
- [Incident response in Amazon Aurora DSQL](#)
- [Compliance validation for Amazon Aurora DSQL](#)
- [Resilience in Amazon Aurora DSQL](#)

- [Infrastructure Security in Amazon Aurora DSQL](#)
- [Configuration and vulnerability analysis in Amazon Aurora DSQL](#)
- [Cross-service confused deputy prevention](#)
- [Security best practices for Aurora DSQL](#)

AWS managed policies for Amazon Aurora DSQL

An AWS managed policy is a standalone policy that is created and administered by AWS. AWS managed policies are designed to provide permissions for many common use cases so that you can start assigning permissions to users, groups, and roles.

Keep in mind that AWS managed policies might not grant least-privilege permissions for your specific use cases because they're available for all AWS customers to use. We recommend that you reduce permissions further by defining [customer managed policies](#) that are specific to your use cases.

You cannot change the permissions defined in AWS managed policies. If AWS updates the permissions defined in an AWS managed policy, the update affects all principal identities (users, groups, and roles) that the policy is attached to. AWS is most likely to update an AWS managed policy when a new AWS service is launched or new API operations become available for existing services.

For more information, see [AWS managed policies](#) in the *IAM User Guide*.

AWS managed policy: AmazonAuroraDSQFullAccess

You can attach AmazonAuroraDSQFullAccess to your users, groups, and roles.

This policy grants permissions that allows full administrative access to Aurora DSQL. Principals with these permissions can:

- Create, delete, and update Aurora DSQL clusters, including multi-Region clusters
- Manage cluster inline policies (create, view, update, and delete policies)
- Add and remove tags from clusters
- List clusters and view information about individual clusters

- See tags attached to Aurora DSQL clusters
- Connect to the database as any user, including admin
- Perform backup and restore operations for Aurora DSQL clusters, including starting, stopping, and monitoring backup and restore jobs
- Use customer-managed AWS KMS keys for cluster encryption
- View any metrics from CloudWatch their account account
- Use AWS Fault Injection Service (AWS FIS) to inject failures into Aurora DSQL clusters for fault tolerance testing
- Create service-linked roles for the `dsql.amazonaws.com` service, which is required for creating clusters

Permissions details

This policy includes the following permissions.

- `dsql`—grants principals full access to Aurora DSQL.
- `cloudwatch`—grants permission to publish metric data points to Amazon CloudWatch.
- `iam`—grants permission to create a service-linked role.
- `backup and restore`—grants permissions to start, stop, and monitor backup and restore jobs for Aurora DSQL clusters.
- `kms`—grants permissions required to validate access to customer-managed keys used for Aurora DSQL cluster encryption when creating, updating, or connecting to clusters.
- `fis`—grants permissions to use AWS Fault Injection Service (AWS FIS) to inject failures into Aurora DSQL clusters for fault tolerance testing.

You can find the `AmazonAuroraDSQLFullAccess` policy in the IAM console and in the [AWS Managed Policy Reference Guide](#).

AWS managed policy: AmazonAuroraDSQLReadOnlyAccess

You can attach `AmazonAuroraDSQLReadOnlyAccess` to your users, groups, and roles.

Allows read access to Aurora DSQL. Principals with these permissions can list clusters and view information about individual clusters. They can see the tags attached to Aurora DSQL clusters,

and view cluster inline policies. They can retrieve and see any metrics from CloudWatch on your account.

Permissions details

This policy includes the following permissions.

- `dsql` – grants read only permissions to all resources in Aurora DSQL.
- `cloudwatch` – grants permission to retrieve batch amounts of CloudWatch metric data and perform metric math on retrieved data

You can find the `AmazonAuroraDSQLReadOnlyAccess` policy in the IAM console and the [AWS Managed Policy Reference Guide](#).

AWS managed policy: `AmazonAuroraDSQLConsoleFullAccess`

You can attach `AmazonAuroraDSQLConsoleFullAccess` to your users, groups, and roles.

Allows full administrative access to Amazon Aurora DSQL via the AWS Management Console. Principals with these permissions can:

- Create, delete, and update Aurora DSQL clusters, including multi-Region clusters, with the console
- Manage cluster inline policies through the console (create, view, update, and delete policies)
- List clusters and view information about individual clusters
- See tags on any resource on your account
- Connect to the database as any user, including the admin
- Perform backup and restore operations for Aurora DSQL clusters, including starting, stopping, and monitoring backup and restore jobs
- Use customer-managed AWS KMS keys for cluster encryption
- Launch AWS CloudShell from the AWS Management Console
- View any metrics from CloudWatch on your account
- Use AWS Fault Injection Service (AWS FIS) to inject failures into Aurora DSQL clusters for fault tolerance testing

- Create service linked roles for the `dsql.amazonaws.com` service, which is required for creating clusters

You can find the `AmazonAuroraDSQLConsoleFullAccess` policy on the IAM console and [AmazonAuroraDSQLConsoleFullAccess](#) in the AWS Managed Policy Reference Guide.

Permissions details

This policy includes the following permissions.

- `dsql`—grants full administrative permissions to all resources in Aurora DSQL via the AWS Management Console.
- `cloudwatch`—grants permission to retrieve batch amounts of CloudWatch metric data and perform metric math on retrieved data.
- `tag`—grants permission to returns tag keys and values currently in use in the specified AWS Region for the calling account.
- `backup and restore`—grants permissions to start, stop, and monitor backup and restore jobs for Aurora DSQL clusters.
- `kms`—grants permissions required to validate access to customer-managed keys used for Aurora DSQL cluster encryption when creating, updating, or connecting to clusters.
- `cloudshell`—grants permissions to launch AWS CloudShell to interact with Aurora DSQL.
- `ec2`—grants permission to view Amazon VPC endpoint information needed for Aurora DSQL connections.
- `fis`—grants permissions to use AWS FIS to inject failures into Aurora DSQL clusters for fault tolerance testing.
- `access-analyzer:ValidatePolicy` grants permission for the linter in the policy editor, which provides real-time feedback about errors, warnings, and security issues in the current policy.
- `fis`—grants permissions to use AWS Fault Injection Service (AWS FIS) to inject failures into Aurora DSQL clusters for fault tolerance testing.

You can find the `AmazonAuroraDSQLConsoleFullAccess` policy in the IAM console and the [AWS Managed Policy Reference Guide](#).

AWS managed policy: AuroraDSQLServiceRolePolicy

You can't attach AuroraDSQLServiceRolePolicy to your IAM entities. This policy is attached to a service-linked role that allows Aurora DSQL to access account resources.

You can find the AuroraDSQLServiceRolePolicy policy on the IAM console and [AuroraDSQLServiceRolePolicy](#) in the AWS Managed Policy Reference Guide.

Aurora DSQL updates to AWS managed policies

View details about updates to AWS managed policies for Aurora DSQL since this service began tracking these changes. For automatic alerts about changes to this page, subscribe to the RSS feed on the Aurora DSQL Document history page.

Change	Description	Date
AmazonAuroraDSQLFu llAccess and AmazonAur oraDSQLConsoleFullAccess update	Added support for AWS Fault Injection Service (AWS FIS) integration with Aurora DSQL. This allows you to inject failures into single-Region and multi-Region Aurora DSQL clusters to test fault tolerance of your applications. You can create experiment templates in the AWS FIS console to define failure scenarios and target specific Aurora DSQL clusters for testing. For more on these policies, see AmazonAuroraDSQLFu llAccess and AmazonAur oraDSQLConsoleFullAccess.	August 19, 2025

Change	Description	Date
AmazonAuroraDSQLFu llAccess, AmazonAur oraDSQLReadOnlyAccess, and AmazonAuroraDSQLCo nsoleFullAccess update	Added resource-based policy (RBP) support with new permissions: PutClusterPolicy , GetClusterPolicy , and DeleteClusterPolicy . These permissions allow managing inline policies attached to Aurora DSQL clusters for fine-grained access control. For more information, see AmazonAuroraDSQLFu llAccess, AmazonAur oraDSQLReadOnlyAccess, and AmazonAuroraDSQLCo nsoleFullAccess.	October 15, 2025
AmazonAuroraDSQLFu llAccess update	Adds the capability to perform backup and restore operations for Aurora DSQL clusters, including starting, stopping, and monitoring jobs. It also adds the capability to use customer-managed KMS keys for cluster encryption. For more information, see AmazonAuroraDSQLFu llAccess and Using service-linked roles in Aurora DSQL .	May 21, 2025

Change	Description	Date
AmazonAuroraDSQLEnsoleFullAccess update	Adds the capability to perform backup and restore operations for Aurora DSQL clusters through the AWS Console Home. This includes starting, stopping, and monitoring jobs. It also supports using customer-managed KMS keys for cluster encryption and launching AWS CloudShell. For more information, see AmazonAuroraDSQLEnsoleFullAccess and Using service-linked roles in Aurora DSQL.	May 21, 2025

Change	Description	Date
AmazonAuroraDSQLFu llAccess update	The policy adds four new permissions to create and manage database clusters across multiple AWS Regions: <code>PutMultiRegionProperties</code> , <code>PutWitnessRegion</code> , <code>AddPeerCluster</code> , and <code>RemovePeerCluster</code> . These permissions include resource-level controls and condition keys so you can control which clusters users you can modify. The policy also adds the <code>GetVpcEndpointServiceName</code> permission to help you connect to your Aurora DSQL clusters through AWS PrivateLink. For more information, see AmazonAuroraDSQLFu llAccess and Using service-linked roles in Aurora DSQL .	May 13, 2025

Change	Description	Date
AmazonAuroraDSQLReadOnlyAccess update	Includes the ability to determine the correct VPC endpoint service name when connecting to your Aurora DSQL clusters through AWS PrivateLink Aurora DSQL creates unique endpoints per cell, so this API helps ensure you can identify the correct endpoint for your cluster and avoid connection errors. For more information, see AmazonAuroraDSQLReadOnlyAccess and Using service-linked roles in Aurora DSQL .	May 13, 2025

Change	Description	Date
AmazonAuroraDSQLConsoleFullAccess update	Adds new permissions to Aurora DSQL to support multi-Region cluster management and VPC endpoint connection. The new permissions include: PutMultiRegionProperties PutWitnessRegion AddPeerCluster RemovePeerCluster GetVpcEndpointServiceName For more information, see AmazonAuroraDSQLConsoleFullAccess and Using service-linked roles in Aurora DSQL.	May 13, 2025
AuroraDsqlServiceLinkedRole Policy update	Adds the ability to publish metrics to the AWS/AuroraDSQL and AWS/Usage CloudWatch namespaces to the policy. This allows the associated service or role to emit more comprehensive usage and performance data to your CloudWatch environment. For more information, see AuroraDsqlServiceLinkedRole Policy and Using service-linked roles in Aurora DSQL.	May 8, 2025

Change	Description	Date
Page created	Started tracking AWS managed policies related to Amazon Aurora DSQL	December 3, 2024

Data protection in Amazon Aurora DSQL

The [shared responsibility model](#) applies to data protection in . As described in this model, is responsible for protecting the global infrastructure that runs all of the AWS Cloud. You are responsible for maintaining control over your content that is hosted on this infrastructure. You are also responsible for the security configuration and management tasks for the that you use. For more information about data privacy, see the [Data Privacy FAQ](#). For information about data protection in Europe, see the [Shared Responsibility Model and GDPR](#) blog post on the *Security Blog*.

For data protection purposes, we recommend that you protect credentials and set up individual users with AWS IAM Identity Center or AWS Identity and Access Management. That way, each user is given only the permissions necessary to fulfill their job duties. We also recommend that you secure your data in the following ways:

- Use multi-factor authentication (MFA) with each account.
- Use SSL/TLS to communicate with resources. We require TLS 1.2 and recommend TLS 1.3.
- Set up API and user activity logging with AWS CloudTrail. For information about using trails to capture activities, see [Working with trails](#) in the *User Guide*.
- Use encryption solutions, along with all default security controls within AWS services.
- Use advanced managed security services such as Amazon Macie, which assists in discovering and securing sensitive data that is stored in Amazon S3.

We strongly recommend that you never put confidential or sensitive information, such as your customers email addresses, into tags or free-form text fields such as a **Name** field. This includes when you work with or other using the console, API, AWS CLI, or AWS SDKs. Any data that you enter into tags or free-form text fields used for names may be used for billing or diagnostic logs. If you provide a URL to an external server, we strongly recommend that you do not include credentials information in the URL to validate your request to that server.

Data encryption

Amazon Aurora DSQL provides a highly durable storage infrastructure designed for mission-critical and primary data storage. Data is redundantly stored on multiple devices across multiple facilities in an Aurora DSQL Region.

Encryption in transit

By default, encryption in transit is configured for you. Aurora DSQL uses TLS to encrypt all traffic between your SQL client and Aurora DSQL.

Encryption and signing of data in transit between AWS CLI, SDK, or API clients and Aurora DSQL endpoints:

- Aurora DSQL provides HTTPS endpoints for encrypting data in transit.
- To protect the integrity of API requests to Aurora DSQL, API calls must be signed by the caller. Calls are signed by an X.509 certificate or the customer's AWS secret access key according to the Signature Version 4 Signing Process (Sigv4). For more information, see [Signature Version 4 Signing Process](#) in the *AWS General Reference*.
- Use the AWS CLI or one of the AWS SDKs to make requests to AWS. These tools automatically sign the requests for you with the access key that you specify when you configure the tools.

FIPS compliance

Aurora DSQL dataplane endpoints (cluster endpoints used for database connections) use FIPS 140-2 validated cryptographic modules by default. No separate FIPS endpoints are required for cluster connections.

For control plane operations, Aurora DSQL provides dedicated FIPS endpoints in supported regions. For more information about control plane FIPS endpoints, see [Aurora DSQL endpoints and quotas](#) in the *AWS General Reference*.

For encryption at rest, see [Encryption at rest in Aurora DSQL](#).

Inter-network traffic privacy

Connections are protected both between Aurora DSQL and on-premises applications and between Aurora DSQL and other AWS resources within the same AWS Region.

You have two connectivity options between your private network and AWS:

- An AWS Site-to-Site VPN connection. For more information, see [What is AWS Site-to-Site VPN?](#)
- An AWS Direct Connect connection. For more information, see [What is AWS Direct Connect?](#)

You get access to Aurora DSQL through the network by using AWS-published API operations. Clients must support the following:

- Transport Layer Security (TLS). We require TLS 1.2 and recommend TLS 1.3.
- Cipher suites with perfect forward secrecy (PFS) such as DHE (Ephemeral Diffie-Hellman) or ECDHE (Elliptic Curve Ephemeral Diffie-Hellman). Most modern systems such as Java 7 and later support these modes.

Data Protection in witness Regions

When you create a multi-Region cluster, a witness Region helps enable automated failure recovery by participating in synchronous replication of encrypted transactions. If a peered cluster becomes unavailable, the witness Region remains available to validate and process database writes, ensuring no loss of availability.

Witness Regions protect and secure your data through these design features:

- The witness Region receives and stores only encrypted transaction logs. It never hosts, stores or transmits your encryption keys.
- The witness Region focuses solely on write transaction logging and quorum functions. It can't read your data by design.
- The witness Region operates without cluster connection endpoints or query processors. This prevents user database access.

For more information on witness Regions, see [Configuring multi-Region clusters](#).

Configuring SSL/TLS certificates for Aurora DSQL connections

Aurora DSQL requires all connections to use Transport Layer Security (TLS) encryption. To establish secure connections, your client system must trust the Amazon Root Certificate Authority (Amazon Root CA 1). This certificate is pre-installed on many operating systems. This section provides instructions for verifying the pre-installed Amazon Root CA 1 certificate on various operating systems, and guides you through the process of manually installing the certificate if it is not already present.

We recommend using PostgreSQL version 17.

Important

For production environments, we recommend using `verify-full` SSL mode to ensure the highest level of connection security. This mode verifies that the server certificate is signed by a trusted certificate authority and that the server hostname matches the certificate.

Verifying pre-installed certificates

In most operating systems, **Amazon Root CA 1** is already pre-installed. To validate this, you can follow the steps below.

Linux (RedHat/CentOS/Fedora)

Run the following command in your terminal:

```
trust list | grep "Amazon Root CA 1"
```

If the certificate is installed, you see the following output:

```
label: Amazon Root CA 1
```

macOS

1. Open Spotlight Search (**Command + Space**)
2. Search for **Keychain Access**
3. Select **System Roots** under **System Keychains**
4. Look for **Amazon Root CA 1** in the certificate list

Windows

Note

Due to a known issue with the psql Windows client, using system root certificates (`sslrootcert=system`) may return the following error: `SSL error: unregistered`

scheme. You can follow the [Connecting from Windows](#) as an alternative way to connect to your cluster using SSL.

If **Amazon Root CA 1** is not installed in your operating system, follow the steps below.

Installing certificates

If the Amazon Root CA 1 certificate is not pre-installed on your operating system, you will need to manually install it in order to establish secure connections to your Aurora DSQL cluster.

Linux certificate installation

Follow these steps to install the Amazon Root CA certificate on Linux systems.

1. Download the Root Certificate:

```
wget https://www.amazontrust.com/repository/AmazonRootCA1.pem
```

2. Copy the certificate to the trust store:

```
sudo cp ./AmazonRootCA1.pem /etc/pki/ca-trust/source/anchors/
```

3. Update the CA trust store:

```
sudo update-ca-trust
```

4. Verify the installation:

```
trust list | grep "Amazon Root CA 1"
```

macOS certificate installation

These certificate installation steps are optional. The [Linux certificate installation](#) also work for a macOS.

1. Download the Root Certificate:

```
wget https://www.amazontrust.com/repository/AmazonRootCA1.pem
```

2. Add the certificate to the System keychain:

```
sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain
AmazonRootCA1.pem
```

3. Verify the installation:

```
security find-certificate -a -c "Amazon Root CA 1" -p /Library/Keychains/
System.keychain
```

Connecting with SSL/TLS verification

Before configuring SSL/TLS certificates for secure connections to your Aurora DSQL cluster, ensure you have the following prerequisites.

- PostgreSQL version 17 installed
- AWS CLI configured with appropriate credentials
- Aurora DSQL cluster endpoint information

Connecting from Linux

1. Generate and set the authentication token:

```
export PGPASSWORD=$(aws dsq1 generate-db-connect-admin-auth-token --region=your-
cluster-region --hostname your-cluster-endpoint)
```

2. Connect using system certificates (if pre-installed):

```
PGSSLROOTCERT=system \  
PGSSLMODE=verify-full \  
psql --dbname postgres \  
--username admin \  
--host your-cluster-endpoint
```

3. Or, connect using a downloaded certificate:

```
PGSSLROOTCERT=/full/path/to/root.pem \  
PGSSLMODE=verify-full \  
psql --dbname postgres \  
--username admin \  

```

```
--host your-cluster-endpoint
```

Note

For more on PGSSLMODE settings, see [sslmode](#) in the PostgreSQL 17 [Database Connection Control Functions](#) documentation.

Connecting from macOS

1. Generate and set the authentication token:

```
export PGPASSWORD=$(aws dsq1 generate-db-connect-admin-auth-token --region=your-cluster-region --hostname your-cluster-endpoint)
```

2. Connect using system certificates (if pre-installed):

```
PGSSLROOTCERT=system \  
PGSSLMODE=verify-full \  
psql --dbname postgres \  
--username admin \  
--host your-cluster-endpoint
```

3. Or, download the root certificate and save it as `root.pem` (if certificate is not pre-installed)

```
PGSSLROOTCERT=/full/path/to/root.pem \  
PGSSLMODE=verify-full \  
psql -dbname postgres \  
--username admin \  
--host your_cluster_endpoint
```

4. Connect using `psql`:

```
PGSSLROOTCERT=/full/path/to/root.pem \  
PGSSLMODE=verify-full \  
psql -dbname postgres \  
--username admin \  
--host your_cluster_endpoint
```

Connecting from Windows

Using Command Prompt

1. Generate the authentication token:

```
aws dsq1 generate-db-connect-admin-auth-token ^  
--region=your-cluster-region ^  
--expires-in=3600 ^  
--hostname=your-cluster-endpoint
```

2. Set the password environment variable:

```
set "PGPASSWORD=token-from-above"
```

3. Set SSL configuration:

```
set PGSSLROOTCERT=C:\full\path\to\root.pem  
set PGSSLMODE=verify-full
```

4. Connect to the database:

```
"C:\Program Files\PostgreSQL\17\bin\psql.exe" --dbname postgres ^  
--username admin ^  
--host your-cluster-endpoint
```

Using PowerShell

1. Generate and set the authentication token:

```
$env:PGPASSWORD = (aws dsq1 generate-db-connect-admin-auth-token --region=your-cluster-region --expires-in=3600 --hostname=your-cluster-endpoint)
```

2. Set SSL configuration:

```
$env:PGSSLROOTCERT='C:\full\path\to\root.pem'  
$env:PGSSLMODE='verify-full'
```

3. Connect to the database:

```
"C:\Program Files\PostgreSQL\17\bin\psql.exe" --dbname postgres `
```

```
--username admin `
--host your-cluster-endpoint
```

Additional resources

- [PostgreSQL SSL documentation](#)
- [Amazon Trust Services](#)

Data encryption for Amazon Aurora DSQL

Amazon Aurora DSQL encrypts all user data at rest. For enhanced security, this encryption uses AWS Key Management Service (AWS KMS). This functionality helps reduce the operational burden and complexity involved in protecting sensitive data. Encryption at rest helps you:

- Reduce the operational burden of protecting sensitive data
- Build security-sensitive applications that meet strict encryption compliance and regulatory requirements
- Add an extra layer of data protection by always securing your data in an encrypted cluster
- Comply with organizational policies, industry or government regulations, and compliance requirements

With Aurora DSQL, you can build security-sensitive applications that meet strict encryption compliance and regulatory requirements. The following sections explain how to configure encryption for new and existing Aurora DSQL databases and manage your encryption keys.

Topics

- [KMS key types for Aurora DSQL](#)
- [Encryption at rest in Aurora DSQL](#)
- [Using AWS KMS and data keys with Aurora DSQL](#)
- [Authorizing use of your AWS KMS key for Aurora DSQL](#)
- [Aurora DSQL encryption context](#)
- [Monitoring Aurora DSQL interaction with AWS KMS](#)
- [Creating an encrypted Aurora DSQL cluster](#)
- [Removing or updating a key for your Aurora DSQL cluster](#)

- [Considerations for encryption with Aurora DSQL](#)

KMS key types for Aurora DSQL

Aurora DSQL integrates with AWS KMS to manage the encryption keys for your clusters. To learn more about key types and states, see [AWS Key Management Service concepts](#) in the *AWS Key Management Service Developer Guide*. When you create a new cluster, you can choose from the following KMS key types to encrypt your cluster:

AWS owned key

Default encryption type. Aurora DSQL owns the key at no additional charge to you. Amazon Aurora DSQL transparently decrypts cluster data when you access an encrypted cluster. You don't need to change your code or applications to use or manage encrypted clusters, and all Aurora DSQL queries work with your encrypted data.

Customer managed key

You create, own, and manage the key in your AWS account. You have full control over the KMS key. AWS KMS charges apply.

Encryption at rest using the AWS owned key is available at no additional charge. However, AWS KMS charges apply for customer managed keys. For more information, see the [AWS KMS Pricing](#) page.

You can switch between these key types at any time. For more information about key types, see [Customer managed keys](#) and [AWS owned keys](#) in the *AWS Key Management Service Developer Guide*.

Note

Aurora DSQL encryption at rest is available in all AWS Regions where Aurora DSQL is available.

Encryption at rest in Aurora DSQL

Amazon Aurora DSQL uses 256-bit Advanced Encryption Standard (AES-256) to encrypt your data at rest. This encryption helps protect your data from unauthorized access to the underlying

storage. AWS KMS manages the encryption keys for your clusters. You can use the default [AWS owned keys](#), or choose to use your own AWS KMS [Customer managed keys](#). To learn more about specifying and managing keys for your Aurora DSQL clusters, see [Creating an encrypted Aurora DSQL cluster](#) and [Removing or updating a key for your Aurora DSQL cluster](#).

Topics

- [AWS owned keys](#)
- [Customer managed keys](#)

AWS owned keys

Aurora DSQL encrypts all clusters by default with AWS owned keys. These keys are free to use and rotate annually to protect your account resources. You don't need to view, manage, use, or audit these keys, so there's no action required for data protection. For more information about AWS owned keys, see [AWS owned keys](#) in the *AWS Key Management Service Developer Guide*.

Customer managed keys

You create, own, and manage customer managed keys in your AWS account. You have full control over these KMS keys, including their policies, encryption material, tags, and aliases. For more information about managing permissions, see [Customer managed keys](#) in the *AWS Key Management Service Developer Guide*.

When you specify a customer managed key for cluster-level encryption, Aurora DSQL encrypts the cluster and all its regional data with that key. To prevent data loss and maintain cluster access, Aurora DSQL needs access to your encryption key. If you disable your customer managed key, schedule your key for deletion, or have a policy that restricts your service access, the encryption status for your cluster changes to `KMS_KEY_INACCESSIBLE`. When Aurora DSQL can't access the key, users can't connect to the cluster, the encryption status for the cluster changes to `KMS_KEY_INACCESSIBLE`, and the service loses access to the cluster data.

For multi-Region clusters, customers can configure each region's AWS KMS encryption key separately, and each regional cluster uses its own cluster-level encryption key. If Aurora DSQL can't access the encryption key for a peer in a multi-Region cluster, the status for that peer becomes `KMS_KEY_INACCESSIBLE` and it becomes unavailable for read and write operations. Other peers continue normal operations.

Note

If Aurora DSQL can't access your customer managed key, your cluster encryption status changes to `KMS_KEY_INACCESSIBLE`. After you restore key access, service will automatically detect the restoration within 15 minutes. For more information, see [Cluster idling](#).

For multi-Region clusters, if key access is lost for an extended time, the cluster restoration time depends on how much data was written while the key was inaccessible.

Using AWS KMS and data keys with Aurora DSQL

The Aurora DSQL encryption at rest feature uses an AWS KMS key and a hierarchy of data keys to protect your cluster data.

We recommend that you plan your encryption strategy before implementing your cluster in Aurora DSQL. If you store sensitive or confidential data in Aurora DSQL, consider including client-side encryption in your plan. This way you can encrypt data as close as possible to its origin, and ensure its protection throughout its lifecycle.

Topics

- [Using AWS KMS keys with Aurora DSQL](#)
- [Using cluster keys with Aurora DSQL](#)
- [Cluster key caching](#)

Using AWS KMS keys with Aurora DSQL

Encryption at rest protects your Aurora DSQL cluster under an AWS KMS key. By default, Aurora DSQL uses an AWS owned key, a multi-tenant encryption key that is created and managed in a Aurora DSQL service account. But you can encrypt your Aurora DSQL clusters under a customer managed key in your AWS account. You can select a different KMS key for each cluster, even if it participates in a multi-Region setup.

You select the KMS key for a cluster when you create or update the cluster. You can change the KMS key for a cluster at any time, either in the Aurora DSQL console or by using the `UpdateCluster` operation. The process of switching keys doesn't require downtime or degrade service.

 Important

Aurora DSQL supports only symmetric KMS keys. You can't use an asymmetric KMS key to encrypt your Aurora DSQL clusters.

A customer managed key provides the following benefits.

- You create and manage the KMS key, including setting the key policies and IAM policies to control access to the KMS key. You can enable and disable the KMS key, enable and disable automatic key rotation, and delete the KMS key when it is no longer in use.
- You can use a customer managed key with imported key material or a customer managed key in a custom key store that you own and manage.
- You can audit the encryption and decryption of your Aurora DSQL cluster by examining the Aurora DSQL API calls to AWS KMS in AWS CloudTrail logs.

However, the AWS owned key is free of charge and its use doesn't count against AWS KMS resource or request quotas. Customer managed keys incur a charge for each API call and AWS KMS quotas apply to these keys.

Using cluster keys with Aurora DSQL

Aurora DSQL uses the AWS KMS key for the cluster to generate and encrypt a unique data key for the cluster, known as the **cluster key**.

The cluster key is used as a key encryption key. Aurora DSQL uses this cluster key to protect data encryption keys that are used to encrypt the cluster data. Aurora DSQL generates a unique data encryption key for each underlying structure in a cluster, but multiple cluster items might be protected by the same data encryption key.

To decrypt the cluster key, Aurora DSQL sends a request to AWS KMS when you first access an encrypted cluster. To keep the cluster available, Aurora DSQL periodically verifies decrypt access to the KMS key, even when you're not actively accessing the cluster.

Aurora DSQL stores and uses the cluster key and data encryption keys outside of AWS KMS. It protects all keys with Advanced Encryption Standard (AES) encryption and 256-bit encryption keys. Then, it stores the encrypted keys with the encrypted data so they are available to decrypt the cluster data on demand.

If you change the KMS key for your cluster, Aurora DSQL re-encrypts the existing cluster key with the new KMS key.

Cluster key caching

To avoid calling AWS KMS for every Aurora DSQL operation, Aurora DSQL caches the plaintext cluster keys for each caller in memory. If Aurora DSQL gets a request for the cached cluster key after 15 minutes of inactivity, it sends a new request to AWS KMS to decrypt the cluster key. This call will capture any changes made to the access policies of the AWS KMS key in AWS KMS or AWS Identity and Access Management (IAM) after the last request to decrypt the cluster key.

Authorizing use of your AWS KMS key for Aurora DSQL

If you use a customer managed key in your account to protect your Aurora DSQL cluster, the policies on that key must give Aurora DSQL permission to use it on your behalf.

You have full control over the policies on a customer managed key. Aurora DSQL does not need additional authorization to use the default AWS owned key to protect the Aurora DSQL clusters in your AWS account.

Key policy for a customer managed key

When you select a customer managed key to protect a Aurora DSQL cluster, Aurora DSQL needs permission to use the AWS KMS key on behalf of the principal who makes the selection. That principal, a user or role, must have the permissions on the AWS KMS key that Aurora DSQL requires. You can provide these permissions in a key policy, or an IAM policy.

At a minimum, Aurora DSQL requires the following permissions on a customer managed key:

- `kms:Encrypt`
- `kms:Decrypt`
- `kms:ReEncrypt*` (for `kms:ReEncryptFrom` and `kms:ReEncryptTo`)
- `kms:GenerateDataKey`
- `kms:DescribeKey`

For example, the following example key policy provides only the required permissions. The policy has the following effects:

- Allows Aurora DSQL to use the AWS KMS key in cryptographic operations, but only when it is acting on behalf of principals in the account who have permission to use Aurora DSQL. If the principals specified in the policy statement don't have permission to use Aurora DSQL, the call fails, even when it comes from the Aurora DSQL service.
- The `kms:ViaService` condition key allows the permissions only when the request comes from Aurora DSQL on behalf of the principals listed in the policy statement. These principals can't call these operations directly.
- Gives the AWS KMS key administrators (users who can assume the `db-team` role) read-only access to the AWS KMS key

Before using an example key policy, replace the example principals with actual principals from your AWS account.

```
{
  "Sid": "Enable dsq1 IAM User Permissions",
  "Effect": "Allow",
  "Principal": {
    "Service": "dsq1.amazonaws.com"
  },
  "Action": [
    "kms:Decrypt",
    "kms:GenerateDataKey",
    "kms:Encrypt",
    "kms:ReEncryptFrom",
    "kms:ReEncryptTo"
  ],
  "Resource": "*",
  "Condition": {
    "StringLike": {
      "kms:EncryptionContext:aws:dsq1:ClusterId": "w4abucpbwuxx",
      "aws:SourceArn": "arn:aws:dsq1:us-east-2:111122223333:cluster/w4abucpbwuxx"
    }
  }
},
{
  "Sid": "Enable dsq1 IAM User Describe Permissions",
  "Effect": "Allow",
  "Principal": {
    "Service": "dsq1.amazonaws.com"
  },
}
```

```
"Action": "kms:DescribeKey",
"Resource": "*",
"Condition": {
  "StringLike": {
    "aws:SourceArn": "arn:aws:dsql:us-east-2:111122223333:cluster/w4abucpbwuxx"
  }
}
```

Aurora DSQL encryption context

An encryption context is a set of key–value pairs that contain arbitrary nonsecret data. When you include an encryption context in a request to encrypt data, AWS KMS cryptographically binds the encryption context to the encrypted data. To decrypt the data, you must pass in the same encryption context.

Aurora DSQL uses the same encryption context in all AWS KMS cryptographic operations. If you use a customer managed key to protect your Aurora DSQL cluster, you can use the encryption context to identify use of the AWS KMS key in audit records and logs. It also appears in plaintext in logs such as those in AWS CloudTrail.

The encryption context can also be used as a condition for authorhorization in policies.

In its requests to AWS KMS, Aurora DSQL uses an encryption context with a key-value pair:

```
"encryptionContext": {
  "aws:dsql:ClusterId": "w4abucpbwuxx"
},
```

The key–value pair identifies the cluster that Aurora DSQL is encrypting. The key is `aws:dsql:ClusterId`. The value is the identifier of the cluster.

Monitoring Aurora DSQL interaction with AWS KMS

If you use a customer managed key to protect your Aurora DSQL clusters, you can use AWS CloudTrail logs to track the requests that Aurora DSQL sends to AWS KMS on your behalf.

Expand the following sections to learn how Aurora DSQL uses the AWS KMS operations `GenerateDataKey` and `Decrypt`.

GenerateDataKey

When you enable encryption at rest on a cluster, Aurora DSQL creates a unique cluster key. It sends a `GenerateDataKey` request to AWS KMS that specifies the AWS KMS key for the cluster.

The event that records the `GenerateDataKey` operation is similar to the following example event. The user is the Aurora DSQL service account. The parameters include the Amazon Resource Name (ARN) of the AWS KMS key, a key specifier that requires a 256-bit key, and the encryption context that identifies the cluster.

```
{
  "eventVersion": "1.11",
  "userIdentity": {
    "type": "AWSService",
    "invokedBy": "dsql.amazonaws.com"
  },
  "eventTime": "2025-05-16T18:41:24Z",
  "eventSource": "kms.amazonaws.com",
  "eventName": "GenerateDataKey",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "dsql.amazonaws.com",
  "userAgent": "dsql.amazonaws.com",
  "requestParameters": {
    "encryptionContext": {
      "aws:dsql:ClusterId": "w4abucpbwuxx"
    },
    "keySpec": "AES_256",
    "keyId": "arn:aws:kms:us-east-1:982127530226:key/8b60dd9f-2ff8-4b1f-8a9c-bf570cbfdb5e"
  },
  "responseElements": null,
  "requestID": "2da2dc32-d3f4-4d6c-8a41-aff27cd9a733",
  "eventID": "426df0a6-ba56-3244-9337-438411f826f4",
  "readOnly": true,
  "resources": [
    {
      "accountId": "AWS Internal",
      "type": "AWS::KMS::Key",
      "ARN": "arn:aws:kms:us-east-1:982127530226:key/8b60dd9f-2ff8-4b1f-8a9c-bf570cbfdb5e"
    }
  ]
}
```

```

    }
  ],
  "eventType": "AwsApiCall",
  "managementEvent": true,
  "recipientAccountId": "111122223333",
  "sharedEventID": "f88e0dd8-6057-4ce0-b77d-800448426d4e",
  "vpcEndpointId": "AWS Internal",
  "vpcEndpointAccountId": "vpce-1a2b3c4d5e6f1a2b3",
  "eventCategory": "Management"
}

```

Decrypt

When you access an encrypted Aurora DSQL cluster, Aurora DSQL needs to decrypt the cluster key so that it can decrypt the keys below it in the hierarchy. It then decrypts the data in the cluster. To decrypt the cluster key, Aurora DSQL sends a Decrypt request to AWS KMS that specifies the AWS KMS key for the cluster.

The event that records the Decrypt operation is similar to the following example event. The user is the principal in your AWS account who is accessing the cluster. The parameters include the encrypted cluster key (as a ciphertext blob) and the encryption context that identifies the cluster. AWS KMS derives the ID of the AWS KMS key from the ciphertext.

```

{
  "eventVersion": "1.05",
  "userIdentity": {
    "type": "AWSService",
    "invokedBy": "dsql.amazonaws.com"
  },
  "eventTime": "2018-02-14T16:42:39Z",
  "eventSource": "kms.amazonaws.com",
  "eventName": "Decrypt",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "dsql.amazonaws.com",
  "userAgent": "dsql.amazonaws.com",
  "requestParameters": {
    "keyId": "arn:aws:kms:us-east-1:111122223333:key/1234abcd-12ab-34cd-56ef-1234567890ab",
    "encryptionContext": {
      "aws:dsql:ClusterId": "w4abucpbwuxx"
    }
  },
}

```

```

    "encryptionAlgorithm": "SYMMETRIC_DEFAULT"
  },
  "responseElements": null,
  "requestID": "11cab293-11a6-11e8-8386-13160d3e5db5",
  "eventID": "b7d16574-e887-4b5b-a064-bf92f8ec9ad3",
  "readOnly": true,
  "resources": [
    {
      "ARN": "arn:aws:kms:us-east-1:111122223333:key/1234abcd-12ab-34cd-56ef-1234567890ab",
      "accountId": "AWS Internal",
      "type": "AWS::KMS::Key"
    }
  ],
  "eventType": "AwsApiCall",
  "managementEvent": true,
  "recipientAccountId": "111122223333",
  "sharedEventID": "d99f2dc5-b576-45b6-aa1d-3a3822edbeeb",
  "vpcEndpointId": "AWS Internal",
  "vpcEndpointAccountId": "vpce-1a2b3c4d5e6f1a2b3",
  "eventCategory": "Management"
}

```

Creating an encrypted Aurora DSQL cluster

All Aurora DSQL clusters are encrypted at rest. By default, clusters use an AWS owned key at no cost, or you can specify a custom AWS KMS key. Follow these steps to create your encrypted cluster from either the AWS Management Console or the AWS CLI.

Console

To create an encrypted cluster in the AWS Management Console

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql/>.
2. In the navigation pane on the left side of the console, choose **Clusters**.
3. Choose **Create Cluster** on the top right and select **Single-Region**.
4. In the **Cluster encryption settings**, choose one of the following options.
 - Accept the default settings to encrypt with an AWS owned key at no additional cost.

- Select **Customize encryption settings (advanced)** to specify a custom KMS key. Then, search for or enter the ID or alias of your KMS key. Alternatively, choose **Create an AWS KMS key** to create a new key in the AWS KMS Console.

5. Choose **Create cluster**.

To confirm the encryption type for your cluster, navigate to the **Clusters** page and select the ID of the cluster to view the cluster details. Review the **Cluster settings** tab the **Cluster KMS key** setting shows **Aurora DSQL default key** for clusters that use AWS owned keys or the key ID for other encryption types.

 Note

If you select to own and manage your own key, make sure you set the KMS key policy appropriately. For examples and more information, see [the section called “Key policy for a customer managed key”](#).

CLI

To create a cluster that's encrypted with the default AWS owned key

- Use the following command to create an Aurora DSQL cluster.

```
aws dsq1 create-cluster
```

As shown in the following encryption details, the encryption status for the cluster is enabled by default, and the default encryption type is AWS owned key. The cluster is now encrypted with the default AWS owned key in the Aurora DSQL service account.

```
"encryptionDetails": {  
  "encryptionType" : "AWS_OWNED_KMS_KEY",  
  "encryptionStatus" : "ENABLED"  
}
```


To create a cluster that's encrypted with your customer managed key

- Use the following command to create an Aurora DSQL cluster, replacing the key ID in red text with the ID of your customer managed key.

```
aws dsq1 create-cluster \  
--kms-encryption-key d41d8cd98f00b204e9800998ecf8427e
```

As shown in the following encryption details, the encryption status for the cluster is enabled by default, and the encryption type is customer managed KMS key. The cluster is now encrypted with your key.

```
"encryptionDetails": {  
  "encryptionType" : "CUSTOMER_MANAGED_KMS_KEY",  
  "kmsKeyArn" : "arn:aws:kms:us-east-1:111122223333:key/  
d41d8cd98f00b204e9800998ecf8427e",  
  "encryptionStatus" : "ENABLED"  
}
```

Removing or updating a key for your Aurora DSQL cluster

You can use the AWS Management Console or the AWS CLI to update or remove the encryption keys on existing clusters in Amazon Aurora DSQL. If you remove a key without replacing it, Aurora DSQL uses the default AWS owned key. Follow these steps to update the encryption keys of an existing cluster from the Aurora DSQL console or the AWS CLI.

Console

To update or remove an encryption key in the AWS Management Console

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql/>.
2. In the navigation pane on the left side of the console, choose **Clusters**.
3. From the list view, find and select row of the cluster that you want to update.
4. Select the **Actions** menu and then choose **Modify**.
5. In the **Cluster encryption settings**, choose one of the following options to modify your encryption settings.

- If you want to switch from a custom key to an AWS owned key, de-select the **Customize encryption settings (advanced)** option. The default settings will apply and encrypt your cluster with an AWS owned key at no cost.
- If you want to switch from one custom KMS key to another or from an AWS owned key to a KMS key, select the **Customize encryption settings (advanced)** option if it's not already selected. Then, search for and select the ID or alias of the key you want to use. Alternatively, choose **Create an AWS KMS key** to create a new key in the AWS KMS Console.

6. Choose **Save**.

CLI

The following examples show how to use the AWS CLI to update an encrypted cluster.

To update an encrypted cluster with the default AWS owned key

```
aws dsq1 update-cluster \  
--identifier aiabtx6icfp6d53snkhseuiqq \  
--kms-encryption-key "AWS_OWNED_KMS_KEY"
```

The `EncryptionStatus` of the cluster description is set to `ENABLED` and the `EncryptionType` is `AWS_OWNED_KMS_KEY`.

```
"encryptionDetails": {  
  "encryptionType" : "AWS_OWNED_KMS_KEY",  
  "encryptionStatus" : "ENABLED"  
}
```

This cluster is now encrypted using the default AWS owned key in the Aurora DSQL service account.

To update an encrypted cluster with a customer managed key for Aurora DSQL

Update the encrypted cluster, as in the following example:

```
aws dsql update-cluster \  
--identifier aiabtx6icfp6d53snkhseuiqq \  
--kms-encryption-key arn:aws:kms:us-east-1:123456789012:key/abcd1234-abcd-1234-a123-  
ab1234a1b234
```

The `EncryptionStatus` of the cluster description transitions to `UPDATING` and the `EncryptionType` is `CUSTOMER_MANAGED_KMS_KEY`. After Aurora DSQL finishes propagating the new key through the platform, the encryption status will be transitioned to `ENABLED`

```
"encryptionDetails": {  
  "encryptionType" : "CUSTOMER_MANAGED_KMS_KEY",  
  "kmsKeyArn" : "arn:aws:us-east-1:kms:key/abcd1234-abcd-1234-a123-ab1234a1b234",  
  "encryptionStatus" : "ENABLED"  
}
```

Note

If you select to own and manage your own key, make sure you set the KMS key policy appropriately. For examples and more information, see [the section called “Key policy for a customer managed key”](#).

Considerations for encryption with Aurora DSQL

- Aurora DSQL encrypts all cluster data at rest. You can't disable this encryption or encrypt only some items in a cluster.
- AWS Backup encrypts your backups and any clusters restored from these backups. You can encrypt your backup data in AWS Backup using either the AWS owned key or a customer managed key.
- The following data protection states are enabled for Aurora DSQL:
 - **Data at rest** - Aurora DSQL encrypts all static data on persistent storage media
 - **Data in transit** - Aurora DSQL encrypts all communications using Transport Layer Security (TLS) by default

- When you transition to a different key, we recommend that you keep the original key enabled until the transition is complete. AWS needs the original key to decrypt data before it encrypts your data with the new key. The process is complete when the cluster's `encryptionStatus` is `ENABLED` and you see the `kmsKeyArn` of the new customer managed key.
- When you disable your Customer Managed Key or revoke access for Aurora DSQL to use your key, your cluster will go into `IDLE` state.
- The AWS Management Console and Amazon Aurora DSQL API use different terms for encryption types:
 - AWS Console – In the console, you'll see `KMS` when using a Customer managed key and `DEFAULT` when using an AWS owned key.
 - API – The Amazon Aurora DSQL API uses `CUSTOMER_MANAGED_KMS_KEY` for customer managed keys, and `AWS_OWNED_KMS_KEY` for AWS owned keys.
- If you don't specify an encryption key during cluster creation, Aurora DSQL automatically encrypts your data using the AWS owned key.
- You can switch between an AWS owned key and a Customer managed key at any time. Make this change using the AWS Management Console, AWS CLI, or the Amazon Aurora DSQL API.

Identity and access management for Aurora DSQL

AWS Identity and Access Management (IAM) is an AWS service that helps an administrator securely control access to AWS resources. IAM administrators control who can be *authenticated* (signed in) and *authorized* (have permissions) to use Aurora DSQL resources. IAM is an AWS service that you can use with no additional charge.

Topics

- [Audience](#)
- [Authenticating with identities](#)
- [Managing access using policies](#)
- [How Amazon Aurora DSQL works with IAM](#)
- [Identity-based policy examples for Amazon Aurora DSQL](#)
- [Troubleshooting Amazon Aurora DSQL identity and access](#)

Audience

How you use AWS Identity and Access Management (IAM) differs based on your role:

- **Service user** - request permissions from your administrator if you cannot access features (see [Troubleshooting Amazon Aurora DSQL identity and access](#))
- **Service administrator** - determine user access and submit permission requests (see [How Amazon Aurora DSQL works with IAM](#))
- **IAM administrator** - write policies to manage access (see [Identity-based policy examples for Amazon Aurora DSQL](#))

Authenticating with identities

Authentication is how you sign in to AWS using your identity credentials. You must be authenticated as the AWS account root user, an IAM user, or by assuming an IAM role.

You can sign in as a federated identity using credentials from an identity source like AWS IAM Identity Center (IAM Identity Center), single sign-on authentication, or Google/Facebook credentials. For more information about signing in, see [How to sign in to your AWS account](#) in the *AWS Sign-In User Guide*.

For programmatic access, AWS provides an SDK and CLI to cryptographically sign requests. For more information, see [AWS Signature Version 4 for API requests](#) in the *IAM User Guide*.

AWS account root user

When you create an AWS account, you begin with one sign-in identity called the AWS account *root user* that has complete access to all AWS services and resources. We strongly recommend that you don't use the root user for everyday tasks. For tasks that require root user credentials, see [Tasks that require root user credentials](#) in the *IAM User Guide*.

Federated identity

As a best practice, require human users to use federation with an identity provider to access AWS services using temporary credentials.

A *federated identity* is a user from your enterprise directory, web identity provider, or AWS Directory Service that accesses AWS services using credentials from an identity source. Federated identities assume roles that provide temporary credentials.

For centralized access management, we recommend AWS IAM Identity Center. For more information, see [What is IAM Identity Center?](#) in the *AWS IAM Identity Center User Guide*.

IAM users and groups

An [IAM user](#) is an identity with specific permissions for a single person or application. We recommend using temporary credentials instead of IAM users with long-term credentials. For more information, see [Require human users to use federation with an identity provider to access AWS using temporary credentials](#) in the *IAM User Guide*.

An [IAM group](#) specifies a collection of IAM users and makes permissions easier to manage for large sets of users. For more information, see [Use cases for IAM users](#) in the *IAM User Guide*.

IAM roles

An [IAM role](#) is an identity with specific permissions that provides temporary credentials. You can assume a role by [switching from a user to an IAM role \(console\)](#) or by calling an AWS CLI or AWS API operation. For more information, see [Methods to assume a role](#) in the *IAM User Guide*.

IAM roles are useful for federated user access, temporary IAM user permissions, cross-account access, cross-service access, and applications running on Amazon EC2. For more information, see [Cross account resource access in IAM](#) in the *IAM User Guide*.

Managing access using policies

You control access in AWS by creating policies and attaching them to AWS identities or resources. A policy defines permissions when associated with an identity or resource. AWS evaluates these policies when a principal makes a request. Most policies are stored in AWS as JSON documents. For more information about JSON policy documents, see [Overview of JSON policies](#) in the *IAM User Guide*.

Using policies, administrators specify who has access to what by defining which **principal** can perform **actions** on what **resources**, and under what **conditions**.

By default, users and roles have no permissions. An IAM administrator creates IAM policies and adds them to roles, which users can then assume. IAM policies define permissions regardless of the method used to perform the operation.

Identity-based policies

Identity-based policies are JSON permissions policy documents that you attach to an identity (user, group, or role). These policies control what actions identities can perform, on which resources, and under what conditions. To learn how to create an identity-based policy, see [Define custom IAM permissions with customer managed policies](#) in the *IAM User Guide*.

Identity-based policies can be *inline policies* (embedded directly into a single identity) or *managed policies* (standalone policies attached to multiple identities). To learn how to choose between managed and inline policies, see [Choose between managed policies and inline policies](#) in the *IAM User Guide*.

Resource-based policies

Resource-based policies are JSON policy documents that you attach to a resource. Examples include IAM *role trust policies* and Amazon S3 *bucket policies*. In services that support resource-based policies, service administrators can use them to control access to a specific resource. You must [specify a principal](#) in a resource-based policy.

Resource-based policies are inline policies that are located in that service. You can't use AWS managed policies from IAM in a resource-based policy.

Other policy types

AWS supports additional policy types that can set the maximum permissions granted by more common policy types:

- **Permissions boundaries** – Set the maximum permissions that an identity-based policy can grant to an IAM entity. For more information, see [Permissions boundaries for IAM entities](#) in the *IAM User Guide*.
- **Service control policies (SCPs)** – Specify the maximum permissions for an organization or organizational unit in AWS Organizations. For more information, see [Service control policies](#) in the *AWS Organizations User Guide*.
- **Resource control policies (RCPs)** – Set the maximum available permissions for resources in your accounts. For more information, see [Resource control policies \(RCPs\)](#) in the *AWS Organizations User Guide*.
- **Session policies** – Advanced policies passed as a parameter when creating a temporary session for a role or federated user. For more information, see [Session policies](#) in the *IAM User Guide*.

Multiple policy types

When multiple types of policies apply to a request, the resulting permissions are more complicated to understand. To learn how AWS determines whether to allow a request when multiple policy types are involved, see [Policy evaluation logic](#) in the *IAM User Guide*.

How Amazon Aurora DSQL works with IAM

Before you use IAM to manage access to Aurora DSQL, learn what IAM features are available to use with Aurora DSQL.

IAM features you can use with Amazon Aurora DSQL

IAM feature	Aurora DSQL support
Identity-based policies	Yes
Resource-based policies	Yes
Policy actions	Yes
Policy resources	Yes
Policy condition keys	Yes
ACLs	No
ABAC (tags in policies)	Yes
Temporary credentials	Yes
Principal permissions	Yes
Service roles	Yes
Service-linked roles	Yes

To get a high-level view of how Aurora DSQL and other AWS services work with most IAM features, see [AWS services that work with IAM](#) in the *IAM User Guide*.

Identity-based policies for Aurora DSQL

Supports identity-based policies: Yes

Identity-based policies are JSON permissions policy documents that you can attach to an identity, such as an IAM user, group of users, or role. These policies control what actions users and roles can perform, on which resources, and under what conditions. To learn how to create an identity-based policy, see [Define custom IAM permissions with customer managed policies](#) in the *IAM User Guide*.

With IAM identity-based policies, you can specify allowed or denied actions and resources as well as the conditions under which actions are allowed or denied. To learn about all of the elements that you can use in a JSON policy, see [IAM JSON policy elements reference](#) in the *IAM User Guide*.

Identity-based policy examples for Aurora DSQL

To view examples of Aurora DSQL identity-based policies, see [Identity-based policy examples for Amazon Aurora DSQL](#).

Resource-based policies within Aurora DSQL

Supports resource-based policies: Yes

Resource-based policies are JSON policy documents that you attach to a resource. Examples of resource-based policies are IAM role trust policies and Amazon S3 bucket policies. In services that support resource-based policies, service administrators can use them to control access to a specific resource. For the resource where the policy is attached, the policy defines what actions a specified principal can perform on that resource and under what conditions. You must [specify a principal](#) in a resource-based policy. Principals can include accounts, users, roles, federated users, or AWS services. Resource-based policies are inline policies that are located in that service. You can't use AWS managed policies from IAM in a resource-based policy.

To learn how to create and manage resource-based policies for Aurora DSQL clusters, see [Resource-based policies for Aurora DSQL](#).

Policy actions for Aurora DSQL

Supports policy actions: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The **Action** element of a JSON policy describes the actions that you can use to allow or deny access in a policy. Include actions in a policy to grant permissions to perform the associated operation.

To see a list of Aurora DSQL actions, see [Actions Defined by Amazon Aurora DSQL](#) in the *Service Authorization Reference*.

Policy actions in Aurora DSQL use the following prefix before the action:

```
dsql
```

To specify multiple actions in a single statement, separate them with commas.

```
"Action": [  
    "dsql:action1",  
    "dsql:action2"  
]
```

To view examples of Aurora DSQL identity-based policies, see [Identity-based policy examples for Amazon Aurora DSQL](#).

Policy resources for Aurora DSQL

Supports policy resources: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The **Resource** JSON policy element specifies the object or objects to which the action applies. As a best practice, specify a resource using its [Amazon Resource Name \(ARN\)](#). For actions that don't support resource-level permissions, use a wildcard (*) to indicate that the statement applies to all resources.

```
"Resource": "*"
```

To see a list of Aurora DSQL resource types and their ARNs, see [Resources Defined by Amazon Aurora DSQL](#) in the *Service Authorization Reference*. To learn with which actions you can specify the ARN of each resource, see [Actions Defined by Amazon Aurora DSQL](#).

To view examples of Aurora DSQL identity-based policies, see [Identity-based policy examples for Amazon Aurora DSQL](#).

Policy condition keys for Aurora DSQL

Supports service-specific policy condition keys: Yes

Administrators can use AWS JSON policies to specify who has access to what. That is, which **principal** can perform **actions** on what **resources**, and under what **conditions**.

The **Condition** element specifies when statements execute based on defined criteria. You can create conditional expressions that use [condition operators](#), such as equals or less than, to match the condition in the policy with values in the request. To see all AWS global condition keys, see [AWS global condition context keys](#) in the *IAM User Guide*.

To see a list of Aurora DSQL condition keys, see [Condition keys for Amazon Aurora DSQL](#) in the *Service Authorization Reference*. To learn with which actions and resources you can use a condition key, see [Actions defined by Amazon Aurora DSQL](#).

To view examples of Aurora DSQL identity-based policies, see [Identity-based policy examples for Amazon Aurora DSQL](#).

ACLs in Aurora DSQL

Supports ACLs: No

Access control lists (ACLs) control which principals (account members, users, or roles) have permissions to access a resource. ACLs are similar to resource-based policies, although they do not use the JSON policy document format.

ABAC with Aurora DSQL

Supports ABAC (tags in policies): Yes

Attribute-based access control (ABAC) is an authorization strategy that defines permissions based on attributes called tags. You can attach tags to IAM entities and AWS resources, then design ABAC policies to allow operations when the principal's tag matches the tag on the resource.

To control access based on tags, you provide tag information in the [condition element](#) of a policy using the `aws:ResourceTag/key-name`, `aws:RequestTag/key-name`, or `aws:TagKeys` condition keys.

If a service supports all three condition keys for every resource type, then the value is **Yes** for the service. If a service supports all three condition keys for only some resource types, then the value is **Partial**.

For more information about ABAC, see [Define permissions with ABAC authorization](#) in the *IAM User Guide*. To view a tutorial with steps for setting up ABAC, see [Use attribute-based access control \(ABAC\)](#) in the *IAM User Guide*.

Using temporary credentials with Aurora DSQL

Supports temporary credentials: Yes

Temporary credentials provide short-term access to AWS resources and are automatically created when you use federation or switch roles. AWS recommends that you dynamically generate temporary credentials instead of using long-term access keys. For more information, see [Temporary security credentials in IAM](#) and [AWS services that work with IAM](#) in the *IAM User Guide*.

Cross-service principal permissions for Aurora DSQL

Supports forward access sessions (FAS): Yes

Forward access sessions (FAS) use the permissions of the principal calling an AWS service, combined with the requesting AWS service to make requests to downstream services. For policy details when making FAS requests, see [Forward access sessions](#).

Service roles for Aurora DSQL

Supports service roles: Yes

A service role is an [IAM role](#) that a service assumes to perform actions on your behalf. An IAM administrator can create, modify, and delete a service role from within IAM. For more information, see [Create a role to delegate permissions to an AWS service](#) in the *IAM User Guide*.

Warning

Changing the permissions for a service role might break Aurora DSQL functionality. Edit service roles only when Aurora DSQL provides guidance to do so.

Service-linked roles for Aurora DSQL

Supports service-linked roles: Yes

A service-linked role is a type of service role that is linked to an AWS service. The service can assume the role to perform an action on your behalf. Service-linked roles appear in your AWS account and are owned by the service. An IAM administrator can view, but not edit the permissions for service-linked roles.

For details about creating or managing service-linked roles for Aurora DSQL, see [Using service-linked roles in Aurora DSQL](#).

Identity-based policy examples for Amazon Aurora DSQL

By default, users and roles don't have permission to create or modify Aurora DSQL resources. To grant users permission to perform actions on the resources that they need, an IAM administrator can create IAM policies.

To learn how to create an IAM identity-based policy by using these example JSON policy documents, see [Create IAM policies \(console\)](#) in the *IAM User Guide*.

For details about actions and resource types defined by Aurora DSQL, including the format of the ARNs for each of the resource types, see [Actions, Resources, and Condition Keys for Amazon Aurora DSQL](#) in the *Service Authorization Reference*.

Topics

- [Policy best practices](#)
- [Using the Aurora DSQL console](#)
- [Allow users to view their own permissions](#)

Policy best practices

Identity-based policies determine whether someone can create, access, or delete Aurora DSQL resources in your account. These actions can incur costs for your AWS account. When you create or edit identity-based policies, follow these guidelines and recommendations:

- **Get started with AWS managed policies and move toward least-privilege permissions** – To get started granting permissions to your users and workloads, use the *AWS managed policies* that grant permissions for many common use cases. They are available in your AWS account. We recommend that you reduce permissions further by defining AWS customer managed policies that are specific to your use cases. For more information, see [AWS managed policies](#) or [AWS managed policies for job functions](#) in the *IAM User Guide*.

- **Apply least-privilege permissions** – When you set permissions with IAM policies, grant only the permissions required to perform a task. You do this by defining the actions that can be taken on specific resources under specific conditions, also known as *least-privilege permissions*. For more information about using IAM to apply permissions, see [Policies and permissions in IAM](#) in the *IAM User Guide*.
- **Use conditions in IAM policies to further restrict access** – You can add a condition to your policies to limit access to actions and resources. For example, you can write a policy condition to specify that all requests must be sent using SSL. You can also use conditions to grant access to service actions if they are used through a specific AWS service, such as AWS CloudFormation. For more information, see [IAM JSON policy elements: Condition](#) in the *IAM User Guide*.
- **Use IAM Access Analyzer to validate your IAM policies to ensure secure and functional permissions** – IAM Access Analyzer validates new and existing policies so that the policies adhere to the IAM policy language (JSON) and IAM best practices. IAM Access Analyzer provides more than 100 policy checks and actionable recommendations to help you author secure and functional policies. For more information, see [Validate policies with IAM Access Analyzer](#) in the *IAM User Guide*.
- **Require multi-factor authentication (MFA)** – If you have a scenario that requires IAM users or a root user in your AWS account, turn on MFA for additional security. To require MFA when API operations are called, add MFA conditions to your policies. For more information, see [Secure API access with MFA](#) in the *IAM User Guide*.

For more information about best practices in IAM, see [Security best practices in IAM](#) in the *IAM User Guide*.

Using the Aurora DSQL console

To access the Amazon Aurora DSQL console, you must have a minimum set of permissions. These permissions must allow you to list and view details about the Aurora DSQL resources in your AWS account. If you create an identity-based policy that is more restrictive than the minimum required permissions, the console won't function as intended for entities (users or roles) with that policy.

You don't need to allow minimum console permissions for users that are making calls only to the AWS CLI or the AWS API. Instead, allow access to only the actions that match the API operation that they're trying to perform.

To ensure that users and roles can still use the Aurora DSQL console, also attach the Aurora DSQL `AmazonAuroraDSQLConsoleFullAccess` or `AmazonAuroraDSQLReadOnlyAccess` AWS

managed policy to the entities. For more information, see [Adding permissions to a user](#) in the *IAM User Guide*.

Allow users to view their own permissions

This example shows how you might create a policy that allows IAM users to view the inline and managed policies that are attached to their user identity. This policy includes permissions to complete this action on the console or programmatically using the AWS CLI or AWS API.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ViewOwnUserInfo",
      "Effect": "Allow",
      "Action": [
        "iam:GetUserPolicy",
        "iam:ListGroupsForUser",
        "iam:ListAttachedUserPolicies",
        "iam:ListUserPolicies",
        "iam:GetUser"
      ],
      "Resource": ["arn:aws:iam::*:user/${aws:username}"]
    },
    {
      "Sid": "NavigateInConsole",
      "Effect": "Allow",
      "Action": [
        "iam:GetGroupPolicy",
        "iam:GetPolicyVersion",
        "iam:GetPolicy",
        "iam:ListAttachedGroupPolicies",
        "iam:ListGroupPolicies",
        "iam:ListPolicyVersions",
        "iam:ListPolicies",
        "iam:ListUsers"
      ],
      "Resource": "*"
    }
  ]
}
```

Troubleshooting Amazon Aurora DSQL identity and access

Use the following information to help you diagnose and fix common issues that you might encounter when working with Aurora DSQL and IAM.

Topics

- [I am not authorized to perform an action in Aurora DSQL](#)
- [I am not authorized to perform iam:PassRole](#)
- [I want to allow people outside of my AWS account to access my Aurora DSQL resources](#)

I am not authorized to perform an action in Aurora DSQL

If you receive an error that you're not authorized to perform an action, your policies must be updated to allow you to perform the action.

The following example error occurs when the `mateojackson` tries to use the console to view details about the `my-dsql-cluster` resource but doesn't have the `GetCluster` permissions.

```
User: iam::user/mateojackson is not authorized to perform: GetCluster on resource: my-dsql-cluster
```

In this case, the policy for the `mateojackson` user must be updated to allow access to the `my-dsql-cluster` resource by using the `GetCluster` action.

If you need help, contact your administrator. Your administrator is the person who provided you with your sign-in credentials.

I am not authorized to perform iam:PassRole

If you receive an error that you're not authorized to perform the `iam:PassRole` action, your policies must be updated to allow you to pass a role to Aurora DSQL.

Some AWS services allow you to pass an existing role to that service instead of creating a new service role or service-linked role. To do this, you must have permissions to pass the role to the service.

The following example error occurs when an IAM user named `marymajor` tries to use the console to perform an action in Aurora DSQL. However, the action requires the service to have permissions that are granted by a service role. Mary does not have permissions to pass the role to the service.


```
User: arn:aws:iam::123456789012:user/marymajor is not authorized to perform:
iam:PassRole
```

In this case, Mary's policies must be updated to allow her to perform the `iam:PassRole` action.

If you need help, contact your AWS administrator. Your administrator is the person who provided you with your sign-in credentials.

I want to allow people outside of my AWS account to access my Aurora DSQL resources

You can create a role that users in other accounts or people outside of your organization can use to access your resources. You can specify who is trusted to assume the role. For services that support resource-based policies or access control lists (ACLs), you can use those policies to grant people access to your resources.

To learn more, consult the following:

- To learn whether Aurora DSQL supports these features, see [How Amazon Aurora DSQL works with IAM](#).
- To learn how to provide access to your resources across AWS accounts that you own, see [Providing access to an IAM user in another AWS account that you own](#) in the *IAM User Guide*.
- To learn how to provide access to your resources to third-party AWS accounts, see [Providing access to AWS accounts owned by third parties](#) in the *IAM User Guide*.
- To learn how to provide access through identity federation, see [Providing access to externally authenticated users \(identity federation\)](#) in the *IAM User Guide*.
- To learn the difference between using roles and resource-based policies for cross-account access, see [Cross account resource access in IAM](#) in the *IAM User Guide*.

Resource-based policies for Aurora DSQL

Use resource-based policies for Aurora DSQL to restrict or grant access to your clusters through JSON policy documents that attach directly to your cluster resources. These policies provide fine-grained control over who can access your cluster and under what conditions.

Aurora DSQL clusters are accessible from the public internet by default, with IAM authentication as the primary security control. Resource-based policies enable you to add access restrictions, particularly to block access from the public internet.

Resource-based policies work alongside IAM identity-based policies. AWS evaluates both types of policies to determine the final permissions for any access request to your cluster. By default, Aurora DSQL clusters are accessible within an account. If an IAM user or role has Aurora DSQL permissions, they can access clusters with no resource-based policy attached.

Note

Changes to resource-based policies are eventually consistent and typically take effect within one minute.

For more information about the differences between identity-based and resource-based policies, see [Identity-based policies and resource-based policies](#) in the *IAM User Guide*.

When to use resource-based policies

Resource-based policies are particularly useful in these scenarios:

- *Network-based access control* — Restrict access based on the VPC or IP address that requests originate from, or block public internet access entirely. Use condition keys like `aws:SourceVpc` and `aws:SourceIp` to control network access.
- *Multiple teams or applications* — Grant access to the same cluster for multiple teams or applications. Rather than managing individual IAM policies for each principal, you define access rules once on the cluster.
- *Complex conditional access* — Control access based on multiple factors like network attributes, request context, and user attributes. You can combine multiple conditions in a single policy.
- *Centralized security governance* — Enable cluster owners to control access using familiar AWS policy syntax that integrates with your existing security practices.

Note

Cross-account access is not yet supported for Aurora DSQL resource-based policies but will be available in future releases.

When someone tries to connect to your Aurora DSQL cluster, AWS evaluates your resource-based policy as part of the authorization context, along with any relevant IAM policies, to determine whether the request should be allowed or rejected.

Resource-based policies can grant access to principals within the same AWS account as the cluster. For multi-Region clusters, each regional cluster has its own resource-based policy, allowing for Region-specific access controls when needed.

Note

Condition context keys may vary between Regions (such as VPC IDs).

Topics

- [Creating clusters with resource-based policies](#)
- [Adding and editing resource-based policies for clusters](#)
- [Viewing resource-based policies](#)
- [Removing resource-based policies](#)
- [Common resource-based policy examples](#)
- [Blocking public access with resource-based policies in Aurora DSQL](#)
- [Aurora DSQL API Operations and Resource-Based Policies](#)

Creating clusters with resource-based policies

You can attach resource-based policies when creating a new cluster to ensure access controls are in place from the start. Each cluster can have a single inline policy attached directly to the cluster.

AWS Management Console

To add a resource-based policy during cluster creation

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql/>.
2. Choose **Create cluster**.
3. Configure your cluster name, tags, and multi-region settings as needed.
4. In the **Cluster settings** section, locate the **Resource-based policy** option.

5. Turn on **Add resource-based policy**.
6. Enter your policy document in the JSON editor. For example, to block public internet access:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Principal": {
        "AWS": "*"
      },
      "Resource": "*",
      "Action": [
        "dsql:DbConnect",
        "dsql:DbConnectAdmin"
      ],
      "Condition": {
        "Null": {
          "aws:SourceVpc": "true"
        }
      }
    }
  ]
}
```

7. You can use **Edit statement** or **Add new statement** to build your policy.
8. Complete the remaining cluster configuration and choose **Create cluster**.

AWS CLI

Use the `--policy` parameter when creating a cluster to attach an inline policy:

```
aws dsql create-cluster --policy '{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Principal": {"AWS": "*"},
    "Resource": "*",
    "Action": ["dsql:DbConnect", "dsql:DbConnectAdmin"],
    "Condition": {
      "StringNotEquals": { "aws:SourceVpc": "vpc-123456" }
    }
  }]
}
```

```
    }}
}'
```

AWS SDKs

Python

```
import boto3
import json

client = boto3.client('dsql')

policy = {
    "Version": "2012-10-17",
    "Statement": [{
        "Effect": "Deny",
        "Principal": {"AWS": "*"},
        "Resource": "*",
        "Action": ["dsql:DbConnect", "dsql:DbConnectAdmin"],
        "Condition": {
            "StringNotEquals": { "aws:SourceVpc": "vpc-123456" }
        }
    }]
}

response = client.create_cluster(
    policy=json.dumps(policy)
)

print(f"Cluster created: {response['identifier']}")
```

Java

```
import software.amazon.awssdk.services.dsql.DsqlClient;
import software.amazon.awssdk.services.dsql.model.CreateClusterRequest;
import software.amazon.awssdk.services.dsql.model.CreateClusterResponse;

DsqlClient client = DsqlClient.create();

String policy = ""
{
    "Version": "2012-10-17",
    "Statement": [{
```

```
"Effect": "Deny",
"Principal": {"AWS": "*"},
"Resource": "*",
"Action": ["dsql:DbConnect", "dsql:DbConnectAdmin"],
"Condition": {
  "StringNotEquals": { "aws:SourceVpc": "vpc-123456" }
}
}]
}
"";

CreateClusterRequest request = CreateClusterRequest.builder()
    .policy(policy)
    .build();

CreateClusterResponse response = client.createCluster(request);
System.out.println("Cluster created: " + response.identifier());
```

Adding and editing resource-based policies for clusters

AWS Management Console

To add a resource-based policy to an existing cluster

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql/>.
2. Choose your cluster from the cluster list to open the cluster details page.
3. Choose the **Permissions** tab.
4. In the **Resource-based policy** section, choose **Add policy**.
5. Enter your policy document in the JSON editor. You can use **Edit statement** or **Add new statement** to build your policy.
6. Choose **Add policy**.

To edit an existing resource-based policy

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql/>.
2. Choose your cluster from the cluster list to open the cluster details page.

3. Choose the **Permissions** tab.
4. In the **Resource-based policy** section, choose **Edit**.
5. Modify the policy document in the JSON editor. You can use **Edit statement** or **Add new statement** to update your policy.
6. Choose **Save changes**.

AWS CLI

Use the `put-cluster-policy` command to attach a new policy or update an existing policy on a cluster:

```
aws dsq1 put-cluster-policy --identifier your_cluster_id --policy '{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Principal": {"AWS": "*"},
    "Resource": "*",
    "Action": ["dsq1:DbConnect", "dsq1:DbConnectAdmin"],
    "Condition": {
      "Null": { "aws:SourceVpc": "true" }
    }
  }]
}'
```

AWS SDKs

Python

```
import boto3
import json

client = boto3.client('dsq1')

policy = {
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Principal": {"AWS": "*"},
    "Resource": "*",
```

```

        "Action": ["dsql:DbConnect", "dsql:DbConnectAdmin"],
        "Condition": {
            "Null": {"aws:SourceVpc": "true"}
        }
    ]
}

response = client.put_cluster_policy(
    identifier='your_cluster_id',
    policy=json.dumps(policy)
)

```

Java

```

import software.amazon.awssdk.services.dsql.DsqlClient;
import software.amazon.awssdk.services.dsql.model.PutClusterPolicyRequest;

DsqlClient client = DsqlClient.create();

String policy = ""
{
    "Version": "2012-10-17",
    "Statement": [{
        "Effect": "Deny",
        "Principal": {"AWS": "*"},
        "Resource": "*",
        "Action": ["dsql:DbConnect", "dsql:DbConnectAdmin"],
        "Condition": {
            "Null": {"aws:SourceVpc": "true"}
        }
    }]
}
"";

PutClusterPolicyRequest request = PutClusterPolicyRequest.builder()
    .identifier("your_cluster_id")
    .policy(policy)
    .build();

client.putClusterPolicy(request);

```


Viewing resource-based policies

You can view resource-based policies attached to your clusters to understand the current access controls in place.

AWS Management Console

To view resource-based policies

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql/>.
2. Choose your cluster from the cluster list to open the cluster details page.
3. Choose the **Permissions** tab.
4. View the attached policy in the **Resource-based policy** section.

AWS CLI

Use the `get-cluster-policy` command to view a cluster's resource-based policy:

```
aws dsql get-cluster-policy --identifier your_cluster_id
```

AWS SDKs

Python

```
import boto3
import json

client = boto3.client('dsql')

response = client.get_cluster_policy(
    identifier='your_cluster_id'
)

# Parse and pretty-print the policy
policy = json.loads(response['policy'])
print(json.dumps(policy, indent=2))
```

Java

```
import software.amazon.awssdk.services.dsql.DsqlClient;
import software.amazon.awssdk.services.dsql.model.GetClusterPolicyRequest;
import software.amazon.awssdk.services.dsql.model.GetClusterPolicyResponse;

DsqlClient client = DsqlClient.create();

GetClusterPolicyRequest request = GetClusterPolicyRequest.builder()
    .identifier("your_cluster_id")
    .build();

GetClusterPolicyResponse response = client.getClusterPolicy(request);
System.out.println("Policy: " + response.policy());
```

Removing resource-based policies

You can remove resource-based policies from clusters to change access controls.

Important

When you remove all resource-based policies from a cluster, access will be controlled entirely by IAM identity-based policies.

AWS Management Console

To remove a resource-based policy

1. Sign in to the AWS Management Console and open the Aurora DSQL console at <https://console.aws.amazon.com/dsql/>.
2. Choose your cluster from the cluster list to open the cluster details page.
3. Choose the **Permissions** tab.
4. In the **Resource-based policy** section, choose **Delete**.
5. In the confirmation dialog, type **confirm** to confirm the deletion.
6. Choose **Delete**.

AWS CLI

Use the `delete-cluster-policy` command to remove a policy from a cluster:

```
aws dsq1 delete-cluster-policy --identifier your_cluster_id
```

AWS SDKs

Python

```
import boto3

client = boto3.client('dsq1')

response = client.delete_cluster_policy(
    identifier='your_cluster_id'
)

print("Policy deleted successfully")
```

Java

```
import software.amazon.awssdk.services.dsql.DsqlClient;
import software.amazon.awssdk.services.dsql.model.DeleteClusterPolicyRequest;

DsqlClient client = DsqlClient.create();

DeleteClusterPolicyRequest request = DeleteClusterPolicyRequest.builder()
    .identifier("your_cluster_id")
    .build();

client.deleteClusterPolicy(request);
System.out.println("Policy deleted successfully");
```

Common resource-based policy examples

These examples show common patterns for controlling access to your Aurora DSQL clusters. You can combine and modify these patterns to meet your specific access requirements.

Block public internet access

This policy blocks connections to your Aurora DSQL clusters from the public internet (non-VPC). The policy doesn't specify which VPC customers can connect from—only that they must connect from a VPC. To limit access to a specific VPC, use `aws:SourceVpc` with the `StringEquals` condition operator.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Principal": {
        "AWS": "*"
      },
      "Resource": "*",
      "Action": [
        "dsql:DbConnect",
        "dsql:DbConnectAdmin"
      ],
      "Condition": {
        "Null": {
          "aws:SourceVpc": "true"
        }
      }
    }
  ]
}
```

Note

This example uses only `aws:SourceVpc` to check for VPC connections. The `aws:VpcSourceIp` and `aws:SourceVpce` condition keys provide additional granularity but are not required for basic VPC-only access control.

To provide an exception for specific roles, use this policy instead:

```
{
  "Version": "2012-10-17",
  "Statement": [
```

```
{
  "Sid": "DenyAccessFromOutsideVPC",
  "Effect": "Deny",
  "Principal": {
    "AWS": "*"
  },
  "Resource": "*",
  "Action": [
    "dsql:DbConnect",
    "dsql:DbConnectAdmin"
  ],
  "Condition": {
    "Null": {
      "aws:SourceVpc": "true"
    },
    "StringNotEquals": {
      "aws:PrincipalArn": [
        "arn:aws:iam::123456789012:role/ExceptionRole",
        "arn:aws:iam::123456789012:role/AnotherExceptionRole"
      ]
    }
  }
}
```

Restrict access to AWS Organization

This policy restricts access to principals within an AWS Organization:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Principal": {
        "AWS": "*"
      },
      "Action": [
        "dsql:DbConnect",
        "dsql:DbConnectAdmin"
      ],
      "Resource": "arn:aws:dsql:us-east-1:123456789012:cluster:mycluster",
    }
  ]
}
```

```

    "Condition": {
      "StringNotEquals": {
        "aws:PrincipalOrgID": "o-exampleorgid"
      }
    }
  }
]
}

```

Restrict access to specific Organizational Unit

This policy restricts access to principals within a specific Organizational Unit (OU) in an AWS Organization, providing more granular control than organization-wide access:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Principal": {
        "AWS": "*"
      },
      "Action": [
        "dsql:DbConnect"
      ],
      "Resource": "arn:aws:dsql:us-east-1:123456789012:cluster:mycluster",
      "Condition": {
        "StringNotLike": {
          "aws:PrincipalOrgPaths": "o-exampleorgid/r-examplerootid/ou-exampleouid/*"
        }
      }
    }
  ]
}

```

Multi-Region cluster policies

For multi-Region clusters, each regional cluster maintains its own resource policy, allowing for Region-specific controls. Here's an example with different policies per region:

us-east-1 policy:

```

{

```

```

"Version": "2012-10-17",
"Statement": [
  {
    "Effect": "Deny",
    "Principal": {
      "AWS": "*"
    },
    "Resource": "*",
    "Action": [
      "dsql:DbConnect"
    ],
    "Condition": {
      "StringNotEquals": {
        "aws:SourceVpc": "vpc-east1-id"
      },
      "Null": {
        "aws:SourceVpc": "true"
      }
    }
  }
]
}

```

us-east-2 policy:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "*"
      },
      "Resource": "*",
      "Action": [
        "dsql:DbConnect"
      ],
      "Condition": {
        "StringEquals": {
          "aws:SourceVpc": "vpc-east2-id"
        }
      }
    }
  ]
}

```

```
]
}
```

Note

Condition context keys may vary between AWS Regions (such as VPC IDs).

Blocking public access with resource-based policies in Aurora DSQL

Block Public Access (BPA) is a feature that identifies and prevents the attaching of resource-based policies that grant public access to your Aurora DSQL clusters across your AWS accounts. With BPA, you can prevent public access to your Aurora DSQL resources. BPA performs checks during the creation or modification of a resource-based policy and helps improve your security posture with Aurora DSQL.

BPA uses [automated reasoning](#) to analyze the access granted by your resource-based policy and alerts you if such permissions are found at the time of administering a resource-based policy. The analysis verifies access across all resource-based policy statements, actions, and the set of condition keys used in your policies.

⚠ Important

BPA helps protect your resources by preventing public access from being granted through the resource-based policies that are directly attached to your Aurora DSQL resources, such as clusters. In addition to using BPA, carefully inspect the following policies to confirm that they do not grant public access:

- Identity-based policies attached to associated AWS principals (for example, IAM roles)
- Resource-based policies attached to associated AWS resources (for example, AWS Key Management Service (KMS) keys)

You must ensure that the [principal](#) doesn't include a * entry or that one of the specified condition keys restrict access from principals to the resource. If the resource-based policy grants public access to your cluster across AWS accounts, Aurora DSQL will block you from creating or modifying the policy until the specification within the policy is corrected and deemed non-public.

You can make a policy non-public by specifying one or more principals inside the `Principal` block. The following resource-based policy example blocks public access by specifying two principals.

```
{
  "Effect": "Allow",
  "Principal": {
    "AWS": [
      "123456789012",
      "111122223333"
    ]
  },
  "Action": "dsql:*",
  "Resource": "arn:aws:dsql:us-east-1:123456789012:cluster/cluster-id"
}
```

Policies that restrict access by specifying certain condition keys are also not considered public. Along with evaluation of the principal specified in the resource-based policy, the following [trusted condition keys](#) are used to complete the evaluation of a resource-based policy for non-public access:

- `aws:PrincipalAccount`
- `aws:PrincipalArn`
- `aws:PrincipalOrgID`
- `aws:PrincipalOrgPaths`
- `aws:SourceAccount`
- `aws:SourceArn`
- `aws:SourceVpc`
- `aws:SourceVpce`
- `aws:UserId`
- `aws:PrincipalServiceName`
- `aws:PrincipalServiceNamesList`
- `aws:PrincipalIsAWSService`
- `aws:Ec2InstanceSourceVpc`
- `aws:SourceOrgID`

- `aws:SourceOrgPaths`

Additionally, for a resource-based policy to be non-public, the values for Amazon Resource Name (ARN) and string keys must not contain wildcards or variables. If your resource-based policy uses the `aws:PrincipalIsAWSService` key, you must make sure that you've set the key value to true.

The following policy limits access to the user Ben in the specified account. The condition makes the Principal constrained and not be considered as public.

```
{
  "Effect": "Allow",
  "Principal": {
    "AWS": "*"
  },
  "Action": "dsql:*",
  "Resource": "arn:aws:dsql:us-east-1:123456789012:cluster/cluster-id",
  "Condition": {
    "StringEquals": {
      "aws:PrincipalArn": "arn:aws:iam::123456789012:user/Ben"
    }
  }
}
```

The following example of a non-public resource-based policy constrains sourceVPC using the `StringEquals` operator.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "*"
      },
      "Action": "dsql:*",
      "Resource": "arn:aws:dsql:us-east-1:123456789012:cluster/cluster-id",
      "Condition": {
        "StringEquals": {
          "aws:SourceVpc": [
            "vpc-91237329"
          ]
        }
      }
    }
  ]
}
```

```

    }
  }
}
]
}

```

Aurora DSQL API Operations and Resource-Based Policies

Resource-based policies in Aurora DSQL control access to specific API operations. The following sections list all Aurora DSQL API operations organized by category, with an indication of which ones support resource-based policies.

The *Supports RBP* column indicates whether the API operation is subject to resource-based policy evaluation when a policy is attached to the cluster.

Tag APIs

API Operation	Description	Supports RBP
ListTagsForResource	Lists the tags for a Aurora DSQL resource	Yes
TagResource	Adds tags to a Aurora DSQL resource	Yes
UntagResource	Removes tags from a Aurora DSQL resource	Yes

Cluster management APIs

API Operation	Description	Supports RBP
CreateCluster	Creates a new cluster	No
DeleteCluster	Deletes a cluster	Yes
GetCluster	Retrieves information about a cluster	Yes
GetVpcEndpointServiceName	Retrieves the VPC endpoint service name for a cluster	Yes
ListClusters	Lists clusters in your account	No

API Operation	Description	Supports RBP
UpdateCluster	Updates the configuration of a cluster	Yes

Multi-Region property APIs

API Operation	Description	Supports RBP
AddPeerCluster	Adds a peer cluster to a multi-region configuration	Yes
PutMultiRegionProperties	Sets multi-region properties for a cluster	Yes
PutWitnessRegion	Sets the witness region for a multi-region cluster	Yes

Resource-based policy APIs

API Operation	Description	Supports RBP
DeleteClusterPolicy	Deletes the resource-based policy from a cluster	Yes
GetClusterPolicy	Retrieves the resource-based policy for a cluster	Yes
PutClusterPolicy	Creates or updates the resource-based policy for a cluster	Yes

AWS Fault Injection Service APIs

API Operation	Description	Supports RBP
InjectError	Injects errors for fault injection testing	No

Backup and restore APIs

API Operation	Description	Supports RBP
GetBackupJob	Retrieves information about a backup job	No
GetRestoreJob	Retrieves information about a restore job	No
StartBackupJob	Starts a backup job for a cluster	Yes
StartRestoreJob	Starts a restore job from a backup	No
StopBackupJob	Stops a running backup job	No
StopRestoreJob	Stops a running restore job	No

Using service-linked roles in Aurora DSQL

Aurora DSQL uses AWS Identity and Access Management (IAM) [service-linked roles](#). A service-linked role is a unique type of IAM role that is linked directly to Aurora DSQL. Service-linked roles are predefined by Aurora DSQL and include all the permissions that the service requires to call AWS services on behalf of your Aurora DSQL cluster.

Service-linked roles make the setup process easier because you don't have to manually add the necessary permissions to use Aurora DSQL. When you create a cluster, Aurora DSQL automatically creates a service-linked role for you. You can delete the service-linked role only after you delete all of your clusters. This protects your Aurora DSQL resources because you can't inadvertently remove permissions needed for access to the resources.

For information about other services that support service-linked roles, see [AWS services that work with IAM](#) and look for the services that have **Yes** in the **Service-Linked Role** column. Choose a **Yes** with a link to view the service-linked role documentation for that service.

Service-linked roles are available in all supported Aurora DSQL Regions.

Service-linked role permissions for Aurora DSQL

Aurora DSQL uses the service-linked role named `AWSServiceRoleForAuroraDsql` – Allows Amazon Aurora DSQL to create and manage AWS resources on your behalf. This service-linked role is attached to the following managed policy: [AuroraDsqlServiceLinkedRolePolicy](#).

Note

You must configure permissions to allow an IAM entity (such as a user, group, or role) to create, edit, or delete a service-linked role. You might encounter the following error message: You don't have the permissions to create an Amazon Aurora DSQL service-linked role. If you see this message, make sure that you have the following permissions enabled:

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": ["dsql:CreateCluster"],
      "Resource": [
        "arn:aws:dsql:us-east-1:*:cluster/*",
        "arn:aws:dsql:us-east-2:*:cluster/*"
      ],
      "Effect": "Allow"
    }
  ]
}
```

For more information, see [Service-linked role permissions](#).

Create a service-linked role

You don't need to manually create an `AuroraDSQLServiceLinkedRolePolicy` service-linked role. Aurora DSQL creates the service-linked role for you. If the `AuroraDSQLServiceLinkedRolePolicy`

service-linked role has been deleted from your account, Aurora DSQL creates the role when you create a new Aurora DSQL cluster.

Edit a service-linked role

Aurora DSQL doesn't allow you to edit the `AuroraDSQLServiceLinkedRolePolicy` service-linked role. After you create a service-linked role, you can't change the name of the role because various entities might reference the role. However, you can edit the description of the role using the IAM console, the AWS Command Line Interface (AWS CLI), or IAM API.

Delete a service-linked role

If you no longer need to use a feature or service that requires a service-linked role, we recommend that you delete that role. That way, you don't have an unused entity that is not actively monitored or maintained.

Before you can delete a service-linked role for an account, you must delete any clusters in the account.

You can use the IAM console, the AWS CLI, or the IAM API to delete a service-linked role. For more information, see [Create a service-linked role](#) in the IAM User Guide.

Supported Regions for Aurora DSQL service-linked roles

Aurora DSQL supports using service-linked roles in all of the Regions where the service is available. For more information, see [AWS Regions and endpoints](#).

Using IAM condition keys with Amazon Aurora DSQL

When you grant permissions in Aurora DSQL you can specify conditions that determine how a permissions policy takes effect. The following are examples of how you can use condition keys in Aurora DSQL permissions policies.

Example 1: Grant permission to create a cluster in a specific AWS Region

The following policy grants permission to create clusters in the US East (N. Virginia) and US East (Ohio) Regions. This policy uses the resource ARN to limit the allowed Regions, so Aurora DSQL can only create clusters only if that ARN is specified in the Resource section of the policy.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": ["dsql:CreateCluster"],
      "Resource": [
        "arn:aws:dsql:us-east-1:*:cluster/*",
        "arn:aws:dsql:us-east-2:*:cluster/*"
      ],
      "Effect": "Allow"
    }
  ]
}
```

Example 2: Grant permission to create a multi-Region cluster in specific AWS Regions

The following policy grants permission to create multi-Region clusters in the US East (N. Virginia) and US East (Ohio) Regions. This policy uses the resource ARN to limit the allowed Regions, so Aurora DSQL can create multi-Region clusters only if this ARN is specified in the Resource section of the policy. Note that creating multi-Region clusters also requires the `PutMultiRegionProperties`, `PutWitnessRegion`, and `AddPeerCluster` permissions in each specified Region.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dsql:CreateCluster",
        "dsql:PutMultiRegionProperties",
        "dsql:PutWitnessRegion",
        "dsql:AddPeerCluster"
      ],

```



```

    "Resource": [
      "arn:aws:dsql:us-east-1:123456789012:cluster/*",
      "arn:aws:dsql:us-east-2:123456789012:cluster/*"
    ]
  }
]
}

```

Example 3: Grant permission to create a multi-Region cluster with a specific witness Region

The following policy uses an Aurora DSQL `dsql:WitnessRegion` condition key and lets a user create multi-Region clusters with a witness Region in US West (Oregon). If you don't specify the `dsql:WitnessRegion` condition, you can use any Region as the witness Region.

JSON

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dsql:CreateCluster",
        "dsql:PutMultiRegionProperties",
        "dsql:AddPeerCluster"
      ],
      "Resource": "arn:aws:dsql:*:123456789012:cluster/*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "dsql:PutWitnessRegion"
      ],
      "Resource": "arn:aws:dsql:*:123456789012:cluster/*",
      "Condition": {
        "StringEquals": {
          "dsql:WitnessRegion": [
            "us-west-2"
          ]
        }
      }
    }
  ]
}

```

```
}  
}  
}  
]  
}
```

Incident response in Amazon Aurora DSQL

Security is the highest priority at AWS. As part of the AWS Cloud shared responsibility model, AWS manages a data center, network, and software architecture that meets the requirements of the most security-sensitive organizations. AWS is responsible for any incident response with respect to the Amazon Aurora DSQL service itself. Also, as an AWS customer, you share a responsibility for maintaining security in the cloud. This means that you control the security you choose to implement from the AWS tools and features you have access to. In addition, you're responsible for incident response on your side of the shared responsibility model.

By establishing a security baseline that meets the objectives for your applications running in the cloud, you're able to detect deviations that you can respond to. To help you understand the impact that incident response and your choices have on your corporate goals, we encourage you to review the following resources:

- [AWS Security Incident Response Guide](#)
- [AWS Best Practices for Security, Identity, and Compliance](#)
- [Security Perspective of the AWS Cloud Adoption Framework \(CAF\) whitepaper](#)

[Amazon GuardDuty](#) is a managed threat detection service continuously monitoring malicious or unauthorized behavior to help customers protect AWS accounts and workloads and identify suspicious activity potentially before it escalates into an incident. It monitors activity such as unusual API calls or potentially unauthorized deployments indicating possible account or resource compromise or reconnaissance by bad actors. For example, Amazon GuardDuty is able to detect suspicious activity in Amazon Aurora DSQL APIs, such as a user logging in from a new location and creating a new cluster.

Compliance validation for Amazon Aurora DSQL

To learn whether an AWS service is within the scope of specific compliance programs, see [AWS services in Scope by Compliance Program](#) and choose the compliance program that you are interested in. For general information, see [AWS Compliance Programs](#).

You can download third-party audit reports using AWS Artifact. For more information, see [Downloading Reports in AWS Artifact](#).

Your compliance responsibility when using AWS services is determined by the sensitivity of your data, your company's compliance objectives, and applicable laws and regulations. For more information about your compliance responsibility when using AWS services, see [AWS Security Documentation](#).

Resilience in Amazon Aurora DSQL

The AWS global infrastructure is built around AWS Regions and Availability Zones (AZ). AWS Regions provide multiple physically separated and isolated Availability Zones, which are connected with low-latency, high-throughput, and highly redundant networking. With Availability Zones, you can design and operate applications and databases that automatically fail over between zones without interruption. Availability Zones are more highly available, fault tolerant, and scalable than traditional single or multiple data center infrastructures. Aurora DSQL is designed so that you can take advantage of AWS Regional infrastructure while providing the highest database availability. By default, single-Region clusters in Aurora DSQL have Multi-AZ availability, providing tolerance to major component failures and infrastructure disruptions that might impact access to a full AZ. Multi-Region clusters provide all of the benefits from Multi-AZ resiliency while still providing the strongly consistent database availability, even in cases in which AWS Region is inaccessible to application clients.

For more information about AWS Regions and Availability Zones, see [AWS Global Infrastructure](#).

In addition to the AWS global infrastructure, Aurora DSQL offers several features to help support your data resiliency and backup needs.

Backup and restore

Aurora DSQL supports backup and restore with AWS Backup console. You can perform a full backup and restore for your single-Region and multi-Region clusters. For more information, see [Backup and restore for Amazon Aurora DSQL](#).

Replication

By design, Aurora DSQL commits all write transactions to a distributed transaction log and synchronously replicates all committed log data to user storage replicas in three AZs. Multi-Region clusters provide full cross-Region replication capabilities between read and write Regions.

A designated witness Region supports transaction log-only writes and doesn't consume storage. Witness Regions don't have an endpoint. This means that witness Regions store only encrypted transaction logs, require no administration or configuration, and aren't accessible by users.

Aurora DSQL transaction logs and user storage are distributed with all data presented to Aurora DSQL query processors as a single logical volume. Aurora DSQL automatically splits, merges, and replicates data based on database primary key range and access patterns. Aurora DSQL automatically scales read replicas, both up and down, based on read access frequency.

Cluster storage replicas are distributed across a multi-tenant storage fleet. If a component or AZ becomes impaired, Aurora DSQL automatically redirects access to surviving components and asynchronously repairs missing replicas. Once Aurora DSQL fixes the impaired replicas, Aurora DSQL automatically adds them back to the storage quorum and makes them available to your cluster.

High availability

By default, single-Region and multi-Region clusters in Aurora DSQL are active-active, and you don't need to manually provision, configure, or reconfigure any clusters. Aurora DSQL fully automates cluster recovery, which eliminates the need for traditional primary-secondary failover operations. Replication is always synchronous and done in multiple AZs, so there is no risk of data loss due to replication lag or failover to an asynchronous secondary database during failure recovery.

Single-Region clusters provide a Multi-AZ redundant endpoint that automatically enables concurrent access with strong data consistency across three AZs. This means that user storage replicas on any of these three AZs always return the same result to one or more readers and are always available to receive writes. This strong consistency and Multi-AZ resiliency is available across all Regions for Aurora DSQL multi-Region clusters. This means that multi-Region clusters provide two strongly consistent Regional endpoints, so clients can read or write indiscriminately to either Region with zero replication lag on commit.

Aurora DSQL provides 99.99% availability for single-Region clusters and 99.999% for multi-Region clusters.

Fault injection testing

Amazon Aurora DSQL integrates with AWS Fault Injection Service (AWS FIS), a fully managed service for running controlled fault injection experiments to improve an application's resilience. Using AWS FIS, you can:

- Create experiment templates that define specific failure scenarios
- Inject failures (elevated cluster connection error rates) to validate application error handling and recovery mechanisms
- Test multi-Region application behavior to validate application traffic shift between AWS Regions when one AWS Region is experiencing high connection error rates

For example, in a multi-Region cluster spanning US East (N. Virginia) and US East (Ohio), you can run an experiment in US East (Ohio) to test failures there while US East (N. Virginia) continues normal operations. This controlled testing helps you identify and resolve potential issues before they affect production workloads.

See [Action targets](#) in the *AWS FIS user guide* for a complete list of AWS FIS supported actions.

For information about Amazon Aurora DSQL actions available in AWS FIS, see [Aurora DSQL actions reference](#) in the *AWS FIS User Guide*.

To get started running fault injection experiments, see [Planning your AWS FIS experiments](#) in the *AWS FIS User Guide*.

Infrastructure Security in Amazon Aurora DSQL

As a managed service, Amazon Aurora DSQL is protected by the AWS global network security procedures that are described in [Best Practices for Security, Identity, & Compliance](#).

You use AWS published API calls to access Aurora DSQL through the network. Clients must support Transport Layer Security (TLS) 1.2 or later. Clients must also support cipher suites with perfect forward secrecy (PFS) such as DHE (Ephemeral Diffie-Hellman) or ECDHE (Elliptic Curve Ephemeral Diffie-Hellman). Most modern systems such as Java 7 and later support these modes.

Additionally, requests must be signed by using an access key ID and a secret access key that is associated with an IAM principal. Or you can use the [AWS Security Token Service](#) (AWS STS) to generate temporary security credentials to sign requests.

Managing and connecting to Amazon Aurora DSQL clusters using AWS PrivateLink

With AWS PrivateLink for Amazon Aurora DSQL, you can provision interface Amazon VPC endpoints (interface endpoints) in your Amazon Virtual Private Cloud. These endpoints are directly accessible from applications that are on premises over Amazon VPC and AWS Direct Connect, or in a different AWS Region over Amazon VPC peering. Using AWS PrivateLink and interface endpoints, you can simplify private network connectivity from your applications to Aurora DSQL.

Applications within your Amazon VPC can access Aurora DSQL using Amazon VPC interface endpoints without requiring public IP addresses.

Interface endpoints are represented by one or more elastic network interfaces (ENIs) that are assigned private IP addresses from subnets in your Amazon VPC. Requests to Aurora DSQL over interface endpoints stay on the AWS network. For more information about how to connect your Amazon VPC with your on-premises network, see the [AWS Direct Connect User Guide](#) and the [AWS Site-to-Site VPN VPN](#) User Guide.

For general information about interface endpoints, see [Access an AWS service using an interface Amazon VPC endpoint](#) in the [AWS PrivateLink](#) User Guide.

Types of Amazon VPC endpoints for Aurora DSQL

Aurora DSQL requires two different types of AWS PrivateLink endpoints.

1. *Management endpoint*— This endpoint is used for administrative operations, such as get, create, update, delete, and list on Aurora DSQL clusters. See [Managing Aurora DSQL clusters using AWS PrivateLink](#).
2. *Connection endpoint*— This endpoint is used for connecting to Aurora DSQL clusters through PostgreSQL clients. See [Connecting to Aurora DSQL clusters using AWS PrivateLink](#).

Considerations when using AWS PrivateLink for Aurora DSQL

Amazon VPC considerations apply to AWS PrivateLink for Aurora DSQL. For more information, see [Access an AWS service using an interface VPC endpoint](#) and [AWS PrivateLink quotas](#) in the AWS PrivateLink Guide.

Managing Aurora DSQL clusters using AWS PrivateLink

You can use the AWS Command Line Interface or AWS Software Development Kits (SDKs) to manage Aurora DSQL clusters through Aurora DSQL interface endpoints.

Creating an Amazon VPC endpoint

To create an Amazon VPC interface endpoint, see [Create an Amazon VPC endpoint](#) in the AWS PrivateLink Guide.

```
aws ec2 create-vpc-endpoint \  
--region region \  
--service-name com.amazonaws.region.dsql \  
--vpc-id your-vpc-id \  
--subnet-ids your-subnet-id \  
--vpc-endpoint-type Interface \  
--security-group-ids client-sg-id \  

```

To use the default Regional DNS name for Aurora DSQL API requests, do not disable private DNS when you create the Aurora DSQL interface endpoint. When private DNS is enabled, requests to the Aurora DSQL service made from within your Amazon VPC will automatically resolve to the private IP address of the Amazon VPC endpoint, rather than the public DNS name. When private DNS is enabled, Aurora DSQL requests made within your Amazon VPC will automatically resolve to your Amazon VPC endpoint.

If private DNS is not enabled, use the `--region` and `--endpoint-url` parameters with AWS CLI commands to manage Aurora DSQL clusters through Aurora DSQL interface endpoints.

Listing clusters using an endpoint URL

In the following example, replace the AWS Region `us-east-1` and the DNS name of the Amazon VPC endpoint ID `vpce-1a2b3c4d-5e6f.dsql.us-east-1.vpce.amazonaws.com` with your own information.

```
aws dsql --region us-east-1 --endpoint-url https://vpce-1a2b3c4d-5e6f.dsql.us-east-1.vpce.amazonaws.com list-clusters
```

API Operations

Refer to the [Aurora DSQL API reference](#) for documentation on managing resources in Aurora DSQL.

Managing endpoint policies

By thoroughly testing and configuring the Amazon VPC endpoint policies, you can help ensure that your Aurora DSQL cluster is secure, compliant, and aligned with your organization's specific access control and governance requirements.

Example: Full Aurora DSQL access policy

The following policy grants full access to all Aurora DSQL actions and resources through the specified Amazon VPC endpoint.

```
aws ec2 modify-vpc-endpoint \  
  --vpc-endpoint-id vpce-xxxxxxxxxxxxxxxxx \  
  --region region \  
  --policy-document '{  
    "Version": "2012-10-17",  
    "Statement": [  
      {  
        "Effect": "Allow",  
        "Principal": "*",  
        "Action": "dsql:*",  
        "Resource": "*"   
      }  
    ]  
  }'
```

Example: Restricted Aurora DSQL Access Policy

The following policy only permits these Aurora DSQL actions.

- CreateCluster
- GetCluster
- ListClusters

All other Aurora DSQL actions are denied.

JSON

```
{  
  "Version": "2012-10-17",
```



```
"Statement": [  
  {  
    "Effect": "Allow",  
    "Principal": "*",  
    "Action": [  
      "dsql:CreateCluster",  
      "dsql:GetCluster",  
      "dsql:ListClusters"  
    ],  
    "Resource": "*"   
  }  
]
```

Connecting to Aurora DSQL clusters using AWS PrivateLink

Once your AWS PrivateLink endpoint is set up and active, you can connect to your Aurora DSQL cluster using a PostgreSQL client. The connection instructions below outline the steps to construct the proper hostname for connecting through the AWS PrivateLink endpoint.

Setting up an AWS PrivateLink connection endpoint

Step 1: Get the service name for your cluster

When creating an AWS PrivateLink endpoint for connecting to your cluster, you first need to fetch the cluster-specific service name.

AWS CLI

```
aws dsql get-vpc-endpoint-service-name \  
--region us-east-1 \  
--identifier your-cluster-id
```

Example response

```
{  
  "serviceName": "com.amazonaws.us-east-1.dsql-fnh4"  
}
```

The service name includes an identifier, such as `dsql-fnh4` in the example. This identifier is also needed when constructing the hostname for connecting to your cluster.

AWS SDK for Python (Boto3)

```
import boto3

dsql_client = boto3.client('dsql', region_name='us-east-1')
response = dsql_client.get_vpc_endpoint_service_name(
    identifier='your-cluster-id'
)
service_name = response['serviceName']
print(f"Service Name: {service_name}")
```

AWS SDK for Java 2.x

```
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dsql.DsqlClient;
import software.amazon.awssdk.services.dsql.model.GetVpcEndpointServiceNameRequest;
import software.amazon.awssdk.services.dsql.model.GetVpcEndpointServiceNameResponse;

String region = "us-east-1";
String clusterId = "your-cluster-id";

DsqlClient dsqlClient = DsqlClient.builder()
    .region(Region.of(region))
    .credentialsProvider(DefaultCredentialsProvider.create())
    .build();

GetVpcEndpointServiceNameResponse response = dsqlClient.getVpcEndpointServiceName(
    GetVpcEndpointServiceNameRequest.builder()
        .identifier(clusterId)
        .build()
);
String serviceName = response.serviceName();
System.out.println("Service Name: " + serviceName);
```

Step 2: Create the Amazon VPC endpoint

Using the service name obtained in the previous step, create an Amazon VPC endpoint.

Important

The connection instructions below only work for connecting to clusters when private is DNS enabled. Do not use the `--no-private-dns-enabled` flag when creating the endpoint, as this will prevent the connection instructions below from working properly. If you disable private DNS, you will need to create your own wildcard private DNS record that points to the created endpoint.

AWS CLI

```
aws ec2 create-vpc-endpoint \  
  --region us-east-1 \  
  --service-name service-name-for-your-cluster \  
  --vpc-id your-vpc-id \  
  --subnet-ids subnet-id-1 subnet-id-2 \  
  --vpc-endpoint-type Interface \  
  --security-group-ids security-group-id
```

Example response

```
{  
  "VpcEndpoint": {  
    "VpcEndpointId": "vpce-0123456789abcdef0",  
    "VpcEndpointType": "Interface",  
    "VpcId": "vpc-0123456789abcdef0",  
    "ServiceName": "com.amazonaws.us-east-1.dsql-fnh4",  
    "State": "pending",  
    "RouteTableIds": [],  
    "SubnetIds": [  
      "subnet-0123456789abcdef0",  
      "subnet-0123456789abcdef1"  
    ],  
    "Groups": [  
      {  
        "GroupId": "sg-0123456789abcdef0",  
        "GroupName": "default"  
      }  
    ],  
    "PrivateDnsEnabled": true,  
    "RequesterManaged": false,  
  }
```

```

    "NetworkInterfaceIds": [
        "eni-0123456789abcdef0",
        "eni-0123456789abcdef1"
    ],
    "DnsEntries": [
        {
            "DnsName": "*.dsql-fnh4.us-east-1.vpce.amazonaws.com",
            "HostedZoneId": "Z7HUB22UULQXV"
        }
    ],
    "CreationTimestamp": "2025-01-01T00:00:00.000Z"
}

```

SDK for Python

```

import boto3

ec2_client = boto3.client('ec2', region_name='us-east-1')
response = ec2_client.create_vpc_endpoint(
    VpcEndpointType='Interface',
    VpcId='your-vpc-id',
    ServiceName='com.amazonaws.us-east-1.dsql-fnh4', # Use the service name from
previous step
    SubnetIds=[
        'subnet-id-1',
        'subnet-id-2'
    ],
    SecurityGroupIds=[
        'security-group-id'
    ]
)

vpc_endpoint_id = response['VpcEndpoint']['VpcEndpointId']
print(f"VPC Endpoint created with ID: {vpc_endpoint_id}")

```

SDK for Java 2.x

Use an endpoint URL for Aurora DSQL APIs

```

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;

```

```

import software.amazon.awssdk.services.ec2.model.CreateVpcEndpointRequest;
import software.amazon.awssdk.services.ec2.model.CreateVpcEndpointResponse;
import software.amazon.awssdk.services.ec2.model.VpcEndpointType;

String region = "us-east-1";
String serviceName = "com.amazonaws.us-east-1.dsdl-fnh4"; // Use the service name
                    from previous step
String vpcId = "your-vpc-id";

Ec2Client ec2Client = Ec2Client.builder()
    .region(Region.of(region))
    .credentialsProvider(DefaultCredentialsProvider.create())
    .build();

CreateVpcEndpointRequest request = CreateVpcEndpointRequest.builder()
    .vpcId(vpcId)
    .serviceName(serviceName)
    .vpcEndpointType(VpcEndpointType.INTERFACE)
    .subnetIds("subnet-id-1", "subnet-id-2")
    .securityGroupIds("security-group-id")
    .build();

CreateVpcEndpointResponse response = ec2Client.createVpcEndpoint(request);
String vpcEndpointId = response.vpcEndpoint().vpcEndpointId();
System.out.println("VPC Endpoint created with ID: " + vpcEndpointId);

```

Connecting to an Aurora DSQL cluster using an AWS PrivateLink connection endpoint

Once your AWS PrivateLink endpoint is set up and active (check that the State is available), you can connect to your Aurora DSQL cluster using a PostgreSQL client. For instructions on using the AWS SDKs, you can follow the guides in [Programming with Aurora DSQL](#). You must change the cluster endpoint to match the hostname format.

Constructing the hostname

The hostname for connecting through AWS PrivateLink differs from the public DNS hostname. You need to construct it using the following components.

1. Your-cluster-id
2. The service identifier from the service name. For example: dsdl-fnh4
3. The AWS Region

Use the following format: *cluster-id.service-identifier.region*.on.aws

Example: Connection Using PostgreSQL

```
# Set environment variables
export CLUSTERID=your-cluster-id
export REGION=us-east-1
export SERVICE_IDENTIFIER=dsql-fnh4 # This should match the identifier in your service
name

# Construct the hostname
export HOSTNAME="$CLUSTERID.$SERVICE_IDENTIFIER.$REGION.on.aws"

# Generate authentication token
export PGPASSWORD=$(aws dsq1 --region $REGION generate-db-connect-admin-auth-token --
hostname $HOSTNAME)

# Connect using psql
psql -d postgres -h $HOSTNAME -U admin
```

Troubleshooting issues with AWS PrivateLink

Common Issues and Solutions

The following table lists common issues and solutions relating to AWS PrivateLink with Aurora DSQL.

Issue	Possible Cause	Solution
Connection timeout	Security group not properly configured	Use Amazon VPC Reachability Analyzer to ensure your networking setup allows traffic on port 5432.
DNS resolution failure	Private DNS not enabled	Verify that the Amazon VPC endpoint was created with private DNS enabled.
Authentication failure	Incorrect credentials or expired token	Generate a new authentication token and verify the user name.
Service name not found	Incorrect cluster ID	Double-check your cluster ID and AWS Region when fetching the service name.

Related Resources

For more information, see the following resources:

- [Amazon Aurora DSQL User Guide](#)
- [AWS PrivateLink Documentation](#)
- [Access AWS services through AWS PrivateLink](#)

Configuration and vulnerability analysis in Amazon Aurora DSQL

AWS handles basic security tasks like guest operating system (OS) and database patching, firewall configuration, and disaster recovery. These procedures have been reviewed and certified by the appropriate third parties. For more details, see the following resources:

- [Shared responsibility model](#)
- [Amazon Web Services: Overview of security processes \(whitepaper\)](#)

Cross-service confused deputy prevention

The confused deputy problem is a security issue where an entity that doesn't have permission to perform an action can coerce a more-privileged entity to perform the action. In AWS, cross-service impersonation can result in the confused deputy problem. Cross-service impersonation can occur when one service (the *calling service*) calls another service (the *called service*). The calling service can be manipulated to use its permissions to act on another customer's resources in a way it should not otherwise have permission to access. To prevent this, AWS provides tools that help you protect your data for all services with service principals that have been given access to resources in your account.

We recommend using the [aws:SourceArn](#) and [aws:SourceAccount](#) global condition context keys in resource policies to limit the permissions that Amazon Aurora DSQL gives another service to the resource. Use `aws:SourceArn` if you want only one resource to be associated with the cross-service access. Use `aws:SourceAccount` if you want to allow any resource in that account to be associated with the cross-service use.

The most effective way to protect against the confused deputy problem is to use the `aws:SourceArn` global condition context key with the full ARN of the resource. If you don't know

the full ARN of the resource or if you are specifying multiple resources, use the `aws:SourceArn` global context condition key with wildcard characters (*) for the unknown portions of the ARN. For example, `arn:aws:dsql:*:123456789012:*`.

If the `aws:SourceArn` value does not contain the account ID, such as an Amazon S3 bucket ARN, you must use both global condition context keys to limit permissions.

The value of `aws:SourceArn` must be `ResourceDescription`.

The following example shows how you can use the `aws:SourceArn` and `aws:SourceAccount` global condition context keys in Aurora DSQL to prevent the confused deputy problem.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": {
    "Sid": "ConfusedDeputyPreventionExamplePolicy",
    "Effect": "Allow",
    "Principal": {
      "Service": "backup.amazonaws.com"
    },
    "Action": "dsql:GetCluster",
    "Resource": [
      "arn:aws:dsql:*:123456789012:cluster/*"
    ],
    "Condition": {
      "ArnLike": {
        "aws:SourceArn": "arn:aws:backup:*:123456789012:*"
      },
      "StringEquals": {
        "aws:SourceAccount": "123456789012"
      }
    }
  }
}
```


Security best practices for Aurora DSQL

Aurora DSQL provides a number of security features to consider as you develop and implement your own security policies. The following best practices are general guidelines and don't represent a complete security solution. Because these best practices might not be appropriate or sufficient for your environment, treat them as helpful considerations rather than prescriptions.

Topics

- [Detective security best practices for Aurora DSQL](#)
- [Preventative security best practices for Aurora DSQL](#)

Detective security best practices for Aurora DSQL

In addition to the following ways to securely use Aurora DSQL, see [Security](#) in AWS Well-Architected Tool to learn about how cloud technologies improve your security.

Amazon CloudWatch Alarms

Using Amazon CloudWatch alarms, you watch a single metric over a time period that you specify. If the metric exceeds a given threshold, a notification is sent to an Amazon SNS topic or AWS Auto Scaling policy. CloudWatch alarms do not invoke actions because they are in a particular state. Rather the state must have changed and been maintained for a specified number of periods.

Tag your Aurora DSQL resources for identification and automation

You can assign metadata to your AWS resources in the form of tags. Each tag is a simple label consisting of a customer-defined key and an optional value that can make it easier to manage, search for, and filter resources.

Tagging allows for grouped controls to be implemented. Although there are no inherent types of tags, they enable you to categorize resources by purpose, owner, environment, or other criteria. The following are some examples:

- Security – Used to determine requirements such as encryption.
- Confidentiality – An identifier for the specific data-confidentiality level a resource supports.
- Environment – Used to distinguish between development, test, and production infrastructure.

You can assign metadata to your AWS resources in the form of tags. Each tag is a simple label consisting of a customer-defined key and an optional value that can make it easier to manage, search for, and filter resources.

Tagging allows for grouped controls to be implemented. Although there are no inherent types of tags, they let you categorize resources by purpose, owner, environment, or other criteria. The following are some examples.

- Security – used to determine requirements such as encryption.
- Confidentiality – an identifier for the specific data-confidentiality level a resource supports.
- Environment – used to distinguish between development, test, and production infrastructure.

For more information, see [Best Practices for Tagging AWS Resources](#).

Preventative security best practices for Aurora DSQL

In addition to the following ways to securely use Aurora DSQL, see [Security](#) in AWS Well-Architected Tool to learn about how cloud technologies improve your security.

Use IAM roles to authenticate access to Aurora DSQL.

Users, applications, and other AWS services that access Aurora DSQL must include valid AWS credentials in AWS API and AWS CLI requests. You shouldn't store AWS credentials directly in the application or EC2 instances. These are long-term credentials that aren't automatically rotated. There is significant business impact if these credentials are compromised. An IAM role lets you obtain temporary access keys that you can use to access AWS services and resources.

For more information, see [Authentication and authorization for Aurora DSQL](#).

Use IAM policies for Aurora DSQL base authorization.

When you grant permissions, you decide who is getting them, which Aurora DSQL API operations they are getting permissions for, and the specific actions you want to allow on those resources. Implementing least privilege is key in reducing security risk and the impact that can result from errors or malicious intent.

Attach permissions policies to IAM roles and grant permissions to perform operations on Aurora DSQL resources. Also available are [permissions boundaries for IAM entities](#), which let you set the maximum permissions that an identity-based policy can grant to an IAM entity.

Similar to the [root user best practices for your AWS account](#), don't use the admin role in Aurora DSQL to perform everyday operations. Instead, we recommend that you create custom database roles to manage and connect to your cluster. For more information, see [Accessing Aurora DSQL](#) and [Understanding authentication and authorization for Aurora DSQL](#).

Use `verify-full` in production environments.

This setting verifies that the server certificate is signed by a trusted certificate authority and that the server hostname matches the certificate.

Update your PostgreSQL client

Regularly update your PostgreSQL client to the latest version to benefit from security improvements. We recommend using PostgreSQL version 17.

Tagging resources in Aurora DSQL

In AWS, tags are user-defined key-value pairs that you define and associate with Aurora DSQL resources such as clusters. Tags are optional. If you provide a key, the value is optional.

You can use the AWS Management Console, the AWS CLI, or the AWS SDKs to add, list, and delete tags on Aurora DSQL clusters. You can add tags during and after cluster creation using the AWS console. To tag a cluster after creation with the AWS CLI use the `TagResource` operation.

Tagging clusters with a Name

Aurora DSQL creates clusters with a globally unique identifier assigned as the Amazon Resource Name (ARN). If you want to assign a user friendly name to your cluster, we recommend that you use a Tag.

If you create a console with the Aurora DSQL console, Aurora DSQL automatically creates a tag. This tag has a key of **Name** and an automatically generated value that represents the name of the cluster. This value is configurable, so you can assign a more friendly name to your cluster. If a cluster has a Name tag with an associated value, you can see the value throughout the Aurora DSQL console.

Tagging requirements

Tags have the following requirements:

- Keys can't be prefixed with `aws :`.
- Keys must be unique per tag set.
- A key must be between 1 and 128 allowed characters.
- A value must be between 0 and 256 allowed characters.
- Values do not need to be unique per tag set.
- Allowed characters for keys and values are letters, digits, white space, and any of the following symbols: `_ . : / = + - @`.
- Keys and values are case sensitive.

Tagging usage notes

When using tags in Aurora DSQL, consider the following.

- When using the AWS CLI or Aurora DSQL API operations, make sure to provide the Amazon Resource Name (ARN) for the Aurora DSQL resource to work with. For more information, see [Amazon Resource Name \(ARNs\) format for Aurora DSQL resources](#).
- Each resource has one tag set, which is a collection of one or more tags assigned to the resource.
- Each resource can have up to 50 tags per tag set.
- If you delete a resource, any associated tags are deleted.
- You can add tags when you create a resource, you can view and modify tags using the following API operations: `TagResource`, `UntagResource`, and `ListTagsForResource`.
- You can use tags with IAM policies. You can use them to manage access to Aurora DSQL clusters and to control what actions can be applied to those resources. To learn more, see [Controlling access to AWS resources using tags](#).
- You can use tags for various other activities across AWS. To learn more, see [Common tagging strategies](#).

Considerations for working with Amazon Aurora DSQL

Consider the following behaviors when you work with Amazon Aurora DSQL. For more information about PostgreSQL compatibility and support, see [SQL feature compatibility in Aurora DSQL](#). For quotas and limits, see [Cluster quotas and database limits in Amazon Aurora DSQL](#).

- Aurora DSQL doesn't complete `COUNT( *)` operations before transaction timeout for large tables. To retrieve table row count from the system catalog, see [Using systems tables and commands in Aurora DSQL](#).
- Drivers calling `PG_PREPARED_STATEMENTS` might provide an inconsistent view of cached prepared statements for the cluster. You might see more than the expected number of prepared statements per connection for the same cluster and IAM role. Aurora DSQL doesn't preserve statement names that you prepare.
- In rare multi-Region linked-cluster impairment scenarios, it might take longer than expected for transaction commit availability to resume. In general, automated cluster recovery operations can result in transient concurrency control or connection errors. In most cases, you will only see the effects for a percentage of your workload. When you see these transit errors, retry your transaction or reconnect with your client.
- Some SQL clients, such as Datagrip, make expansive calls to system metadata to populate schema information. Aurora DSQL doesn't support all of this information and returns errors. This issue doesn't affect SQL query functionality, but it might affect schema display.
- The admin role has a set of permissions related to database management tasks. By default, these permissions don't extend to objects that other users create. The admin role can't grant or revoke permissions on these user-created objects to other users. The admin user can grant itself any other role to get the necessary permissions on these objects.

Cluster quotas and database limits in Amazon Aurora DSQL

The following sections describe the cluster quotas and database limits for Aurora DSQL.

Cluster quotas

Your AWS account has the following cluster quotas in Aurora DSQL. To request an increase to the service quotas for single-Region and multi-Region clusters within a specific AWS Region, use the [Service Quotas](#) console page. For other quota increases, contact AWS Support.

Description	Default limit	Configurable?	Aurora DSQL error code
Maximum single-Region clusters per AWS account	20 clusters	Yes	API error code ServiceQuotaExceededException
Maximum multi-Region clusters per AWS account	5 clusters	Yes	API error code ServiceQuotaExceededException
Maximum storage per cluster	10 TiB default limit, up to 128 TiB with approved limit increase.	Yes	DISK_FULL(53100)
Maximum connections per cluster	10,000 connections	Yes	T00_MANY_CONNECTIONS(53300)

Description	Default limit	Configurable?	Aurora DSQL error code
Maximum connection rate per cluster	100 connections per second	No	CONFIGURED_LIMIT_EXCEEDED(53400)
Maximum connection burst capacity per cluster	1,000 connections	No	No error code
Maximum concurrent restore jobs	4	No	No error code
Connection Refill Rate	100 connections per second	No	No error code

Database limits in Aurora DSQL

The following table describes the database limits in Aurora DSQL.

Description	Default limit	Configurable?	Aurora DSQL error code	Error message
Maximum combined size of the columns used in a primary key	1 KiB	No	54000	ERROR: key size too large
Maximum combined size	1 KiB	No	54000	ERROR: key size too large

Description	Default limit	Configurable?	Aurora DSQL error code	Error message
of the columns in a secondary index				
Maximum size of a row in a table	2 MiB	No	54000	ERROR: maximum row size exceeds
Maximum size of a column that's not part of an index	1 MiB	No	54000	ERROR: maximum column size exceeds
Maximum number of columns in a primary key or a secondary index	8	No	54011	ERROR: more than 8 column keys are not supported
Maximum number of columns in a table	255	No	54011	ERROR: tables can have at most
Maximum number of indexes in a table	24	No	54000	ERROR: more than 24 indexes per table are not allowed

Description	Default limit	Configurable?	Aurora DSQL error code	Error message
Maximum size of all data modified in a write transaction	10 MiB	No	54000	ERROR: transaction size limit DETAIL: Current transaction size is near the limit of 10mb
Maximum number of table and index rows that can be mutated in a transaction block	3,000 rows per transaction. See Aurora DSQL considerations for PostgreSQL compatibility .	No	54000	ERROR: transaction row limit
Maximum base amount of memory that a query operation can use	128 MiB per transaction	No	53200	ERROR: query requires too much memory out of memory.
Maximum number of schemas defined in a database	10	No	54000	ERROR: more than 10 schemas
Maximum number of tables in a database	1,000 tables	No	54000	ERROR: creating more than 1000 tables is not allowed

Description	Default limit	Configurable?	Aurora DSQL error code	Error message
Maximum number of databases in a cluster	1	No	No error code	ERROR: unsupported statement
Maximum transaction time	5 minutes	No	54000	ERROR: transaction age limit exceeded
Maximum connection duration	60 minutes	No	No error code	No error message
Maximum number of views in a database	5,000	No	54000	ERROR: creating more than 500 allowed
Maximum view definition size	2 MiB	No	54000	ERROR: view definition too large

For data type limits specific to Aurora DSQL, see [Supported data types in Aurora DSQL](#).

Aurora DSQL API reference

In addition to the AWS Management Console and the AWS Command Line Interface (AWS CLI), Aurora DSQL also provides an API interface. You can use the API operations to manage your resources in Aurora DSQL.

For an alphabetical list of API operations, see [Actions](#).

For an alphabetical list of data types, see [Data types](#).

For a list of common query parameters, see [Common parameters](#).

For descriptions of the error codes, see [Common errors](#).

For more information about the AWS CLI, see AWS Command Line Interface reference for Aurora DSQL.

Troubleshooting issues in Aurora DSQL

Note

The following topics provide troubleshooting advice for errors and issues that you might encounter when using Aurora DSQL. If you find an issue that is not listed here, reach out to AWS support

Topics

- [Troubleshooting connection errors](#)
- [Troubleshooting authentication errors](#)
- [Troubleshooting authorization errors](#)
- [Troubleshooting SQL errors](#)
- [Troubleshooting OCC errors](#)
- [Troubleshooting SSL/TLS connections](#)

Troubleshooting connection errors

error: unrecognized SSL error code: 6

Cause: You might be using a `psql` version earlier than [version 14](#), which doesn't support Server Name Indication (SNI). The SNI is required when connecting to Aurora DSQL.

You can check your client version with `psql --version`.

error: NetworkUnreachable

A `NetworkUnreachable` error during connection attempts might indicate that your client doesn't support IPv6 connections, rather than signaling an actual network problem. This error commonly occurs on IPv4-only instances because of how PostgreSQL clients handle dual-stack connections. When a server supports dual-stack mode, these clients first resolve hostnames to both IPv4 and IPv6 addresses. They attempt an IPv4 connection first, then try IPv6 if the initial connection fails. If your system doesn't support IPv6, you'll see a general `NetworkUnreachable` error instead of a clear "IPv6 not supported" message.

Troubleshooting authentication errors

IAM authentication failed for user "..."

When you generate an Aurora DSQL IAM authentication token, the maximum duration you can set is 1 week. After one week, you can't authenticate with that token.

Additionally, Aurora DSQL rejects your connection request if your assumed role has expired. For example, if you try to connect with a temporary IAM role even if your authentication token hasn't expired, Aurora DSQL will reject the connection request.

To learn more about how IAM works with Aurora DSQL, see [Understanding authentication and authorization for Aurora DSQL](#) and [AWS Identity and Access Management in Aurora DSQL](#).

An error occurred (InvalidAccessKeyId) when calling the GetObject operation: The AWS Access Key ID you provided does not exist in our records

IAM rejected your request. For more information, see [Why requests are signed](#).

IAM role <role> does not exist

Aurora DSQL couldn't find your IAM role. For more information, see [IAM roles](#).

IAM role must look like an IAM ARN

See [IAM Identifiers - IAM ARNs](#) for more information.

Troubleshooting authorization errors

Role <role> not supported

Aurora DSQL doesn't support the GRANT operation. See [Supported subsets of PostgreSQL commands in Aurora DSQL](#).

Cannot establish trust with role <role>

Aurora DSQL doesn't support the GRANT operation. See [Supported subsets of PostgreSQL commands in Aurora DSQL](#).

Role <role> does not exist

Aurora DSQL couldn't find specified database user. See [Authorize custom database roles to connect to a cluster](#).

ERROR: permission denied to grant IAM trust with role <role>

To grant access to a database role, you must be connected to your cluster with the admin role. To learn more, see [Authorize database roles to use SQL in a database](#).

ERROR: role <role> must have the LOGIN attribute

Any database roles you create must have the LOGIN permission.

To address this error, make sure that you've created the PostgreSQL Role with the LOGIN permission. For more information, see [CREATE ROLE](#) and [ALTER ROLE](#) in the PostgreSQL documentation.

ERROR: role <role> cannot be dropped because some objects depend on it

Aurora DSQL returns an error if you drop a database role with an IAM relationship until you revoke the relationship using `AWS IAM REVOKE`. To learn more, see [Revoking authorization](#).

Troubleshooting SQL errors

Error: Not supported

Aurora DSQL doesn't support all PostgreSQL-based dialect. To learn about what is supported, see [Supported PostgreSQL features in Aurora DSQL](#).

Error: SELECT FOR UPDATE in a read-only transaction is a no-op

You are attempting an operation that isn't allowed in a read-only transaction. To learn more, see [Understanding concurrency control in Aurora DSQL](#).

Error: use CREATE INDEX ASYNC instead

To create an index on a table with existing rows, you must use the `CREATE INDEX ASYNC` command. To learn more, see [Creating indexes asynchronously in Aurora DSQL](#).

Troubleshooting OCC errors

OC000 "ERROR: mutation conflicts with another transaction, retry as needed"

OC001 “ERROR: schema has been updated by another transaction, retry as needed”

Your PostgreSQL session had a cached copy of the schema catalog. That cached copy was valid at the time was loaded. Let’s call the time T1 and the version V1.

Another transaction updates the catalog at time T2. Let’s call this V2.

When the original session attempts to read from storage at time T2 it’s still using catalog version V1. Aurora DSQL’s storage layer rejects the request because the latest catalog version at T2 is V2.

When you retry at time T3 from the original session, Aurora DSQL refreshes the catalog cache. The transaction at T3 is using catalog V2. Aurora DSQL will finish the transaction as long as no other catalog changes came through since time T2.

Troubleshooting SSL/TLS connections

SSL error: certificate verify failed

This error indicates that the client cannot verify the server's certificate. Ensure that:

1. The Amazon Root CA 1 certificate is properly installed. See [Configuring SSL/TLS certificates for Aurora DSQL connections](#) for instructions on how to validate and install this certificate.
2. The PGSSLROOTCERT environment variable points to the correct certificate file.
3. The certificate file has the correct permissions.

Unrecognized SSL error code: 6

This error occurs with PostgreSQL clients below version 14. Upgrade your PostgreSQL client to version 17 to resolve this issue.

SSL error: unregistered scheme (Windows)

This is a known issue with the Windows psql client when using system certificates. Use the downloaded certificate file method described in the [Connecting from Windows](#) instructions.

Document history for the Amazon Aurora DSQL User Guide

The following table describes the documentation releases for Aurora DSQL.

Change	Description	Date
Resource-based policy support for Amazon Aurora DSQL	Added resource-based policy (RBP) support with new permissions: PutClusterPolicy , GetClusterPolicy , and DeleteClusterPolicy . These permissions allow managing inline policies attached to Aurora DSQL clusters for fine-grained access control. Updated managed policies AmazonAuroraDSQLEntireAccess, AmazonAuroraDSQLReadOnlyAccess, and AmazonAuroraDSQLConsoleFullAccess to include RBP capabilities. For more information, see AWS managed policies for Amazon Aurora DSQL .	October 15, 2025
Aurora DSQL JDBC wrapper	Added documentation for the Aurora DSQL JDBC wrapper, a PgJDBC wrapper that integrates IAM Authentication for connecting Java applications to Amazon Aurora DSQL clusters. For more informati	September 2, 2025

on, see [Using Aurora DSQL JDBC wrapper](#).

[AWS managed policy updates for AWS FIS integration](#)

Updated AmazonAuroraDSQFullAccess and AmazonAuroraDSQConsoleFullAccess policies to support AWS Fault Injection Service integration with Aurora DSQL. This allows you to inject failures into single-Region and multi-Region Aurora DSQL clusters to test fault tolerance of your applications. For more on these policies, see [AWS managed policy updates](#).

August 19, 2025

[General availability \(GA\) of Amazon Aurora DSQL](#)

Amazon Aurora DSQL is now generally available with added support for CloudWatch monitoring, enhanced data protection features, and AWS Backup integration. For more information, see [Monitoring Aurora DSQL with CloudWatch](#), [Backup and restore for Amazon Aurora DSQL](#), and [Data encryption for Amazon Aurora DSQL](#).

May 27, 2025

[AmazonAuroraDSQLFu llAccess update](#)

Adds the capability to perform backup and restore operations for Aurora DSQL clusters, including starting, stopping, and monitoring jobs. It also adds the capability to use customer-managed KMS keys for cluster encryption. For more information, see [AmazonAuroraDSQLFu
llAccess](#) and [Using service-l
inked roles in Aurora DSQL](#).

May 21, 2025

[AmazonAuroraDSQLCo nsoleFullAccess update](#)

Adds the capability to perform backup and restore operations for Aurora DSQL clusters through the AWS Console Home. This includes starting, stopping, and monitoring jobs. It also supports using customer-managed KMS keys for cluster encryption and launching AWS CloudShell. For more information, see [AmazonAur
oraDSQLConsoleFullAccess](#) and [Using service-linked roles
in Aurora DSQL](#).

May 21, 2025

[AmazonAuroraDSQLRe adOnlyAccess update](#)

May 13, 2025

Includes the ability to determine the correct VPC endpoint service name when connecting to your Aurora DSQL clusters through AWS PrivateLink Aurora DSQL creates unique endpoints per cell, so this API helps ensure you can identify the correct endpoint for your cluster and avoid connection errors. For more information, see [AmazonAuroraDSQLRe adOnlyAccess](#) and [Using service-linked roles in Aurora DSQL](#).

[AmazonAuroraDSQLFullAccess update](#)

May 13, 2025

The policy adds four new permissions to create and manage database clusters across multiple AWS Regions: `PutMultiRegionProperties`, `PutWitnessRegion`, `AddPeerCluster`, and `RemovePeerCluster`. These permissions include resource-level controls and condition keys so you can control which clusters users you can modify. The policy also adds the `GetVpcEndpointServiceName` permission to help you connect to your Aurora DSQL clusters through AWS PrivateLink. For more information, see [AmazonAuroraDSQLConsoleFullAccess](#) and [Using service-linked roles in Aurora DSQL](#).

[AmazonAuroraDSQLConsoleFullAccess update](#)

Adds new permissions to Aurora DSQL to support multi-Region cluster management and VPC endpoint connection. The new permissions include: PutMultiRegionProperties PutWitnessRegion AddPeerCluster RemovePeerCluster GetVpcEndpointServiceName . See [AmazonAuroraDSQLConsoleFullAccess](#) and [Using service-linked roles in Aurora DSQL](#) .

May 13, 2025

[AuroraDsqlServiceLinkedRolePolicy update](#)

Adds the ability to publish metrics to the AWS/AuroraDSQL and AWS/Usage CloudWatch namespaces to the policy. This allows the associated service or role to emit more comprehensive usage and performance data to your CloudWatch environment. For more information, see [AuroraDsqlServiceLinkedRolePolicy](#) and [Using service-linked roles in Aurora DSQL](#).

May 8, 2025

[AWS PrivateLink for Amazon Aurora DSQL](#)

Aurora DSQL now supports AWS PrivateLink. With AWS PrivateLink, you can simplify private network connectivity between virtual private clouds (VPCs), Aurora DSQL, and your on-premises data centers using interface Amazon VPC endpoints and private IP addresses. For more information, see [Managing and connecting to Amazon Aurora DSQL clusters using AWS PrivateLink](#).

May 8, 2025

[Initial release](#)

Initial release of the Amazon Aurora DSQL User Guide.

December 3, 2024